

SQL Basics

Student
Handbook

© 2022



All rights reserved.

Copyright Themis, a ProTech Company

Themis, Themis, Inc. and Themis Education Services are trademarks of Themis, Inc.

IBM and Db2 are registered trademarks of IBM and/or its affiliates. Other names may be trademarks of their respective owners.

Microsoft SQL Server 2008 is a registered trademark of Microsoft and/or its affiliates. Other names may be trademarks of their respective owners.

Oracle is a registered trademark of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

PostgreSQL is a registered trademark of PostgreSQL community.

Products and company names mentioned herein may be trademarks of their respective companies. Mention of third party products or software is for reference only and constitutes neither a recommendation nor an endorsement.

Course Description

Come prepared to master Structured Query Language (SQL) across multiple database platforms in this fast paced and hands-on course. The course begins with a discussion of relational concepts and terminology and quickly moves into a very practical discussion of data retrieval and manipulation using SQL. Each topic is reinforced with hands-on workshops.

Examples and workshops are provided for the following platforms with discussion of the relevant differences:

- ◆ Db2 for z/OS
- ◆ Db2 for Linux, Unix & Windows
- ◆ Oracle 10g & 11g
- ◆ Microsoft SQL Server
- ◆ PostgreSQL

Audience

Programmers, Analysts, Database Administrators, and IT Managers who are exposed to, or are required to perform the activities associated with data stored in a relational database such as Db2, Oracle, SQL Server, or PostgreSQL.

Prerequisites

There are no prerequisites for this course.

Table of Contents

CHAPTER 1 RELATIONAL DATABASE CONCEPTS.....	1-1
Overview.....	1-3
Objectives	1-3
Data Management.....	1-5
Database Management.....	1-7
Database Management System	1-9
Relational Catalog	1-9
Buffers	1-9
Locking Mechanisms.....	1-9
Transaction Logs	1-9
The Relational Model of Data	1-11
Relational Terminology	1-13
Structure	1-15
Normalization	1-17
Domains.....	1-19
Relational Structure: A Logical Progression	1-21
Relationships.....	1-23
Integrity and Relationships	1-24
One-to-Many Relationships	1-25
One-to-One Relationships	1-27
Many-to-Many Relationships.....	1-29
Relational Database Vendors.....	1-31
Structured Query Language (SQL)	1-33
Data Definition Language (DDL)	1-33
Data Control Language (DCL)	1-33
Data Manipulation Language (DML)	1-33
Creating Tables.....	1-35
Manipulation	1-37
Restrict.....	1-39
Project	1-41
Combined Operations.....	1-43
Join	1-44
Relational Database Components	1-47
The Optimizer	1-49
CHAPTER 2 BASIC DATA MANIPULATION	2-1
Overview.....	2-3
Objectives	2-3

CREATE TABLE Overview	2-5
Numeric Data Types	2-6
Character Data Types	2-8
Date and Time Data Types	2-10
Null Attribute	2-15
Default Values	2-17
NULL Attribute and Default	2-19
Enforcing Business Rules	2-21
Table Check Constraints	2-21
Referential Integrity	2-22
Basic Form of SELECT	2-25
Using SELECT *	2-26
Projecting Columns onto the Result	2-27
Basic Predicates	2-29
Basic Predicates Examples	2-32
NOT Comparisons	2-33
Expressions in the SELECT Clause	2-35
Renaming Columns	2-39
Expressions in the WHERE Clause	2-41
Ordering Multiple Columns	3-31
CHAPTER 3 INTERMEDIATE DATA MANIPULATION	3-1
Overview	3-3
Objectives	3-3
Constants	3-5
System Values	3-6
Db2	3-7
Oracle	3-7
SQL Server	3-7
System Values - PostgreSQL	3-9
Compound Conditions	3-10
Boolean Logic	3-10
Three-Valued Logic	3-13
Using Parentheses	3-14
Other WHERE Clause Operations	3-16
The IN Comparison Operator	3-17
BETWEEN	3-19
LIKE	3-21
IS NULL	3-23
CASE Expressions	3-24
Simple WHEN	3-25
Searched WHEN	3-27
DISTINCT	3-28
ORDER BY	3-30

Ordering Multiple Columns	3-33
CHAPTER 4 BUILT-IN FUNCTIONS	4-1
Overview.....	4-3
Objectives	4-3
Introduction to Built-In Functions.....	4-5
The CAST Function.....	4-6
Conversion Functions	4-9
String Functions.....	4-11
Parsing Text.....	4-11
Text Length	4-13
Other String Manipulation Functions.....	4-15
Numeric Functions.....	4-17
Displaying Dates	4-19
Db2 Date Formats	4-19
SQL Server Dates	4-21
Common Oracle Date Formats:	4-22
Oracle TO_CHAR Function.....	4-23
PostgreSQL Date Functions	4-26
PostgreSQL TO_CHAR Function	4-27
Date Arithmetic.....	4-29
Date Subtraction	4-29
Db2.....	4-30
SQL Server	4-31
Oracle.....	4-33
PostgreSQL.....	4-35
Removing NULLs	4-37
Ranking and Numbering Rows	4-38
Rank Function.....	4-38
Dense Rank Function.....	4-39
Row Numbering.....	4-41
CHAPTER 5 AGGREGATION	5-1
Overview.....	5-3
Objectives	5-3
Scalar Functions.....	5-4
Column Functions.....	5-5
Common Column Functions	5-6
How Column Functions Work.....	5-7
Example	5-8

COUNT Function	5-9
Column Functions with Columns.....	5-10
The GROUP BY Clause	5-11
Restricting Groups with HAVING	5-16
GROUPing by Multiple Columns.....	5-17
CHAPTER 6 JOINS.....	6-1
Overview.....	6-3
Objectives.....	6-3
Inner Join	6-4
Inner Joins -DEPT Table	6-5
Inner Joins -EMP Table.....	6-6
Inner Joins – Example 1	6-7
Inner Joins – SQL86	6-8
Inner Joins – SQL92	6-9
Inner Joins – Example 2	6-10
Cross Joins.....	6-11
Outer Joins	6-12
Left Outer Join Example.....	6-13
Left Outer – Local Predicate.....	6-15
Right Outer Join Example.....	6-17
Full Outer Join	6-19
Full Outer Join Example	6-20
Left Outer Join – On Clause Extensions	6-23
Self Joins / Other Joins	6-25
Self Referencing Foreign Keys	6-26
Comparing Rows to Other Rows.....	6-28
Joining to a Table Multiple Times.....	6-30
CHAPTER 7 DATA MODIFICATION	7-1
Overview.....	7-3
Objectives.....	7-3
Data Modification Statements	7-4
Insert.....	7-5
Insert with Multiple Values.....	7-6
Db2 Multi-row Insert	7-7
UPDATE.....	7-9
Searched UPDATE	7-9
UPDATE with SELECT	7-10
Merge Statement.....	7-11
Merge with Subselect.....	7-11
MERGE with Values	7-12

Oracle.....7-13

PostgreSQL MERGE Uses ON CONFLICT7-14

DELETE7-15

 SQL TRUNCATE Table7-16

 TRUNCATE Db2 LUW7-17

 TRUNCATE - Oracle.....7-19

 TRUNCATE – SQL Server7-20

 PostgreSQL TRUNCATE.....7-21

 Db2 SELECT FROM7-22

 SELECT FROM MERGE7-23

Unit of Work & Transaction Processing.....7-24

 Db2 Commit.....7-25

 Db2 Rollback7-27

 Oracle Transaction.....7-30

 PostgreSQL Transaction.....7-32

Constraints.....7-33

CHAPTER 8 SQL SUBQUERIES8-1

Overview.....8-3

 Objectives8-3

Why Subqueries?8-5

Non Correlated Subqueries8-7

 Non Correlated Example8-9

Correlated Subqueries8-10

 Simple Correlated Example8-11

 Existence Check.....8-15

 Example Differences8-20

Scalar Fullselects.....8-21

 Example #18-22

 Example #28-23

Chapter 1

Relational Database Concepts

Table of Contents

CHAPTER 1 RELATIONAL DATABASE CONCEPTS.....	1-1
Overview.....	1-3
Objectives	1-3
Data Management.....	1-5
Database Management.....	1-7
Database Management System	1-9
Relational Catalog	1-9
Buffers	1-9
Locking Mechanisms.....	1-9
Transaction Logs	1-9
The Relational Model of Data	1-11
Relational Terminology	1-13
Structure	1-14
Normalization	1-17
Domains.....	1-19
Relational Structure: A Logical Progression	1-21
Relationships.....	1-23
Integrity and Relationships	1-24
One-to-Many Relationships	1-25
One-to-One Relationships	1-27
Many-to-Many Relationships.....	1-29
Relational Database Vendors.....	1-31
Structured Query Language (SQL)	1-33
Data Definition Language (DDL)	1-33
Data Control Language (DCL).....	1-33
Data Manipulation Language (DML)	1-33
Creating Tables.....	1-34
Manipulation	1-36
Restrict.....	1-39
Project	1-41
Combined Operations.....	1-43
Join	1-44
Relational Database Components	1-47
The Optimizer	1-49

Overview

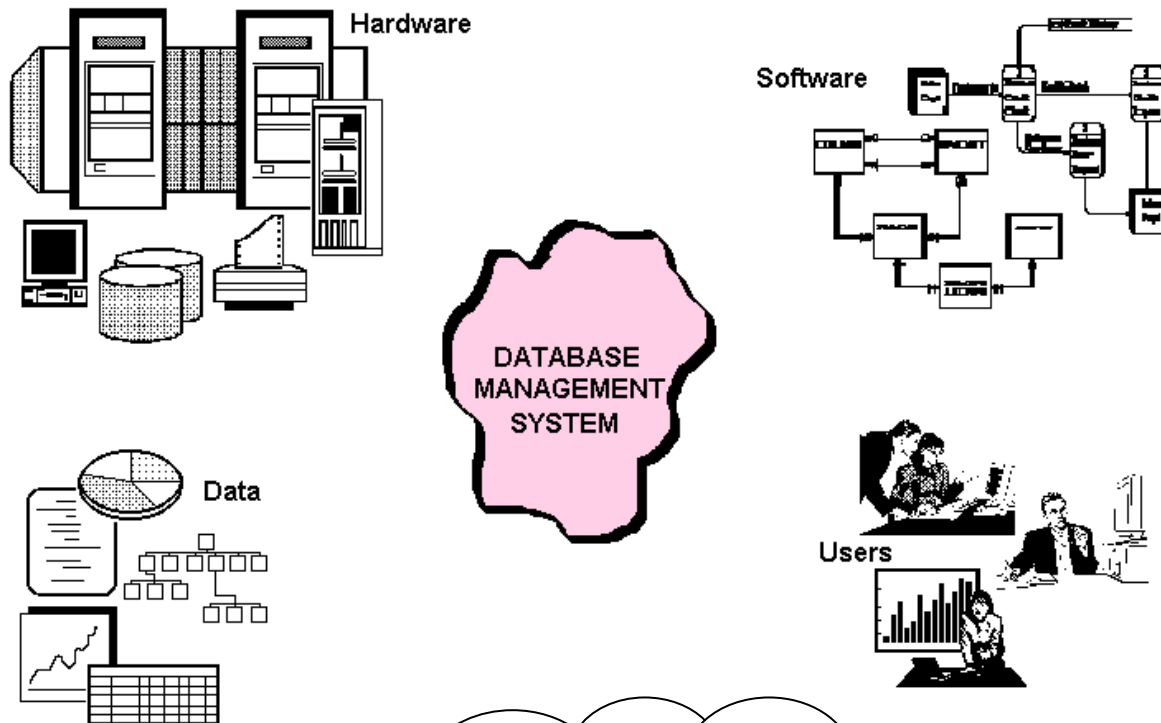
This chapter is an introduction to database management concepts and the Relational Model of data. Common relational terms are utilized throughout the chapter to describe the basic components, architectures, and approaches to database management systems.

Objectives

By the end of this chapter, you should:

- ◆ Be able to define the concept of Data Independence
- ◆ Know the components of a Database Management System
- ◆ Understand the relational operators and their effect on relations
- ◆ Understand the Relational Model of data and the common relational terminology
- ◆ Know the structural components of a relational system
- ◆ Understand two forms of integrity and the structural components that implement them
- ◆ Know the manipulative operations and give examples of each
- ◆ Understand the role of the optimizer and the approaches to access path selection

Data Management



The overall purpose of a database management system is to *maintain information* and to make that information available on demand.

Concept

Data Management

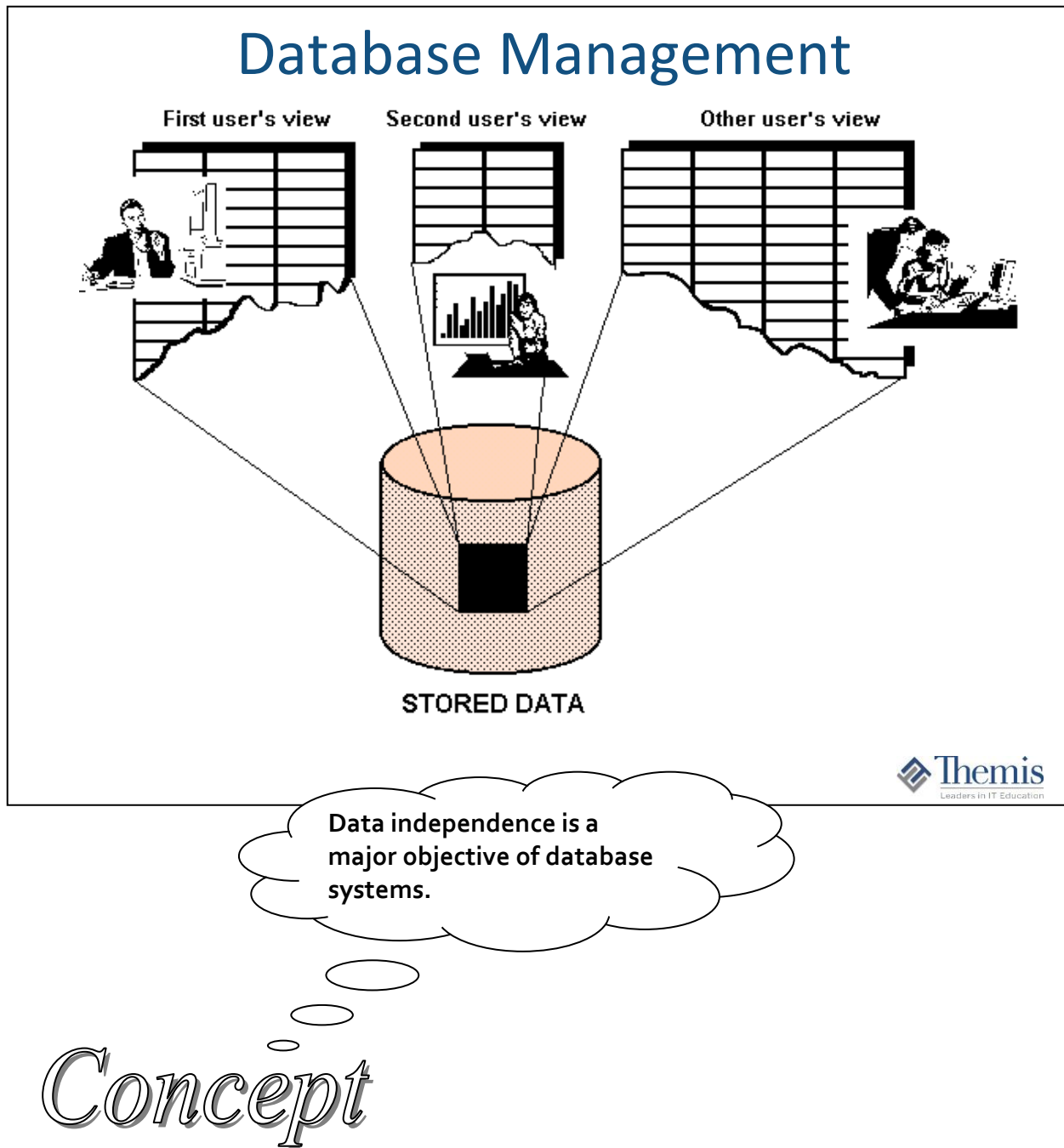
The term database management system (DBMS) is satisfied by many definitions: electronic record keeping system, corporate data store, information repository, just to name a few. While all of these descriptions are in themselves correct, they are more in the line of *properties* or *functions* of modern systems. Today, they are much more than an access method; ***modern database management systems are literally the cornerstone of all corporate data requirements.***

Still, database management systems are not management information systems. Although they are probably the most significant part, a management information system (sometimes referred to as a decision support system) is composed of a graphical user interface (GUI) and a very powerful query facility or data manipulation language (DML). When combined with hardware and a communication network, these components fulfill the requirements of a client/server relationship capable of delivering a wide range of strategic business applications.

The basic ideas of a DBMS are still in effect, even though the complexity of the application and the demand for rapid access to an ever-growing store of detailed data has continued to increase.

The overall purpose of a database management system is to maintain information and to make that information available on demand.

Hardware, software, data and users make up the data management community. Applications represent the needs of the user to manipulate the data, but do not in their own right determine the form or content of the system.



Database Management

In general, the data in a database management system will be both *integrated* and *shared*.

Integrated means that the data can be thought of as a collection of related information, based on its actual meaning (semantics) as it relates to the business.

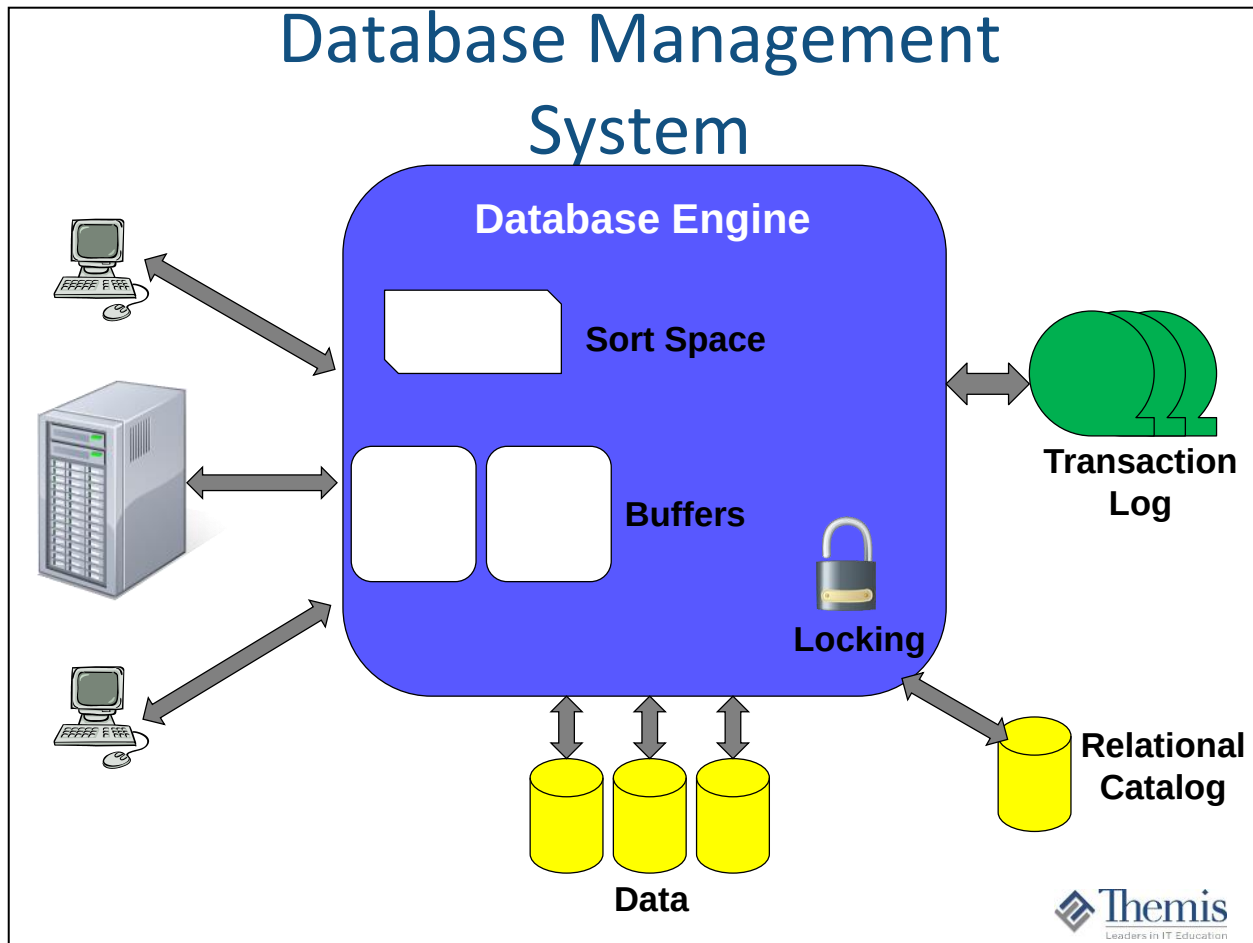
The data is organized around these inherent properties and presented as a sharable resource. **Shared** means that the data stored in the database system can be used by many users, and for different purposes. Indeed, this property of sharability is in fact possible only because of the integrated quality of the data.

The information concerned can be anything that is important to the business and should be returned in a form that is usable by the requester.

In addition, since different applications will need different views of the same data, the database administrator (DBA) must have the freedom to change the storage structure or access technique in response to changing requirements in the real world. These changes must be affected without having to modify existing applications (or user procedures).

Data independence is achieved when applications are unaffected by changes made to the data structure, storage mechanisms, and access techniques.

Various architectural models have been employed to facilitate deriving information in a usable form from the stored data.



Database Management System

A database management system is a set of data structures and code that is used to manage data, access to that data and certain business rules surrounding the data. Common components include:

Relational Catalog

The catalog is itself a relational database, that is, a collection of tables that contains information about the tables, columns, indexes, etc. Oracle refers to this as the data dictionary.

Buffers

Data is cached in memory on the server so that frequently accessed data will not require I/O processing. Database Administrators have a large degree of control over the size and behavior of the buffers.

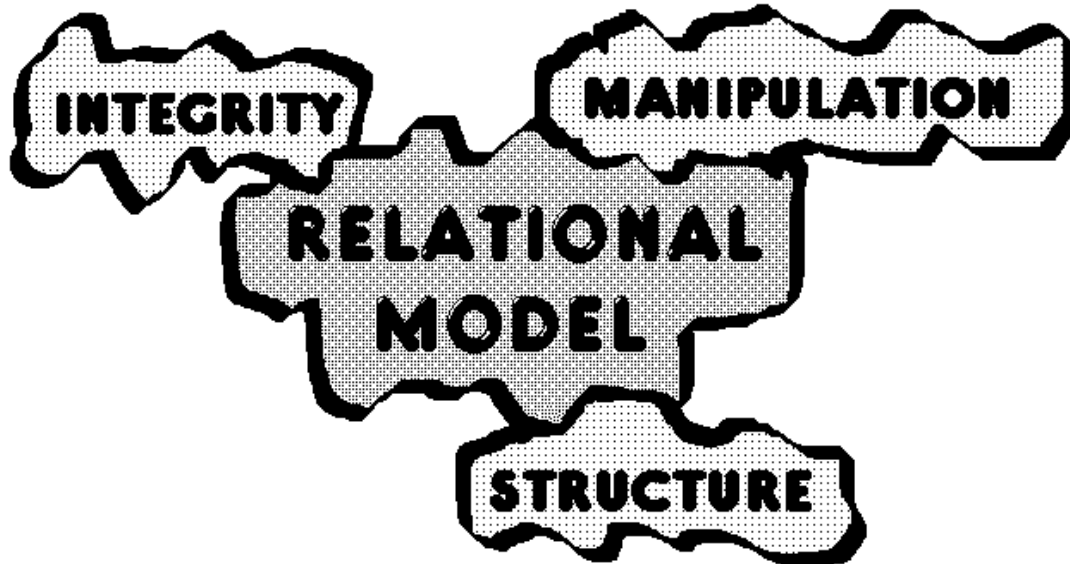
Locking Mechanisms

To facilitate multiple users and applications accessing and changing the data, locking mechanisms are used to protect the data from double updates. Locking can also protect users from reading data that is not in a stable, committed state.

Transaction Logs

Database management systems also use transaction logs to monitor every change to the data. The log typically gets a before and after image of any data row that changes. Transaction logs are used by the database to support Unit of Work (UOW) processing where an application may request a *rollback* or un-doing of all changes since the last commit. Transaction logs may also be used in database recovery to restore to a point in time other than when a backup was taken. Log files are also used for crash recovery when the database restarts after a failure.

The Relational Model Of Data



The Relational Model of Data

The Relational Model of Data was conceived by Dr. Edgar Codd, an IBM Fellow, and presented in a paper entitled *A Relational Model of Data for Large Shared Data Banks* that appeared in the Communications of the ACM in June 1970. This short introduction characterized a new concept in data processing; the separation of the definition and maintenance of the data resource from the definition and maintenance of the process resource. This was the birth of ***data independence*** described earlier in this chapter.

Codd also reasoned that the data should be relatively simple to access via what he called a data sub-language, suitable for both interactive real-time queries (not very prominent at the time) and the more common batch programs. **This *communicability objective*** extended to the definition of the data structures, as well as their manipulation through the same data sub-language.

A third objective, the ***set processing objective***, insists that the processing of the data (update, insert and delete included) be extended to complete quantities of data (sets) rather than the usual record-at-a-time processing.

Within these objectives emerged a model of the data resource with 3 distinct properties.

- ◆ Structure
- ◆ Integrity
- ◆ Manipulation

The relational model is an abstraction of the way the data is physically stored. Details such as indexes, pointer chains, linked lists, randomizing algorithms, and the like are *hidden from the users*, as was described with the conceptual schema.

All of the information content implied by a relational system is represented in one and only one way; explicit data values. All data values are atomic (or scalar). Single values, rather than groups of values, are the standard.

Relational Terminology

Relational Term	Conventional Equivalent
RELATION	TABLE
TUPLE	ROW (or record)
ATTRIBUTE	COLUMN (or field)
CARDINALITY	Number of rows
DEGREE	Number of columns
DOMAIN	Pool of legal values and operations
SCALAR	Smallest semantic unit of data
PRIMARY KEY	Unique identifier
FOREIGN KEY	Relationship identifier

Relational Terminology

Some relational terms come from mathematics in the area of set theory, others are from data processing. The list presented on the facing page represents fundamental terms most commonly used. Writings on the subject of relational databases may use different terms to represent the concepts presented here. Also, the terms used here may be used elsewhere to represent entirely different concepts. A complete investigation of the terminology and notational techniques used in the documentation of a particular product should be considered as a prerequisite for further study.

While all of these terms come from the area of structure, the terms primary key and foreign key overlap into the area of integrity. Of course, structure and integrity (the truth of what the data means) are necessary for successful manipulation and interpretation of the result.

STRUCTURE: An Example Table

**Column
(Vertical Organization)**

EMPNO	FIRSTNAME	LASTNAME	DEPARTMENT	SEX	SALARY	BONUS	BIRTHDATE	HIREDATE	EDUCATION	JOBLEVEL	LOC
000100	BARRY	HARRIS	A00	M	125000.00	50000.00	1946-09-24	1966-10-24	5	18	NJ
000500	HERB	ELLIS	C12	M	-----	-----	1984-06-01	1982-04-10	1	4	NJ
000240	REDD	GARLAND	B01	M	65000.00	10000.00	1958-07-31	1992-01-29	3	14	NY
000940	RAY	BROWN	A00	M	95000.00	25000.00	1950-02-12	1969-08-21	4	12	NJ
000640	NANCY	WILSON	B01	F	50000.00	-----	1962-10-12	1995-03-18	4	4	NY
000460	LINDA	LOU	C12	F	40000.00	-----	1981-05-05	2002-04-10	2	4	PA
000390	TIFFANY	RENEE	B11	F	65000.00	10000.00	1980-11-06	1992-05-21	4	14	NJ
000350	EVAN	JOHN	C12	M	65000.00	4000.00	1984-08-27	2002-04-10	4	8	NJ
000485	MARYLOU	WILLIAMS	C01	F	50000.00	5000.00	1976-10-05	1990-04-16	4	10	NY
000410	BEN	WEBSTER	C12	M	63000.00	10000.00	1975-08-14	1993-11-20	3	10	NJ
000225	LESTER	YOUNG	C01	M	55000.00	-----	1973-12-01	1995-06-05	3	8	NY
000400	ANN	WILSON	C01	F	40000.00	-----	1977-04-12	1998-04-12	1	4	NJ
000300	NICA	COLLINS	C11	F	50000.00	4000.00	1977-03-17	1998-04-12	4	8	NJ
000715	CLIFFORD	BROWN	C11	M	50000.00	1000.00	1970-11-17	1999-02-14	4	8	NJ
000375	BUDD	POWELL	C12	M	50000.00	-----	1970-11-17	1999-02-14	4	6	NY

**Row
(Horizontal Organization)**

The only structure that
exists in the Relational
Model is a table.

Concept

Structure

All data in a relational database is viewed as a collection of related tables. Tables are *repositories* for the information to be remembered about the business entities they represent. The columns are also repositories; each column embodies a single-valued fact about the entity. The *and*ing together of all the attributes contained in the columns of a single row represent facts about one member of the collection.

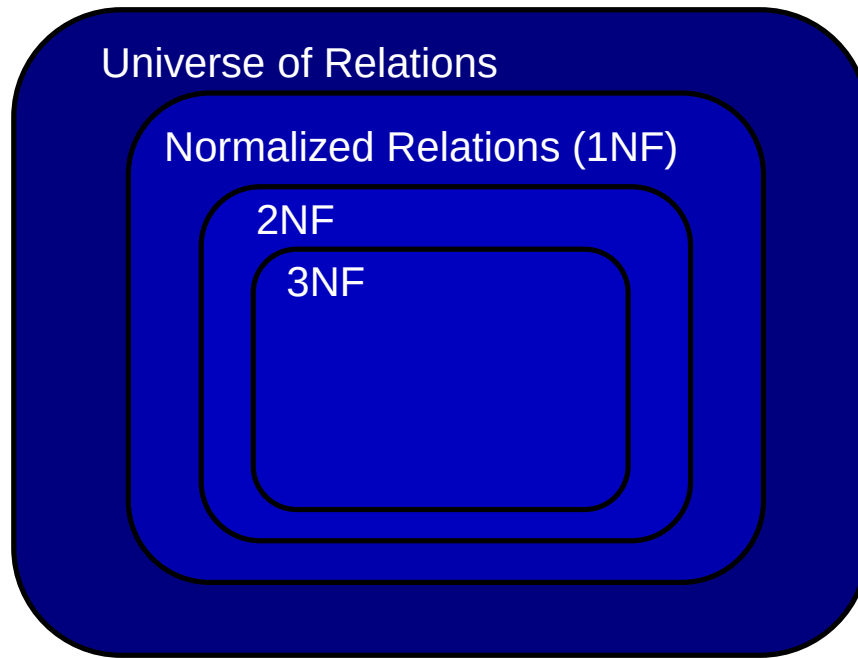
The term Relation is used interchangeably with table. Relation is more formal than table but in common terms they mean the same thing. A relation (value) and a table (storage) share the same relationship as a number (value) and a variable (storage).

Either way, both relations and tables are a 2-dimensional collection (set) consisting of rows and columns. Again, the terms tuple and attribute are more formal but generally mean the same thing. Properties of tables:

- ◆ A table can have 0, 1, or many rows
- ◆ Each row is uniquely identifiable
- ◆ Each row has the same number of columns
- ◆ Each column is of a particular datatype
- ◆ Exactly one value exists the intersection of a row and a column

The only structure that exists in the Relational Model is a table as described above. Remember, however, that ***relational theory encompasses the Conceptual Model not the Internal Model***. The data is not physically stored as a table.

Normalization



Normalization

Normalization is the process of structuring relational database schema such that most ambiguity and redundancy is removed. The stages of normalization are referred to as normal forms and progress from the least restrictive (First Normal Form) through the most restrictive (Fifth Normal Form). Generally, most database designers do not attempt to implement anything higher than Third Normal Form.

Normalization can be broken down into 5 (or 6 if you adhere to the intermediate refined third normal form called the BCNF—Boyce-Codd Normal Form) forms or levels. Normalization Forms are merely guideline rules to follow when identifying detail entities and the attributes that belong to those entities.

First Normal Form: Move repeating groups to a new entity. (Clue to repeating groups is the use of a plural in defining or listing of the data facts.) This avoids the multiple occurrences of attributes within a single entity.

Second Normal Form: Move attributes dependent on only part of the primary key to another entity. This verifies that the attributes are true properties of the entity they exist in.

Third Normal Form: Move attributes dependent on a key other than the primary key to another entity. This is another way to verify that the attributes are true properties of the entity.

Domains

Cardinality



EMPNO	FIRSTNAME	LASTNAME	DEPARTMENT	SEX	SALARY
000100	BARRY	HARRIS	A00	M	125000.00
000500	HERB	ELLIS	C12	M	-----
000240	REDD	GARLAND	B01	M	65000.00
000940	RAY	BROWN	A00	M	95000.00
000640	NANCY	WILSON	B01	F	50000.00
000460	LINDA	LOU	C12	F	40000.00
000390	TIFFANY	RENEE	B11	F	
000350	EVAN	JOHN	C12	M	
000485	MARYLOU	WILLIAMS	C01	F	
000410	BEN	WEBSTER			
000225	LESTER	YOUNG			
000400	ANN				
000300					

Degree

Domains

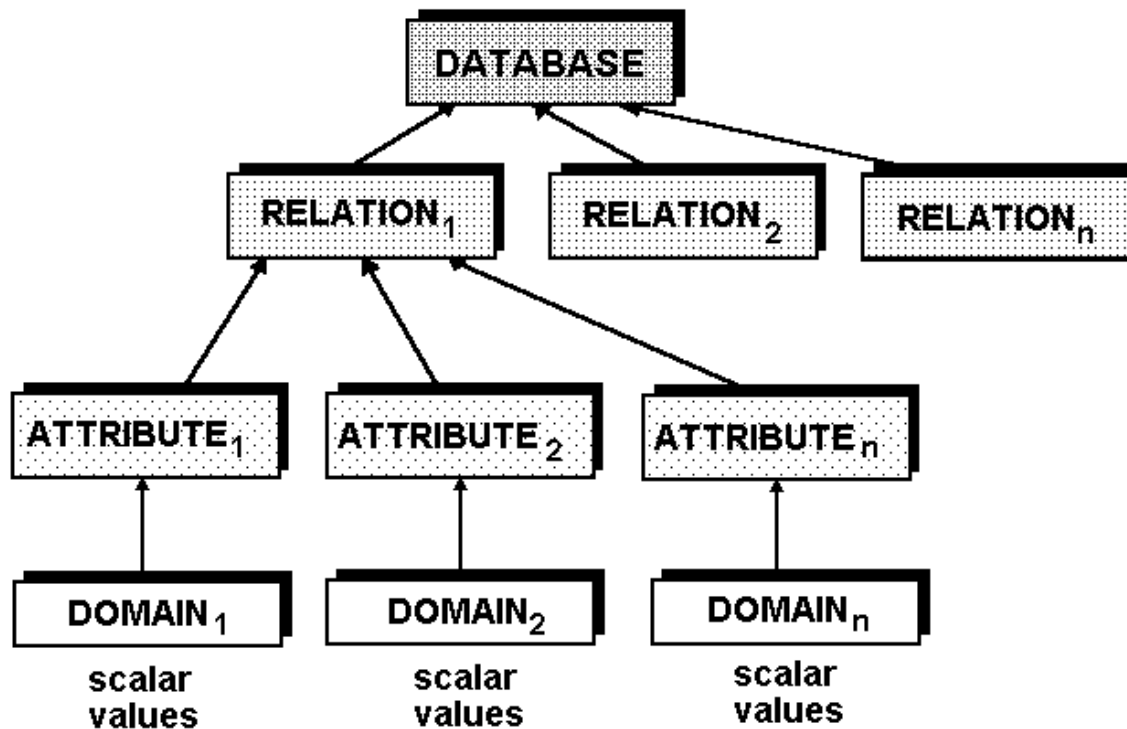
As mentioned above, each column of the table has a particular datatype. Actually, it's more correct to say that each column derives its values from a particular domain.

A domain is the collection of all values deemed legal for assignment to that column.

If two attributes draw their values from the same domain, then comparisons and other relational operations will probably make sense. When operations involve different domains, the comparisons are like 'apples' and 'oranges'. For example, the database might contain a part numbered 100 and also a supplier numbered 100; but is it legal to compare part 100 to supplier 100? As another example consider that an employee may have attributes representing age and years of service. While 35 may be legal for either, it would be impossible for both to have that value. Equally impossible would be any circumstance where years of service were greater than age.

Domains are integral to the Relational Model since they define all operations that are allowed on the values of any column. Users (or programs) that insert or update must always be aware of the domain of the column. Although a value may be legal, it may not be semantically appropriate to be assigned to a particular attribute.

Relational Structure: A Logical Progression



Relational Structure: A Logical Progression

Domains are thus *datatypes* in the programming sense and relational technology, as suggested by the progression on the facing page, is really an abstract programming environment. Note: The diagram uses the more formal relation/attribute designation; in common usage, table/column may be substituted.

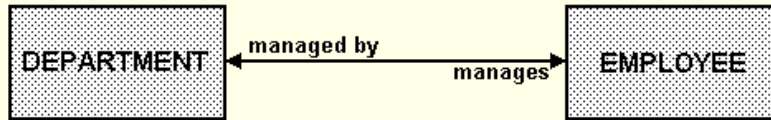
The mappings between the domains and the attributes represent the semantic business rules governing the assignment of values that represent a business entity. Each attribute governed is by its own unique business rule for the assignment of a scalar value.

The meaning of each data item is determined during the process known as data analysis. During data analysis, rules are applied to separate the data into two sets: keys and non-keys.

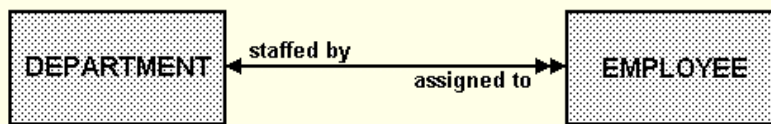
The application of the rules, known as data normalization, is based on the analysis of the dependence that the non-key attributes have on the key attributes. Repetitive application of the rules results in a minimal representation of the data represented as a collection of relations and known as a relational database. This minimal representation is achieved when the model encompasses no redundancy or derivable data, and all non-key attributes are dependent on a key attribute.

Table-Relationship Diagram

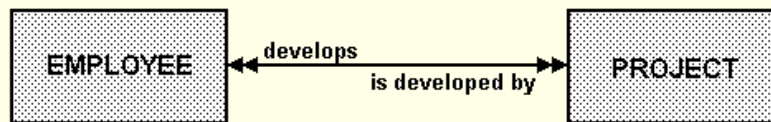
One to One



One to Many



Many to Many



Relationships

Table-Relationship diagrams are a convenient way to represent the data architecture during the analysis and design phases.

Note: we use the term Table-Relationship diagram to mean a graphic representation of the relationships that exist between a set of tables. This should not be confused with the more common Entity-Relationship Diagram or the techniques of Entity-Relationship modeling. In the diagram on the facing page, the rectangles represent the tables while the arrows represent relationships. The tables and relationships shown are representative of the data we will be using in the workshops during this presentation.

Relationships are sets of connections between tables. The cardinality of a relationship is a reference to the concept of how many rows of one table are connected to how many rows of another table. In the diagram on the facing page, single-headed arrows represent one-type relationships while double-headed arrows represent many-type relationships.

Possible combinations are:

- ◆ One-to-One
- ◆ One-to Many/Many-to-One
- ◆ Many-to-Many

Relationships are real-world occurrences and as such are identified during data analysis. *A set of relationships is unique between a given set of tables and uniquely represented within the relational model.*

In the Relational Model, relationships are realized by assigning the roles of **primary key** and **foreign key** to column(s) of the table(s) involved.

Integrity and Relationships

DEPT Table

DEPTNO (PK)	DEPTNAME	MGRNO	ADMRDEPT	LOCATION
A00	SPIFFY COMPUTER SERVICE DIV.	000010	A00	
B01	PLANNING	000020	A00	
C01	INFORMATION CENTER	000030	A00	
D01	DEVELOPMENT CENTER	<null>	A00	
D11	MANUFACTURING SYSTEMS	000060	D01	
D21	ADMINISTRATION SYSTEMS	000070	D01	

EMP Table

EMPNO	LASTNAME	FIRSTNAME	DEPTNO (FK)	JOB
000010	HAAS	CHRISTINE	A00	PRES
000020	THOMPSON	MICHAEL	B01	MANAGER
000030	KWAN	SALLY	C01	MANAGER
000050	GEYER	JOHN	E01	MANAGER
000060	STERN	IRVING	D11	MANAGER
000070	PULASKI	EVA	D21	MANAGER
000110	VINCENZO	LUCCHESI	A00	SALESREP
000120	SEAN	O'CONNELL	A00	CLERK

Integrity and Relationships

The role of primary key and foreign key is on one hand structural, since they are composed of one or more columns of the table. On the other hand, the values assigned to these special column(s) not only falls into the realm of integrity, but actually defines the integrity features of the Relational Model.

One-to-Many Relationships

During the previous presentation on relational structure, it was mentioned that a table can have 0, 1, or many rows and that each row is uniquely identifiable. It is the properties of the Primary Key that makes this enforceable. More formally a primary key is a set of 1 or more columns (including the possibility that *every* column of the table might participate) with 2 distinct properties:

- ◆ No two rows in the table will have the same value on all the columns of the primary key
- ◆ No component of the primary key is allowed to be missing or unknown

Primary keys provide for uniqueness by ensuring that any two rows in a table will differ by at least one columns value. ***These two properties of the primary key are called entity integrity.***

Foreign keys implement bi-directional relationships in the Relational Model. Formally, ***a foreign key is the value of the primary key of a table being referenced***, stored in the table doing the referencing as a normal single-valued column. Remember, in the Relational Model, everything is done by comparing values in columns. This includes relationships as well.

Referential Integrity is being enforced when the system (or the user application) ensures that every Foreign Key that is populated with a value, does in fact derive that value from among the Primary Key values already in existence. Foreign Keys may:

- ◆ Be Unique or non-unique
- ◆ Contain values or be unknown (null)

The diagram on the facing page show the use of Primary and Foreign Keys to implement one-to-one (1:1) and one-to-many/many-to-one relationships (1:M/M:1). Note: the concept of null in relational databases will be fully covered in later chapters on manipulation. Generally speaking, ***a column is null if its value is unknown.***

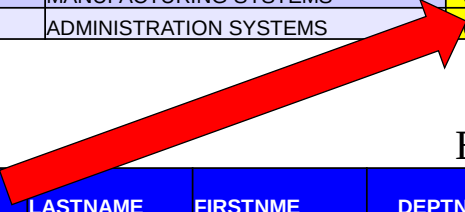
The table being referenced is the parent and the table doing the referencing is the child. This is logical since the value of the foreign key is dependent on the existence of the value of the primary key.

Integrity and Relationships

DEPT Table

DEPTNO (PK)	DEPTNAME	MGRNO (FK)	ADMRDEPT	LOCATION
A00	SPIFFY COMPUTER SERVICE DIV.	000010	A00	
B01	PLANNING	000020	A00	
C01	INFORMATION CENTER	000030	A00	
D01	DEVELOPMENT CENTER	<null>	A00	
D11	MANUFACTURING SYSTEMS	000060	D01	
D21	ADMINISTRATION SYSTEMS	000070	D01	

EMP Table



EMPNO (PK)	LASTNAME	FIRSTNAME	DEPTNO	JOB
000010	HAAS	CHRISTINE	A00	PRES
000020	THOMPSON	MICHAEL	B01	MANAGER
000030	KWAN	SALLY	C01	MANAGER
000050	GEYER	JOHN	E01	MANAGER
000060	STERN	IRVING	D11	MANAGER
000070	PULASKI	EVA	D21	MANAGER
000110	VINCENZO	LUCCHESI	A00	SALESREP
000120	SEAN	O'CONNELL	A00	CLERK

One-to-One Relationships

The tables shown here are actually related in two different ways. The first diagram showed a 1:M relationship between departments and employees who work in those departments. This diagram shows a 1:1 relationship between a department and the manager of that department. The MGRNO column on the DEPT table is a foreign key referencing an employee who is the manager of that department. This will become important when joining these two tables together. How the join is coded will depend on which of these two relationships is being interrogated.

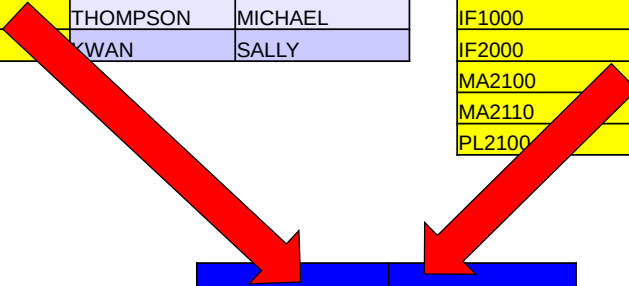
Integrity and Relationships

EMP Table

EMPNO (PK)	LASTNAME	FIRSTNAME
000010	HAAS	CHRISTINE
000020	THOMPSON	MICHAEL
000030	KWAN	SALLY

PROJ Table

PROJNO (PK)	PROJNAME
AD3100	ADMIN SERVICES
IF1000	QUERY SERVICES
IF2000	USER EDUCATION
MA2100	WELD LINE AUTOMATION
MA2110	W L PROGRAMMING
PL2100	WELD LINE PLANNING



EMPNO (FK)	PROJNO (FK)
000010	MA2100
000010	MA2110
000010	AD3100
000020	PL2100
000030	IF2000
000030	MA2100

EMPPROJ Table

Many-to-Many Relationships

Many-to-Many cardinality (M:M) represent complexity in relationships. The diagram depicts the real-world situation where an employee can work on many projects, and a single project can have many employees working on it. The implementation of solutions to this very situation is what inspired the development of database management in the early 1960s. *The implementation of many-to-many mappings requires three structures (tables).*

This third structure serves to decompose the M:M structure into two 1:M structures. Of course, in the Relational Model there is only one structure to consider; a table. **Note: *this third table has no special meaning to the database management system.*** It is simply a table, just like any (every) other table in the system. The fact that it came into existence because of a relationship between employees and projects is not known to the DBMS. It is the manipulation of the table by the user (or application) that gives it meaning. Inserting a row into this association table links the two rows referenced by the semantics of M:M relationship. In our example, *it's the act of assigning an employee to work on a project.*

In modeling nomenclature, this third table is called an *association table*, (the M:M mapping is called an associative relationship) that associates (maps) the rows of one table to the rows of another table and vice versa.

Also like any other table, the association table has a primary key. In our example, the primary key is the combination of the columns EMPNO and PROJNO. Taken together this key is unique in the association table. This is commonly known as a ***compound key***. Compound keys are primary keys that overlap one or more foreign keys.

Referential integrity remains the same. To assign an employee to a project, both foreign key values must exist. In relational terms, in order to insert a row into the association table, the foreign key EMPNO and the Foreign Key PROJNO must be validated by assuring that primary keys with matching values do in fact exist in the respective tables.

Relational Database Vendors

- ***Oracle Database®*** from Oracle Corporation
- ***Db2®*** from IBM
- ***SQL Server®*** from Microsoft

Relational Database Vendors

Many different vendors have implemented relational database management systems (RDBMSs). The most popular on the market today are:

- ◆ *Oracle Database®* from Oracle Corporation
- ◆ *Db2®* from IBM
- ◆ *SQL Server®* from Microsoft

Oracle runs mostly on the Unix and Windows platforms.

Db2 has 2 distinct flavors; one running on Unix and Windows, and the other on z/OS (mainframe) systems. The two flavors of Db2 have a different code base, but share much of the same functionality regarding SQL.

SQL Server runs exclusively on the Windows platform.

There are also many other implementations of these concepts. Most of the SQL in this course will work on most database platforms. The focus of the course is on the three platforms mentioned above. Differences in syntax will be noted where appropriate.

Structured Query Language (SQL)

- Data Definition Language (DDL)
 - CREATE
 - ALTER
 - DROP
- Data Control Language (DCL)
 - GRANT
 - REVOKE
- Data Manipulation Language (DML)
 - SELECT
 - INSERT
 - UPDATE
 - DELETE

Structured Query Language (SQL)

The language used to maintain and access relational databases is *Structured Query Language* (SQL). All the major relational database vendors use SQL as the primary language for interacting with the data stored in a relational database management system (RDBMS). SQL was adopted as a standard by the American National Standards Institute (ANSI) in 1986. The original standard is known as SQL-86. SQL was also adopted by the International Organization for Standardization (ISO) in 1987. A major revision to the standard was adopted in 1992 (known as SQL-92), and further enhancements were added in 1999, 2003, 2006 and 2008. Each of the RDBMS vendors implements their own flavor of SQL and will publish deviations from the standard.

SQL can be broken into 3 categories of statements:

Data Definition Language (DDL)

DDL is used to build and maintain the database environment. This is usually the responsibility of a database administrator (DBA).

Data Control Language (DCL)

DCL is used to control access to the environment. This is usually done by DBAs or security administrators.

Data Manipulation Language (DML)

DML is used by any authorized user to read or update the data in a relational database. DML will be the primary focus of this course.

Creating Tables

```
CREATE TABLE DEPT
(DEPTNO    CHAR(3)          NOT NULL,
DEPTNAME   VARCHAR(36)     NOT NULL,
MGRNO      CHAR(6)          ,
ADMNDEPT   CHAR(3)          NOT NULL,
LOCATION     CHAR(16)         ,
PRIMARY KEY (DEPTNO) )
```

```
CREATE TABLE EMP
(EMPNO      CHAR(6)          NOT NULL,
. . .
DEPTNO      CHAR(3)          NOT NULL,
. . .

PRIMARY KEY (EMPNO) ,
FOREIGN KEY (DEPTNO) REFERENCES DEPT)
```

Support for referential
integrity does not require
procedural code.

Concept

Creating Tables

The example on the facing page illustrates the use of the PRIMARY KEY and FOREIGN KEY...REFERENCES clauses of the CREATE TABLE statement in SQL. This example is by no means complete and is not intended to be. It is included here to depict the concept of *declarative referential integrity*.

As shown, support for referential integrity does not require procedural code. The tables and columns participating in the referential constraint are identified and assigned the required roles.

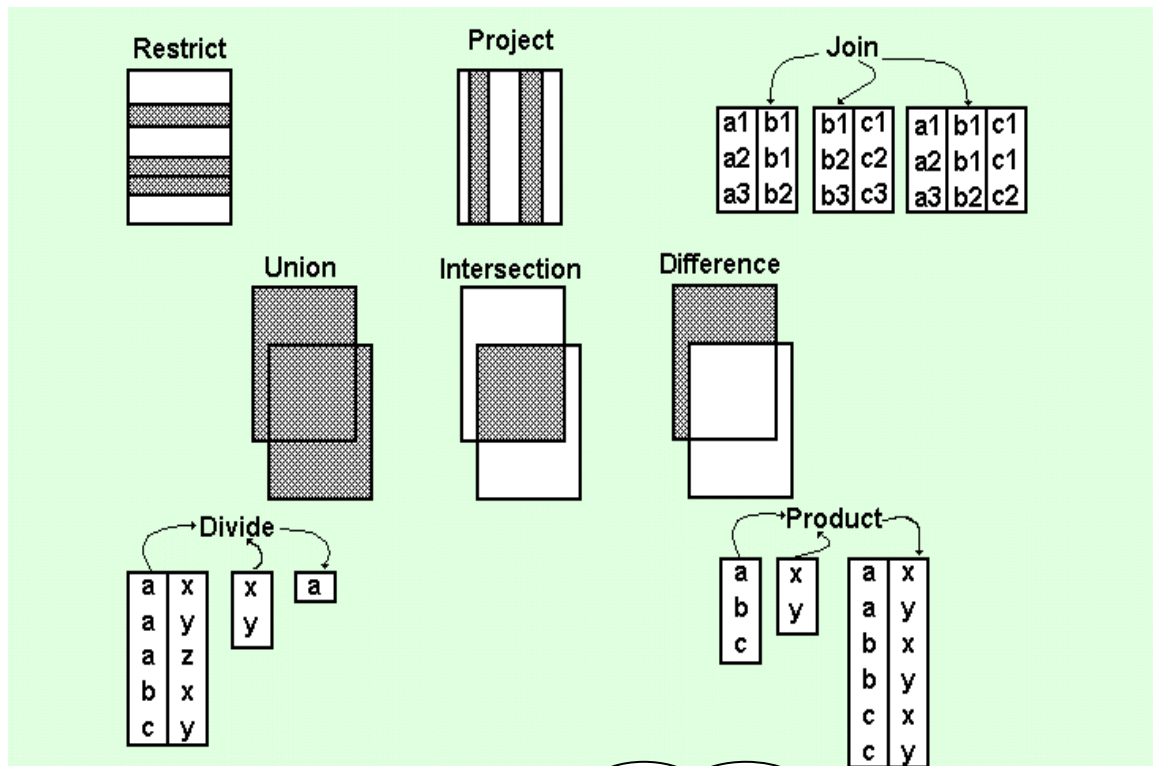
In the example:

- ◆ The primary key of the table DEPT is DEPTNO
- ◆ The primary key of the table EMP is EMPNO
- ◆ The column DEPT in the EMP table is defined as a foreign key referencing the table DEPT

Note how the foreign key clause need only identify the referenced table. The Primary Key of DEPT is already known to the relational system, since its creation had to precede the definition of the EMP table (this is merely the implementation of referential integrity in the catalog itself).

Once the declarations are recognized by the relational system the enforcement is automatic. The value of the foreign key will be checked against the primary key for each insert into EMP, and for each update of the column DEPTNO.

Manipulation



Manipulation makes
Tables out of Tables

Concept

Manipulation

The diagram on the facing page illustrates 8 operations that can be performed on tables. Keep in mind that tables are the only structure present in the Relational Model. All manipulation starts and ends with a table. Tables that are persistent (those that are actually stored somewhere on an auxiliary device) are called **base tables**, all other tables represent some form of **derived table**, i.e. derived from the base tables.

Manipulation starts with a table of some cardinality and degree and results in a table of some cardinality and degree. Since the result of any manipulative operation is a table, it is suitable for input to another manipulative operation. This is called the **Relational Closure Property** and is just an extension of the same closure property that exists in arithmetic and algebra. Closure allows for the nesting of queries and great degrees of flexibility in the handling of relational manipulation.

In the original literature Dr. Codd grouped the eight operations as the relational operations restrict (which he called select), project, join, and product. The others (intersect, union, difference and divide) were called set-processing operations from their counterparts in set theory.

Today most literature on the subject recognizes three groupings of the eight operations: three relational (restrict, project and join), three set-processing (intersect, union and difference) and two algebraic (product and divide). The examples appearing on the next few pages appeal to the intuitive nature of the basic operation of making tables out of tables.

During this presentation we will look at each of these operations except for divide. The course heavily emphasizes the operations restrict, project and join while defining the set operations (with the exception of union) in terms of these basics.

Restricting Rows

DEPT Table

DEPTNO	DEPTNAME	MGRNO	ADMRDEPT	LOCATION
A00	SPIFFY COMPUTER SERVICE DIV.	000010	A00	
B01	PLANNING	000020	A00	
C01	INFORMATION CENTER	000030	A00	
D01	DEVELOPMENT CENTER	<null>	A00	
D11	MANUFACTURING SYSTEMS	000060	D01	
D21	ADMINISTRATION SYSTEMS	000070	D01	

**RESTRICT the result
to Departments with a
DEPTNO that begins
with the letter 'D'**

Result Table

DEPTNO	DEPTNAME	MGRNO	ADMRDEPT	LOCATION
D01	DEVELOPMENT CENTER	<null>	A00	
D11	MANUFACTURING SYSTEMS	000060	D01	
D21	ADMINISTRATION SYSTEMS	000070	D01	

Restrict

RESTRICT is probably the most common operation performed on a data store whether relational or not. Extracting only the required information is a task formerly done by client application programs. In a relational environment the RESTRICT is carried out at the server (RDBMS) on behalf of the client.

Formally a RESTRICT operation starts with a table and applies search criteria called *predicates* to the columns of each row of the table. The table derived from this operation is composed of all the rows (and their inherent meaning) that resulted from a *true* condition with respect to the applying the predicates to the values in those columns, at that point in time. Rows where the predicates evaluate to false (or unknown) are rejected from the result. The operation is also called SELECT which should not be confused with the SQL SELECT statement. The term *filtering* is also applied to this operation without confusion as to what is actually happening to the data.

RESTRICT operations affect the cardinality of the result; 0, 1, many, or all of the rows may qualify based on the predicate(s) involved.

The example on the facing page is requesting all of the departments where the first letter of the department number is 'D'. The result, as shown, is a table consisting of only the desired rows from the original table.

Programming algorithms to produce this result are among the most basic. ***In the relational query environment, the designer of the query need only know the table name and the column names to accomplish the result.***

In terms of SQL, the WHERE clause of an SQL statement controls the RESTRICT operation.

Projecting Columns

DEPT Table

DEPTNO	DEPTNAME	MGRNO	ADMRDEPT	LOCATION
A00	SPIFFY COMPUTER SERVICE DIV.	000010	A00	
B01	PLANNING	000020	A00	
C01	INFORMATION CENTER	000030	A00	
D01	DEVELOPMENT CENTER	<null>	A00	
D11	MANUFACTURING SYSTEMS	000060	D01	
D21	ADMINISTRATION SYSTEMS	000070	D01	

Show only
DEPTNO and
DEPTNAME
columns.

Result Table

DEPTNO	DEPTNAME
A00	SPIFFY COMPUTER SERVICE DIV.
B01	PLANNING
C01	INFORMATION CENTER
D01	DEVELOPMENT CENTER
D11	MANUFACTURING SYSTEMS
D21	ADMINISTRATION SYSTEMS

Project

While RESTRICT produced a subset of rows, PROJECT produces a subset of columns. A PROJECT operation controls the degree of the result table. When a row meets the conditions set forth in the RESTRICT operation, only the requested columns appear in the final result, in the order that they are requested.

RESTRICT and PROJECT are separate operations. Once a row qualifies for the result, the requested columns are PROJECTed (copied) to the result table.

PROJECT operations are very beneficial to data independence. Result tables, derived by specifically requesting the required columns, are insulated from columns added to base tables after their initial creation. The example on the facing page shows a request for only certain information about all of the departments.

In terms of SQL, the SELECT clause of an SQL statement controls the PROJECT operation.

Combined Operations

DEPT Table

DEPTNO	DEPTNAME	MGRNO	ADMRDEPT	LOCATION
A00	SPIFFY COMPUTER SERVICE DIV.	000010	A00	
B01	PLANNING	000020	A00	
C01	INFORMATION CENTER	000030	A00	
D01	DEVELOPMENT CENTER	<null>	A00	
D11	MANUFACTURING SYSTEMS	000060	D01	
D21	ADMINISTRATION SYSTEMS	000070	D01	

Result Table

DEPTNO	DEPTNAME
D01	DEVELOPMENT CENTER
D11	MANUFACTURING SYSTEMS
D21	ADMINISTRATION SYSTEMS

**Show only DEPTNO
and DEPTNAME
columns for
departments that
begin with 'D'.**

Combined Operations

RESTRICT and PROJECT operations can be combined to provide for dynamically creating result tables of various cardinality and degree.

Since they are separate relational operations:

- ◆ Columns used to RESTRICT rows need not appear in the final result
- ◆ PROJECTed columns need not be mentioned in the predicate(s) controlling the RESTRICTion

The combination of these two heavily used relational operations characterize a wide range of application requirements. The example on the facing page simply shows the combination of the operations of the previous two examples.

Join

DEPT Table

DEPTNO (PK)	DEPTNAME	MGRNO	ADMRDEPT	LOCATION
A00	SPIFFY COMPUTER SERVICE DIV.	000010	A00	
B01	PLANNING	000020	A00	
C01	INFORMATION CENTER	000030	A00	
D01	DEVELOPMENT CENTER	<null>	A00	
D11	MANUFACTURING SYSTEMS	000060	D01	
D21	ADMINISTRATION SYSTEMS	000070	D01	

PROJ Table

PROJNO	PROJNAME	DEPTNO (FK)	EMPNO	PRSTAFF	PRSTDATE
AD3100	ADMIN SERVICES	D01	000010	6.5	1982-01-01
AD3110	GENERAL AD SYSTEMS	D21	000070	6	1982-01-01
AD3111	PAYROLL PROGRAMMING	D21	000230	2	1982-01-01
AD3112	PERSONNEL PROGRAMMG	D21	000250	1	1982-01-01
AD3113	ACCOUNT.PROGRAMMING	D21	000270	2	1982-01-01
IF1000	QUERY SERVICES	C01	000030	2	1982-01-01

A result containing **PROJNO** and **PROJNAME** from the **PROJ** table, along with **DEPTNO** and **DEPTNAME** from the **DEPT** table for the department that funds each project

Join

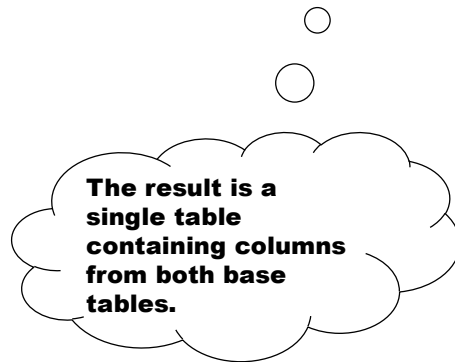
JOIN operations are used to combine the rows of two or more tables to form a single row of data in the result. Joins are the most dynamic aspects of relational manipulation. *The derived table resulting from a join is part of the conceptual model, but does not itself exist as a stored table (base table).* Whenever the columns of the final result come from more than one table, the tables must be joined to obtain the result. RESTRICT and PROJECT operations may appear with the JOIN (and they usually do).

JOINS are performed by matching the values of columns that are common to the tables being JOINed. Very often these columns are the primary key of one table and the foreign key of another table, but the use of these keys is not a necessary condition for performing a JOIN.

Join

Result Table

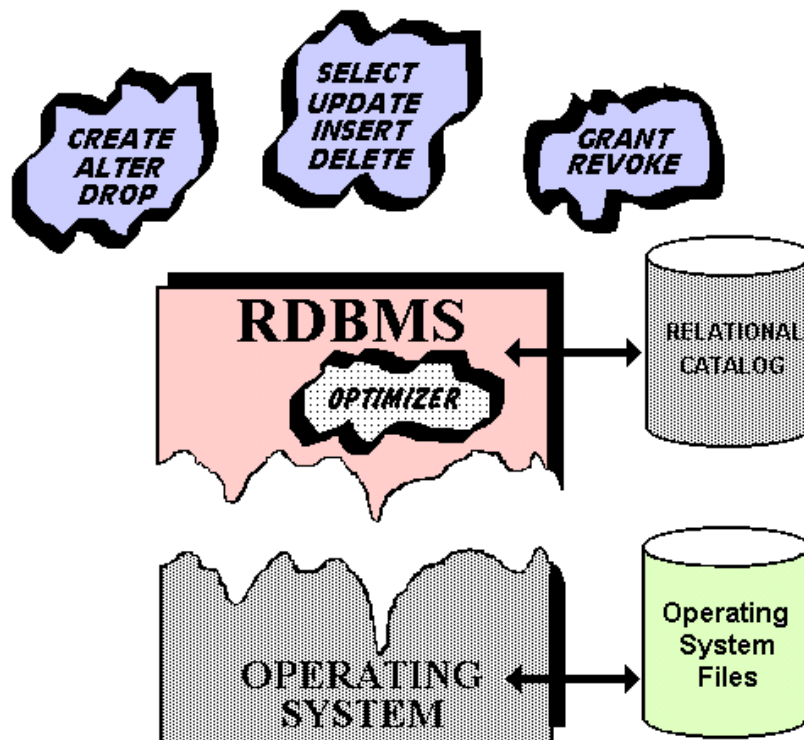
PROJNO	PROJNAME	DEPTNO	DEPTNAME
AD3100	ADMIN SERVICES	D01	DEVELOPMENT CENTER
AD3110	GENERAL AD SYSTEMS	D21	ADMINISTRATION SYSTEMS
AD3111	PAYROLL PROGRAMMING	D21	ADMINISTRATION SYSTEMS
AD3112	PERSONNEL PROGRAMMG	D21	ADMINISTRATION SYSTEMS
AD3113	ACCOUNT.PROGRAMMING	D21	ADMINISTRATION SYSTEMS
IF1000	QUERY SERVICES	C01	INFORMATION CENTER



This example is exploiting a relationship that exists between the department table (DEPT) and the project table (PROJ). Each project is funded by a single department. The foreign key in the PROJ table (DEPTNO) identifies the department via the primary key (DEPTNO) as is usual. In this case, the column that provides the match has the same name on both tables, but this is not necessary. The JOIN operation must identify each table being JOINed by name and specify a special predicate, the JOIN predicate, to control the matching of the rows in each table.

JOINS of more than two tables are not only possible, but quite common. Each pair of tables being JOINed must be controlled by their own JOIN predicate.

Relational Database Components



Relational Database Components

This diagram shows the three groupings of features encompassed by the relational data sub-language.

- ◆ SELECT, INSERT, UPDATE, and DELETE, which comprise the *Data Manipulation Language (DML)*
- ◆ CREATE, ALTER, and DROP, which comprise the *Data Definition Language (DDL)*
- ◆ GRANT and REVOKE, which comprise the *Data Control Language (DCL)*.

From this point on in the presentation, all language-related examples will be in terms of Structured Query Language (SQL).

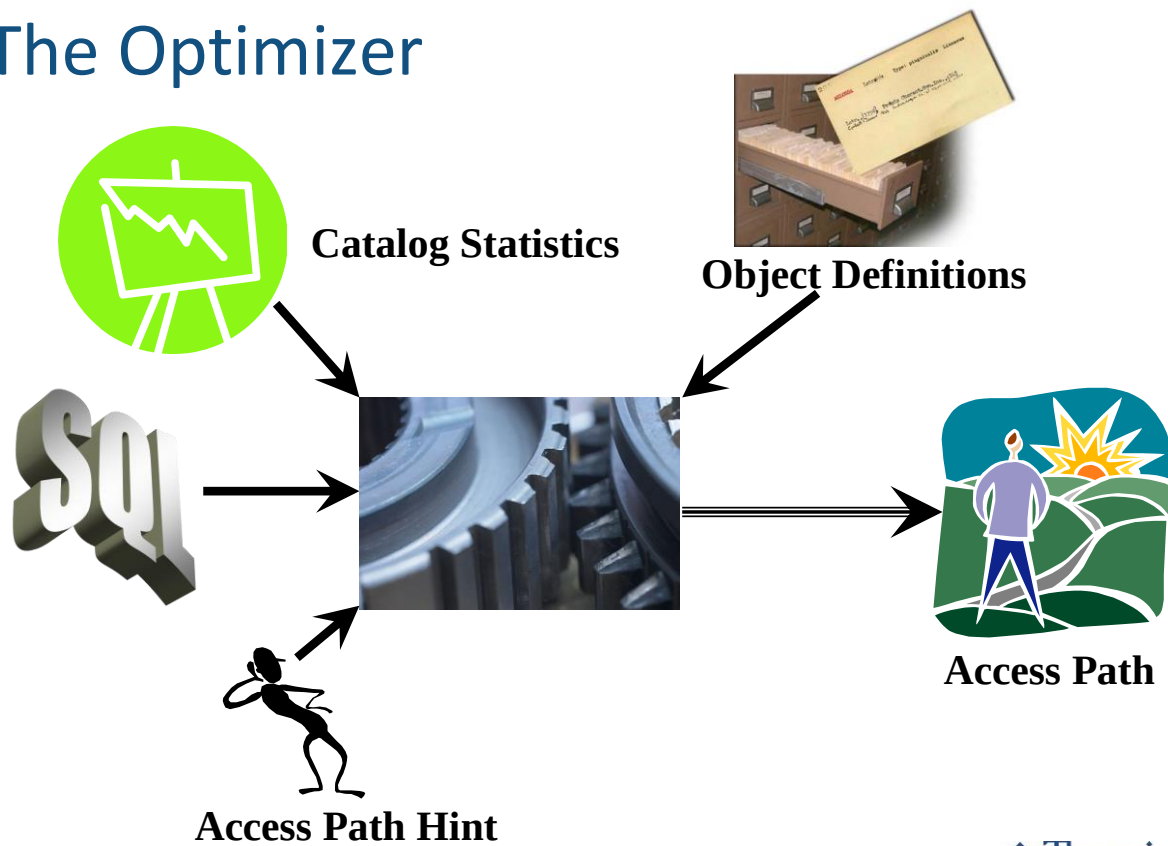
In SQL, the first word of any statement is one of the above and defines the statements intent; manipulation, definition, or control.

Also depicted in the diagram is the presence of the relational catalog. *The relational catalog is a relational database.* The tables of this database record information about user tables and all other relational objects. For instance, there is a table in the relational catalog that contains one row per table defined to the relational system, including the catalog table itself. Other tables in the catalog contain information about the columns of each table as well as the indexes and spaces that contain them. The catalog serves as a repository and knowledge base for the relational system. Its existence is part of relational theory; the relational system is providing information about itself in order to make that information available on demand.

Relational systems make heavy use of the file management and storage capabilities of the underlying operating system. *Although the operating system is responsible for reading and writing the basic units of information, the responsibility of providing the relational view rests purely with the relational system.*

Implemented relational systems contain a powerful optimizer to interpret requests made via the query language. Modern relational query languages are non-procedural, each request is the definition of a derived table representing the response. Query-language requests may be thought of as *specifications* for a table containing the result of applying the criteria defined in the statement itself. The Optimizer writes the programs associated with achieving the desired result.

The Optimizer



The Optimizer

Through the Data Manipulation Language (DML) the user of a database supplies the “WHAT”; that is, the data that is needed from the database to satisfy the business requirements. The database then uses the information in the catalog to resolve “WHERE” the data resides. The optimizer is then responsible for determining the all important “HOW” to access the data most efficiently.

Ideally, the user of a relational database is not concerned with how the system accesses data. This is probably true for an end user of the data who writes SQL queries quickly for one-time or occasional use. It is less true for developers who write application programs and transactions, some of which will be executed thousands of times a day. For these cases, some attention to access methods can significantly improve performance.

Access paths can be influenced in four ways:

- ◆ By rewriting a query in a more efficient form.
- ◆ By creating, altering, or dropping indexes.
- ◆ By updating the catalog statistics that the database uses to estimate access costs.
- ◆ By utilizing Optimizer Hints.

This page intentionally left blank

Chapter 2

Basic Data Manipulation

Table of Contents

CHAPTER 2 BASIC DATA MANIPULATION2-1

Overview.....2-3

 Objectives 2-3

CREATE TABLE Overview2-5

 Numeric Data Types..... 2-6

 Character Data Types 2-8

 Date and Time Data Types2-10

 Null Attribute2-15

 Default Values2-17

 NULL Attribute and Default.....2-19

Enforcing Business Rules.....2-21

 Table Check Constraints.....2-21

 Referential Integrity.....2-22

Basic Form of SELECT2-25

 Using SELECT *2-26

 Projecting Columns onto the Result.....2-27

 Basic Predicates2-29

 Basic Predicates Examples.....2-32

 NOT Comparisons2-33

 Expressions in the SELECT Clause.....2-35

 Renaming Columns.....2-39

 Expressions in the WHERE Clause2-41

Overview

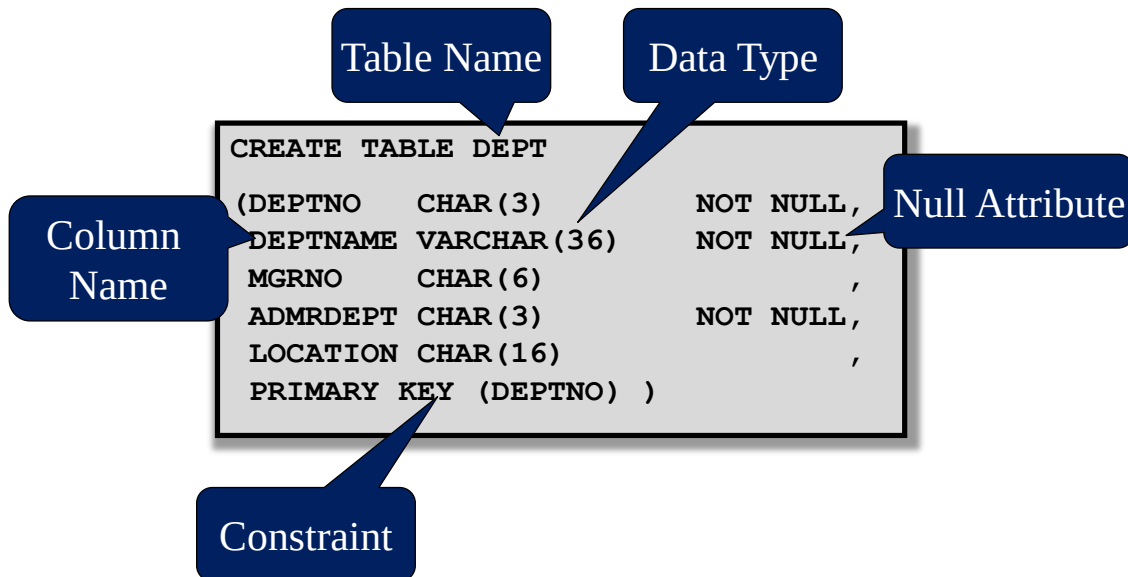
Chapter 2 presents the concepts and basic syntax of the CREATE TABLE statement along with an introduction to the data manipulation SELECT statement. Although often thought of as a physical database design topic the information contained within CREATE TABLE is necessary for anyone needing semantic information about tables. Data types, constraints and treatment of missing information will also be covered. Simple SELECTs are introduced with respect to the basic relational operations of restriction and projection in preparation for the first workshop in SQL.

Objectives

By the end of this chapter, you should:

- ◆ Understand column definitions on a table.
- ◆ Be familiar with the data types used on the major RDBMS platforms.
- ◆ Understand how a database supports missing information.
- ◆ Be able to code restricts and projects in SQL using basic predicates and expressions.

CREATE TABLE



CREATE TABLE Overview

CREATE TABLE is the data definition language statement that defines a base table. Information contained in the statement is fundamental to the eventual manipulation of the data. After creation, some of the properties of the table can be changed with **ALTER**. A table is removed from the database environment with **DROP**.

Table names may be up to 30 characters long in an Oracle database and up to 128 characters long in a Db2 or SQL Server database. Column names may be up to 30 characters in an Oracle or Db2 database and up to 128 characters in a SQL Server database. For portability it may be prudent to keep all table and column names under 30 characters.

Multiple words in a table or column name are usually separated by the underscore character (_). Tables also have a 2 part name so that more than one copy of a table may exist in an environment.

*The purpose of this brief introduction is to emphasize the information in the **CREATE TABLE** statement that is necessary for coding queries.* Keep in mind that the database does not store a wealth of information on the semantics of the data values. The responsibility for what the data actually means falls on the users of the tables.

The smallest unit of data that can be manipulated in SQL is called a value. Values are interpreted based on the data type of their source. Columns, constants, expressions, functions, host variables, and special registers are all sources of values. There is no default data type for a column.

Databases support both internally defined data types called ***Built-in Data Types*** as well as user-defined data types called ***Distinct Types***. This section will cover some of the most commonly used built-in data types on the popular platforms.

Numeric Data Types

Db2

SMALLINT
INTEGER
BIGINT

FLOAT
REAL
DOUBLE

DECIMAL

SQL Server

TINYINT
SMALLINT
INTEGER
BIGINT

FLOAT
REAL

DECIMAL
NUMERIC

Oracle

NUMBER

BINARY FLOAT
BINARY FLOAT DOUBLE

Numeric Data Types

The ANSI standard calls for several different numeric data types. A small integer is a whole number that allows +/- 32,767 values. An integer takes up more space but allows +/- 2.1 Billion values. Db2 and SQL Server implement these data types in addition to a BIGINT type that allows for very large whole numbers. SQL Server also allows for a TINYINT which allows for +/- 128 values.

Decimal columns may be specified to allow fractional numbers. The size and precision are specified as DECIMAL(x,y), where x is the total number of digits, y is the number of digits to the right of the decimal point. Db2 and SQL Server also implement the DECIMAL data type. SQL Server uses NUMERIC as an alias for DECIMAL.

Oracle implements a datatype called NUMBER that encompasses all the above datatypes and is stored in a variable length format. Oracle will tolerate the specification of all the ANSI datatypes but will internally store them as NUMBERS.

All three platforms allow for floating point decimals which use binary precision. These datatypes are outside the scope of this class.

Numeric Data Types

PostgreSQL

SMALLINT

INTEGER

BIGINT

REAL

DOUBLE PRECISION

DECIMAL

NUMERIC

PostgreSQL has all the basic numeric data types as the other RDBMS and a few others not covered in this material (like smallserial, serial, and bigserial) which are incrementing integers. These serial data types are not true types, but merely a notational convenience for creating unique numbers (much like a sequence object). Excluding these from an insert statement will get the next available value auto assigned.

NUMERIC data type has a variable storage size depending on how any exact precision is specified. It can be used as NUMERIC(Precision #, Scale #). NUMERIC by itself as a data type can hold up to 131072 digits before the decimal point; up to 16383 digits after the decimal point.

Character Data Types

Db2

CHAR
VARCHAR

GRAPHIC
VARGRAPHIC

CLOB

SQL Server

CHAR
VARCHAR

NCHAR
NVARCHAR

TEXT

Oracle

CHAR
VARCHAR2

NCHAR
NVARCHAR2

CLOB

Character Data Types

All three platforms allow for fixed length and variable length character columns to store textual data. Fixed length character data has a constant length for every row on the table. If a string is entered that is smaller than the maximum length, the column is padded with spaces. The CHAR data type is used on all three platforms for this purpose.

Variable length character columns will store only the string actually entered along with two bytes of length to determine how long the string actually is. VARCHAR is used by Db2 and SQL Server. Oracle will tolerate VARCHAR, but actually uses a newer datatype called VARCHAR2.

The GRAPHIC(Db2) and NCHAR(Oracle & SQL Server) types are used to store textual data using character sets that require a double byte format.

Character Large Object (CLOB on Db2 and Oracle, TEXT on SQL Server) is used for storing text that can be very large. The storage mechanism for this data type is generally different from traditional text data and should be used with care.

Character Data Types

PostgreSQL

CHAR or CHARACTER

VARCHAR or CHARACTER VARYING

TEXT (Unlimited Length)

PostgreSQL has all the same character data types as the other RDBMS. The TEXT type would be used as a character large object.

It also contains support for storing data in a variety of character sets (both single and double byte) and set at the database level. Following is a few of the many character sets.

Table 22-1. PostgreSQL Character Sets

Name	Description	Language	Server?	Bytes/Char
BIG5	Big Five	Traditional Chinese	No	1-2
EUC_CN	Extended UNIX Code-CN	Simplified Chinese	Yes	1-3
EUC_JP	Extended UNIX Code-JP	Japanese	Yes	1-3
EUC_JIS_2004	Extended UNIX Code-JP, JIS X 0213	Japanese	Yes	1-3
EUC_KR	Extended UNIX Code-KR	Korean	Yes	1-3
EUC_TW	Extended UNIX Code-TW	Traditional Chinese, Taiwanese	Yes	1-3
GB18030	National Standard	Chinese	No	1-4

Date & Time Data Types

Db2

DATE
TIME
TIMESTAMP

SQL Server

DATE
TIME

DATETIME

TIMESTAMP

Oracle

TIMESTAMP
DATE

INTERVAL

Date and Time Data Types

One of the biggest differences between the three platforms from an application perspective is the storage of date and time data. The ANSI standard calls for 3 datatypes:

DATE – a single date value

TIME – a time of day with precision to the second

TIMESTAMP – the combination of date and time with subsecond precision (microsecond).

Db2 implements these three datatypes.

Until 2008 SQL Server implemented a single datatype called DATETIME that is used for all three ANSI types. Both date and time are stored and a query may choose which parts of the date or time to retrieve. While the DATETIME datatype is still widely used on SQL Server, the ANSI types DATE and TIME were introduced as options in SQL SERVER 2008. The SQL Server timestamp data type has nothing to do with times or dates. SQL Server timestamps are binary numbers that help determine the sequence in which data modifications occurred.

Oracle uses the traditional timestamp as well as a DATE data type which includes a date and time with precision down to the micro seconds. Like SQL Server, the requestor may determine which parts of the date or time should be displayed.

Oracle datetime datatypes include DATE, TIMESTAMP, and TIMESTAMP WITH TIMEZONE.

Date and Time Data Types

PostgreSQL

DATE

TIME

TIME WITHOUT TIMEZONE

TIME WITH TIMEZONE

TIMESTAMP

TIMESTAMP WITHOUT TIMEZONE

TIMESTAMP WITH TIMEZONE

Date and Time Data Types

PostgreSQL has all the basic date and time formats, and also has the ability to store a time zone.

Time zones give specific areas on the earth a time of day that is earlier or later than the neighboring time zones. Greenwich Mean Time is now called UTC (Coordinated Universal Time). UTC is the time standard of the world. All other parts of the world are offset (plus or minus) according to their longitude.

NULL Attribute

Oracle SQL Server

SALARY DECIMAL(7,2) NULL

or

SALARY DECIMAL(7,2) NOT NULL

Allows Nulls

Db2

SALARY DECIMAL(7,2) NOT NULL

No Nulls
Allowed

Null Attribute

Some columns cannot have a meaningful value in every row. In other instances, all the values may not be known at the time the row is to be inserted. To handle these situations, the database uses a special value indicator to stand for an unknown or missing value. Although the term null value is commonly used, it's actually a misnomer since the entire situation arises from the fact that the column doesn't have a value. Null is not a zero value, a blank, or an empty string. It is a special indicator that the database interprets to mean that no data is present.

If you do not specify NOT NULL for a column, the default is that the column is nullable (i.e. it can indicate this unknown or missing condition). Rows can be inserted into the table without providing a value for the column.

If you use NOT NULL without using the DEFAULT clause, you must provide a non-null value for that column whenever you insert a row, that is, it is a required value.

Examples:

A *must-enter* column

```
LASTNAME  VARCHAR(15) NOT NULL
```

Two ways to allow NULLs,

```
Manager   CHAR(6)  or
```

```
Manager   CHAR(6)  DEFAULT NULL
```

In both cases the column is nullable and will indicate null if the value is not provided when inserting a row. Nullable columns can also be the target of an update whereby the column is assigned NULL.

The entire topic of missing information and the way it's implemented in SQL has been widely debated. If the decision has been made to allow NULL for a column, everyone need be aware of how this will affect the results of a query: Nulls do not satisfy any condition in an SQL statement; any comparison or expression involving nulls returns an unknown result. Conditional logic is no longer limited to the familiar true or false, formally known as a *two-valued logic* system. The presence of NULL results in a *three-valued logic* whereby the possible outcomes for the evaluation of a predicate are true, false, and *unknown*. Rows still qualify for the result based on true conditions in the where-clause predicate. Under three-valued logic, rejection from the result will occur for false and unknown conditions.

Specific rules for the manipulation of columns that are null will be developed later in the presentation.

Default Values

Oracle SQL Server

```
SALARY DECIMAL(7,2) NOT NULL DEFAULT 0
```

Db2

```
SALARY DECIMAL(7,2) NOT NULL WITH DEFAULT
```

or

```
SALARY DECIMAL(7,2) NOT NULL WITH DEFAULT 0
```

Default Values

A default value may be specified when creating a column. This is the value that the database will assign to the column when it is not specified at INSERT time.

In Db2 if WITH DEFAULT is specified without the actual value, the default will be:

- 0 for numeric columns

- Spaces for character columns

- CURRENT DATE, TIME or TIMESTAMP for date / time values

The other platforms require a default to be explicitly specified.

NULL Attribute and Default

PostgreSQL

Both
Allow Nulls

```
SALARY DECIMAL(7,2) NULL  
SALARY DECIMAL(7,2)  
or  
SALARY DECIMAL(7,2) NOT NULL
```

```
SALARY DECIMAL(7,2) NOT NULL DEFAULT 0  
UPD_TSP TIMESTAMP NOT NULL  
                DEFAULT CURRENT_TIMESTAMP or  
                DEFAULT NOW()
```

NULL Attribute and Default

Default values must be specifically specified. `NOW()` get current timestamp with timezone.

CHECK Constraints

A check condition on a column is satisfied if it does not evaluate false:

```
SALARY DECIMAL(7,2) NOT NULL  
CONSTRAINT SALTEST  
CHECK(SALARY >= 0)
```

Enforcing Business Rules

Constraints are rules that limit the values that you can insert, delete, or update in a table. Databases allow for declarative constraints so that business rules may be enforced without procedural code. There are two types of constraints:

- ◆ **Table check constraints** determine the values that a column can contain (simple domain check).
- ◆ **Referential constraints** preserve relationships between tables (referential integrity).

The database will enforce constraints when:

- ◆ A row is inserted into the table.
- ◆ A row of the table is updated.

Table Check Constraints

A check constraint is enforced by applying its search condition to each row that is inserted or updated. If the result of the search condition is false, an error occurs indicating that the row failed the test. The check condition can evaluate to unknown if the column allows nulls. An unknown result does not violate the check condition. *Check integrity is the condition that exists when each row of a table conforms to the check constraints defined on that table.* Check constraints can be named using the **CONSTRAINT** keyword. This is recommended for identification purposes, the constraint name will appear in the error message accompanying those rows failing the check. The constraint name must be unique among all other constraints defined on the table.

Example, negative salaries are not allowed:

```
CONSTRAINT SALTEST CHECK (SALARY >= 0)
or the shorthand
CHECK (SALARY >= 0)
```

The syntax of the constraint is checked when the constraint is defined, but the meaning of the constraint is not checked.

Example, allowable but problematic check constraints, assume COL1 INTEGER NOT NULL:

```
CHECK (COL1 > 8 AND COL1 < 4)
CHECK (COL1 IS NULL)
```

Referential Integrity

```
CREATE TABLE DEPT
(DEPTNO    CHAR(3)          NOT NULL ,
DEPTNAME  VARCHAR(36)      NOT NULL ,
MGRNO     CHAR(6)          ,
ADMNDEPT  CHAR(3)          NOT NULL ,
LOCATION    CHAR(16)         ,
PRIMARY KEY (DEPTNO)
FOREIGN KEY (MGRNO) REFERENCES
EMP (EMPNO)
)
```

Parent table

Column on
this table

Column on
parent table

Relational databases support
declarative referential integrity.

Concept

Referential Integrity

*Databases support declarative referential integrity through the **PRIMARY KEY** clause and the **FOREIGN KEY.....REFERENCES** clauses of the **CREATE TABLE** statement.* Additionally, these integrity roles may be added to the table after its creation with ALTER TABLE statements.

The CREATE allows for the primary key constraint to be named via the **CONSTRAINT** keyword. This name is a long identifier and appears in the error messages accompanying violations of uniqueness on INSERT and UPDATE operations. For primary key support a unique index must be created on the columns of the primary key. Some platforms will automatically create this index and some will not.

In case of referential integrity violations, the constraint name appears in the error message. Foreign keys do not require indexes but it's strongly recommended that they be considered.

This example shows the declaration of a table with a primary and foreign key. The foreign key clause also defines delete rules for the parent (primary key) row in the referenced table. Specification of these rules are beyond the scope of this presentation.

PostgreSQL can also define by using the REFERENCES key. For example:

```
MGRNO CHAR(6)REFERENCES EMP(EMPNO)
```

Basic Form of SELECT

SELECT	column(s)
FROM	table(s)
WHERE	condition(s)

The list of columns requested may contain:

- column names
- expressions
- functions
- constants
- scalar full select clauses

The WHERE clause is optional. If omitted the statement refers to all rows of the table(s).

Basic Form of SELECT

The SELECT statement is used to read data from a relational database. It embodies all of the relational operations presented in chapter 1. The statement is divided into a series of *clauses* and *keywords*. Various form and syntax requirements will be introduced with each topic on an as-needed basis.

From previous presentations, we are aware of three basic relational operations: restrict, project and join. ***The SELECT clause and the WHERE clause are the primary vehicles for restrict and project. The FROM and WHERE clause are the focus of join operations.***

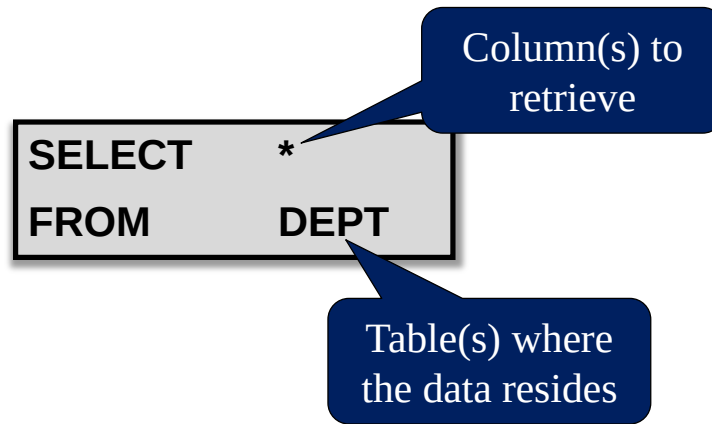
The WHERE clause contains conditional predicates and is subject to the rules for three-valued logic for those columns that are allowed to represent missing information.

Remember, data independence allows us to concentrate on table names, column names, and the domains and semantics of the values when composing queries. Storage representations and the order of occurrence of data values need not be considered.

Every SELECT statement is the definition of a table in the conceptual model. These result tables (or result sets) are derived from base tables or a combination of base tables and other derived tables.

Performance Tip. Never request more rows or columns than you actually need.

Using SELECT *



DEPTNO	DEPTNAME	MGRNO	ADMRDEPT	LOCATION
A00	SPIFFY COMPUTER SERVICE DIV.	000010	A00	<null>
B01	PLANNING	000020	A00	<null>
C01	INFORMATION CENTER	000030	A00	<null>
D01	DEVELOPMENT CENTER	<null>	A00	<null>
D11	MANUFACTURING SYSTEMS	000060	D01	<null>
D21	ADMINISTRATION SYSTEMS	000070	D01	<null>
...	...	...	...	...

Using SELECT *

`SELECT * FROM tablename` is the most minimal SQL retrieval possible. Although brief, the scope of this statement is the entire table referenced. The * is shorthand for all the columns. *Since there is no WHERE clause, the statement applies to every row in the identified table.*

The columns appear in the result in the order they appeared in the CREATE TABLE statement. This order is preserved in the database management system's catalog.

SELECT * should be avoided in application programs. While convenient in the interactive environment, statements like this tend to limit data independence when embedded in programming languages. `SELECT *` means all of the columns. Over time the table may change via ALTER statements that add columns to the base table. In embedded SQL each value returned from the table must be matched with a variable defined in the program. Since `SELECT *` may return more columns today than it did yesterday, the program is dependent on the changes made to the table.

Projecting Columns Onto the Result

- A projection of three columns of DEPT onto the result

```
SELECT    LOCATION, DEPTNAME, DEPTNO
FROM      DEPT
```

LOCATION	DEPTNAME	DEPTNO
<null>	SPIFFY COMPUTER SERVICE DIV.	A00
<null>	PLANNING	B01
<null>	INFORMATION CENTER	C01
<null>	DEVELOPMENT CENTER	D01
<null>	MANUFACTURING SYSTEMS	D11
<null>	ADMINISTRATION SYSTEMS	D21
...	...	...

- Columns appear in the order they were selected

Projecting Columns onto the Result

Projections represent the dynamic, column sensitive operations possible in relational systems. ***Only the requested columns appear in the result and in the order that they were requested.***

The SELECT clause controls this operation. Expressions are specified in a comma-delimited list with each expression becoming a column in the result table. Projections control the *degree* of the result table.

The expressions that form the columns of the result can be simple column names, arithmetic or string expressions, special registers or constants.

This example results in a subset of the columns, via the project operation in the SELECT clause. All rows of the target table (DEPT) appear due to the absence of the WHERE clause.

Basic Predicates

- A predicate specifies a condition that is either true, false, or unknown when applied to the values of each row of a table.

The format of a basic predicate is:

<expression> <comparison operator> <expression>

The comparison operators are:

=, <>, >, <, >=, <=

Example predicates:

EXPRESSION	COMPARISON OPERATOR	EXPRESSION
EMPNO	=	'000010'
SALARY / 52	>	800.00
5000	<=	SALARY / 12
HIREDATE	>	'2009-12-01'*
DEPTNO	=	'D01'

- When a predicate is used in the WHERE clause, a given row appears in the final result, if and only if the predicate evaluated to TRUE for the row.

This date format will work for Db2 and SQL Server. Oracle represents this date as '01-DEC-2009'. Other formats may be used, but must be recast. This will be covered later.

Character Comparisons: Character data needs to be compared to literal values by using single quotes around the value, or syntax errors occur: For example if coded DEPTNO = D01:

- Oracle Error : Error: ORA-01722: invalid number
- SQL Server Error: Invalid column name 'D01'
- Db2 Error: D01 is not valid in the context where it is used
- PostgreSQL: Invalid column D01. SQLSTATE 42703.

Numeric Comparisons: Numeric datatypes should be compared to literal values *without* single quotes. Db2, Oracle, SQL Server and PostgreSQL all execute implicit conversions to make sure there is no character data within the single quotes.

Basic Predicates

The WHERE clause contains predicates or search conditions that are applied to each row of the target table. Rows where the predicates evaluate to true are part of the result. Rows that evaluate to false or unknown are rejected from the result.

Conceptually, every table has an associated predicate. The *and*ing together of all the values of each row of a table produces a true condition and establishes base-line facts about the entity being represented. The WHERE clause establishes the predicate for the result table and represents the criteria for membership in that table. Values and predicates are strongly linked to the domain of the column.

A domain defines the legal values for an attribute and is thus part of the model of the real world. *A data type defines a representation of that value in a computer system* and hence is part of the implementation of that model. Even so, it makes no sense to ask for values greater than 9,999 in a column designed to hold values no larger than 999.

In any event, understanding the semantics of the values is no easy task. As stated earlier, the database does not record, or manipulate semantic data. The requirement for understanding what the data values mean and what the relationships mean in the real world is prerequisite to coding SQL and interpreting the results.

All character constants (literals) are framed with single quotes (apostrophes); numeric constants are not. Date/time constants are represented as character strings.

In addition to the comparison operators listed here, SQL also supports use of the `<>` to mean 'not'. The "NOT" keyword may also be used outside a predicate to negate the TRUE/FALSE analysis:

`SALARY <> BONUS` is the same as `NOT SALARY = BONUS`

However, `SALARY NOT = BONUS` is invalid.

Basic Predicates

Example: Return all the information about department A00

```
SELECT      *
FROM        DEPT
WHERE       DEPTNO = 'A00'
```

Result:

DEPTNO	DEPTNAME	MGRNO	ADMRDEPT	LOCATION
A00	SPIFFY COMPUTER SERVICE DIV.	000010	A00	<null>

- The predicate `DEPTNO = 'A00'` is applied to each row.
- The result contains exactly 1 row if the department exists.
- The result is empty if the department doesn't exist.

Basic Predicates

This example shows a basic predicate in the WHERE clause. *Conceptually the database applies the conditions in the WHERE clause to every row of the table(s) specified in the FROM clause.* Whenever the condition evaluates to TRUE, that row is among the rows of the result table, since they meet the conditions set forth in the predicate.

This is a conceptual evaluation because in reality the database may use indexes or other devices to evaluate the predicate before actually accessing the row. The requirement that DEPTNO='A00' may be evaluated through the use of an index. Only the qualifying rows would actually be retrieved. If no row qualified during the index search, no data access would be attempted.

Empty tables are equivalent to the concept of zero in an arithmetic system. They are just as much a table as any other table. They represent the true fact that no row qualified.

Basic Predicate Examples

```
SELECT  DEPTNAME, ADMRDEPT
FROM    DEPT
WHERE   DEPTNO = 'B11'
```

DEPTNAME	ADMRDEPT
PLANNING	A00

```
SELECT  EMPNO, FIRSTNME, LASTNAME
FROM    EMP
WHERE   EDLEVEL > 18
```

EMPNO	FIRSTNME	LASTNAME
000030	SALLY	KWAN
000110	VINCENZO	LUCCHESI

Basic Predicates Examples

These examples are further illustrations of the use of basic predicates. The fact that restrict and project are separate, independent operations are emphasized in the SQL statements.

Both examples use restrict operations in the WHERE clause but the columns doing the restriction doesn't appear in the result table. Restrict operations happen first. Once a row is qualified for the result, the required columns are projected.

The order of execution is different than the order represented in the syntax of SELECT. The implicit order of execution is:

FROM → WHERE → SELECT

The table is accessed first (either directly or through in index). The WHERE clause predicates are applied next. Finally, for each qualifying row, the projection specified in the SELECT clause is performed producing the final result.

NOT Comparisons

```
SELECT  DEPTNAME, ADMRDEPT
FROM    DEPT
WHERE   DEPTNO <> 'B11'
```

```
SELECT  DEPTNAME, ADMRDEPT
FROM    DEPT
WHERE   NOT DEPTNO = 'B11'
```

NOT Comparisons

Several NOT operators are supported. The two shown here are the most common. Both statements shown will return all departments except B11. The NOT keyword must be used on the outside of the entire predicate.

col NOT = 'value'

is not allowed.

Expressions in the SELECT Clause

Oracle
Db2

```
SELECT  EMPNO,  
        LASTNAME || ',' || FIRSTNAME,  
        SALARY,  
        BONUS,  
        SALARY + BONUS  
FROM EMP  
WHERE DEPTNO = 'A00'
```

String
Expression

Arithmetic
Expression

EMPNO		SALARY	BONUS	
000010	HAAS,CHRISTINE	52750.00	1000.00	53750.00
000011	HAAS,CHRISTINE	52750.00	1000.00	53750.00
000110	LUCCHESI,VINCENZO	46500.00	900.00	47400.00
000120	O'CONNELL,SEAN	29250.00	600.00	29850.00

Expressions in the SELECT Clause

As mentioned earlier, the SELECT clause is actually the specifications for the columns of the result table. The actual values that appear in the result can be simple column values from the source table or columns that are derived from expressions.

The most typical expressions are arithmetic, but in programming string expressions are common as well. Arithmetic expressions are formed using arithmetic operators: **+**, **-**, ***** and **/** along with parenthesis. Basic string expressions are limited to the concatenation of strings using the keyword, CONCAT or the concatenation operator **||**. Note: many powerful string operations are available using built-in functions supplied with the RDBMS. Functions are addressed later.

This example shows a combination of string, and arithmetic operations in the SELECT clause. The syntax shown here works in Db2 and Oracle. The equivalent syntax for SQL Server is:

LASTNAME + ' ' + FIRSTNME

Since the values from the string expression and the arithmetic expression don't come from the EMP table (or any table for that matter) they are represented in the result table as *unnamed columns* in Db2 and SQL Server. Oracle will show a column name in the result set with the entire expression.

Expressions in the SELECT Clause

PostgreSQL

```
SELECT EMPNO,  
       LASTNAME || ',' || FIRSTNAME,  
       CONCAT(LASTNAME, FIRSTNAME)  
       SALARY + BONUS  
FROM EMP  
WHERE DEPTNO = 'A00'
```

Expressions in the SELECT Clause

PostgreSQL uses both the concatenation '||' and the CONCAT followed by the strings to be concatenated.

This does NOT work in PostgreSQL ➔ `SELECT LASTNAME CONCAT FIRSTNME`

Renaming Columns

```
SELECT    EMPNO AS EMP_NUMBER,  
          LASTNAME || ',' || FIRSTNAME AS NAME,  
          SALARY,  
          BONUS,  
          SALARY + BONUS AS TOT_PAY  
FROM EMP  
WHERE DEPTNO = 'A00'
```

EMP_NUMBER	NAME	SALARY	BONUS	TOT_PAY
000010	HAAS,CHRISTINE	52750.00	1000.00	53750.00
000011	HAAS,CHRISTINE	52750.00	1000.00	53750.00
000110	LUCCHESI,VINCENZO	46500.00	900.00	47400.00
000120	O'CONNELL,SEAN	29250.00	600.00	29850.00

Renaming Columns

Early versions of SQL (prior to SQL86) simply ignored the condition even going so far as to providing keywords to handle results containing the unnamed columns. The current ANSI standard supports column aliases. *A column alias provides a method of naming the otherwise unnamed columns in a result table.*

This example shows the use of the column alias to provide names for the columns resulting from the use of expressions. *The alias definition follows the column expression in the **SELECT** clause.* The keyword AS is optional and can be omitted. Both of the following forms are legal and have the same effect:

```
SELECT COL1, COL2 AS COLX, COL3
```

```
SELECT COL1, COL2 COLX, COL3
```

Every column of every table should be named.

Any column in a result set may be renamed (notice that EMPNO has been renamed). Aliases, however, are most commonly used when an expression would result in an otherwise unnamed column.

Expressions in the WHERE Clause

```
SELECT    EMPNO AS EMP_NUMBER,  
          LASTNAME || ',' || FIRSTNAME AS NAME,  
          SALARY,  
          BONUS,  
          SALARY + BONUS AS TOT_PAY  
FROM EMP  
WHERE TOT_PAY > 50000
```



Expressions in the WHERE Clause

This example represents a common error resulting from the misuse of a column alias. It may seem logical at first, since TOT_PAY appears in the SELECT clause, and is the sum of the columns SALARY and BONUS. But the column named TOT_PAY is a column of the result table. Predicates coded in the WHERE clause only apply to columns from tables appearing in the FROM clause. In order to evaluate the predicate TOT_PAY > 125000.00, the database looks for the column TOT_PAY in the EMP table (the only table appearing in the FROM clause) and generates an error when it doesn't find a column named TOTAL_PAY in EMP.

It can however be used the 'Order By' clause, because it is now part of the result set that will be ordered. For Example:

```
SELECT EMPNO AS EMP_NUMBER,  
       LASTNAME || ' ' || FIRSTNAME AS NAME  
FROM EMP  
ORDER BY NAME
```

Notes:

1. Concatenation can only be done with character data
2. When defining an alias for an expression in SQL, if the alias name is to be 2 parts, then the 2 part name must be enclosed in double quotes. For example:

```
SELECT LASTNAME || ' ' || FIRSTNAME AS EMPLOYEE_NAME
```

or

```
SELECT LASTNAME || ' ' || FIRSTNAME AS "EMPLOYEE NAME"
```

Expressions in the WHERE Clause

```
SELECT    EMPNO AS EMP_NUMBER,  
          LASTNAME || ',' || FIRSTNAME AS NAME,  
          SALARY,  
          BONUS,  
          SALARY + BONUS AS TOT_PAY  
FROM EMP  
WHERE SALARY + BONUS > 50000
```

EMP_NUMBER	NAME	SALARY	BONUS	TOT_PAY
000010	HAAS,CHRISTINE	52750.00	1000.00	53750.00
000011	HAAS,CHRISTINE	52750.00	1000.00	53750.00

Expressions in the WHERE clause

Expressions in the WHERE clause can involve any column of the row being evaluated. In is example, there is interest in employees who make more than \$125,000. Since the total salary is the sum of the base salary and bonus, if any, the expression to do the calculation appears in the WHERE clause. For each candidate row, the total salary is computed and the comparison is made.

This example shows the correct way to code the predicate.

Performance Tip: Expressions in the WHERE clause when used with a column may result in performance problems and the inability to use indexes. Use with caution! For example:

```
SELECT * FROM EMP  
WHERE HIREDATE - 7 DAYS > ?
```

Should be coded as:
WHERE HIREDATE > ? + 7 DAYS

Functions and mathematics on columns in the WHERE portion of a query typically results in what is called a non-indexable predicate.

This Page Intentionally Left Blank

Chapter 3

Intermediate Data Manipulation

Table of Contents

CHAPTER 3 INTERMEDIATE DATA MANIPULATION3-1

Overview.....3-3

 Objectives 3-3

Constants3-5

System Values3-6

 Db2..... 3-7

 Oracle..... 3-7

 SQL Server 3-7

 System Values - PostgreSQL 3-9

Compound Conditions3-10

 Boolean Logic3-10

 Three-Valued Logic3-13

 Using Parentheses.....3-14

Other WHERE Clause Operations.....3-16

 The IN Comparison Operator.....3-17

 BETWEEN3-19

 LIKE3-21

 IS NULL3-23

CASE Expressions3-24

 Simple WHEN3-25

 Searched WHEN.....3-27

DISTINCT3-28

ORDER BY.....3-30

 Ordering Multiple Columns3-33

Overview

Chapter 3 continues the concepts and syntax of data manipulation with SQL. Compound conditions specified with normal boolean operators AND, OR, and NOT are presented within the framework of three-valued logic and the presence of NULLs. The basic predicates that can be specified in the WHERE clause are extended with SQL keywords and powerful conditional operators that simplify many often performed operations. The chapter concludes with a discussion of operations that only apply to the final result.

Objectives

By the end of this chapter, you should be able to:

- ◆ Code SQL statements using compound conditions implemented via boolean operators.
- ◆ Differentiate between two-valued and three-valued logic systems.
- ◆ Define the syntax and usage of the SQL CASE statement.
- ◆ Code where-clause predicates utilizing special SQL keywords for restriction
- ◆ Code SQL statements that manipulate the final result.

Constants

```
SELECT    'The Department Name is:',  
          DEPTNAME  
FROM      DEPT
```

	DEPTNAME
<i>The Department Name is:</i>	SPIFFY COMPUTER SERVICE DIV.
<i>The Department Name is:</i>	PLANNING
<i>The Department Name is:</i>	INFORMATION CENTER
<i>The Department Name is:</i>	DEVELOPMENT CENTER
<i>The Department Name is:</i>	MANUFACTURING SYSTEMS
<i>The Department Name is:</i>	ADMINISTRATION SYSTEMS
...	...

Constants

A *constant* can appear in a SELECT clause. The constant will appear in every row of the result. The data type of the column in the result takes its domain from the constant. Constants in the result were prevalent in the early days of relational databases when the technology was used primarily for report writing. The result table was the main vehicle for producing a report. This example shows a constant used for formatting. Constants today are useful in many situations, especially when relational data is being generated to feed the input requirements of older, file-based systems.

System Values

Db2

```
SELECT    CURRENT DATE
FROM      SYSIBM.SYSDUMMY1
```

Oracle

```
SELECT    SYSDATE
FROM      DUAL
```

SQL Server

```
SELECT    GETDATE ()
```

System Values

Some system values may be returned via SQL from the DBMS. The most commonly used system value is the current date or time. Each of the three vendors implement these values differently. These values may generally be used in either the SELECT clause or the WHERE clause.

Other system values may such as the userid of the person running the query will available by using similar functions.

Db2

Db2 has several special registers for date and time:

CURRENT DATE - returns the system date

CURRENT TIME – returns the current time of day

CURRENT TIMESTAMP – returns the current timestamp

CURRENT TIMEZONE – provides the difference between the system's time and GMT.

Db2 provides a special one row table for selecting special values called **SYSIBM.SYSDUMMY1**.

Oracle

Oracle has the following built in functions with similar functionality:

SYSDATE – returns the current date and time

SYSTIMESTAMP – returns the current date and time with fractional seconds as well as the timezone of the system.

DBTIMEZONE – provides either a timezone offset or a timezone name depending on installation settings.

The **DUAL** table in Oracle serves the same purpose as **SYSDUMMY1** above.

SQL Server

SQL Server has these built in functions:

GETDATE() – current date and time from the local system

GETUTCDATE() – current date and time GMT

SYSDATETIME() – more precision than **GETDATE()**

SYSUTCDATETIME() – more precision than **GETUTCDATE()**

The **FROM** clause is not required in SQL Server. No dummy 1 row table is needed.

System Values

PostgreSQL

```
SELECT NOW() → Get current TSP w /Timezone  
CURRENT_TIME  
CURRENT_TIMESTAMP  
LOCALTIME  
LOCALTIMESTAMP  
TIMEOFDAY()  
CURRENT_DATE
```

Many different system values can be returned: Can be used in any query or coded directly without any DUMMY/DUAL type table.

NOW() → Returns timestamp with timezone

CURRENT_TIME → Returns time with timezone

CURRENT_TIMESTAMP → Returns timestamp with timezone

LOCALTIME → Time without timezone

LOCALTIMESTAMP → Timestamp without timezone

TIMEOFDAY → Current date and time during the transaction. If this was coded twice you most likely see a difference in the microseconds. Output comes out as a formatted string – Thu Feb 21 11:04:38.0433865 2019 EST

CURRENT_DATE → Current date

System Values - PostgreSQL

Precision can be used when retrieving Data, Time, Timestamp data.

`SELECT LOCALTIME(0)` drops the microseconds.

TIME and TIMESTAMP data can be retrieved using precision in parenthesis that states to round up to that many fractional seconds.

There also exists:

TRANSACTION_TIMESTAMP: Same as CURRENT_TIMESTAMP

STATEMENT_TIMESTAMP: Start time of statement

CLOCK_TIMESTAMP: Returns current time, and can change during the retrieval of data

DATEPART: Returns a portion of a date or timestamp value. For example:

`Select empno, hiredate, date_part('day', hiredate)`

DATE_TRUNC: Similar to a regular TRUNC function, but you specify for a date, time, timestamp where to TRUNC. For example:

`Select date_trunc('year', current_timestamp)`

EXTRACT: Retrieves sub-fields from date and time data. For example:

`Select empno, hiredate,
extract(year from hiredate)`

Compound Conditions - OR

```
SELECT EMPNO, EDLEVEL, JOB
FROM EMP
WHERE EDLEVEL = 14 OR
      JOB = 'MANAGER'
```

EMPNO	EDLEVEL	JOB
000020	18	MANAGER
000030	20	MANAGER
000050	16	MANAGER
000060	16	MANAGER
000070	16	MANAGER
000090	16	MANAGER
000100	14	MANAGER
000120	14	CLERK
000230	14	CLERK
000300	14	OPERATOR
000330	14	FIELDREP

Compound Conditions

Boolean Logic

Predicates appearing the WHERE clause can be connected by the usual Boolean operators AND, OR, and NOT. AND and OR are binary operators; they take two arguments. NOT is a unary operator taking just one argument and negating it. This is of course how NOT works in normal two-valued logic. See the topic on the next page for information about three-valued logic and NOT.

Performance Tip. When coding a series of ANDs or Ors in a WHERE clause, always code the *most restrictive* predicates first.

Compound Conditions - AND

```
SELECT EMPNO, EDLEVEL, JOB
FROM EMP
WHERE EDLEVEL = 14 AND
      JOB = 'MANAGER'
```

EMPNO	EDLEVEL	JOB
000100	14	MANAGER

For convenience we repeat the truth tables for two-valued logic here.

AND	T	F
T	T	F
F	F	F

OR	T	F
T	T	T
F	T	F

NOT	T	F
T	F	T
F	T	F

Another basic set of identities appear here for review:

$\text{NOT (A AND B)} \equiv \text{NOT A OR NOT B}$

$\text{NOT (A OR B)} \equiv \text{NOT A AND NOT B}$

Three-Valued Logic

TRUE AND UNKNOWN → UNKNOWN
FALSE AND UNKNOWN → FALSE

TRUE OR UNKNOWN → TRUE
FALSE OR UNKNOWN → UNKNOWN

NOT UNKNOWN → UNKNOWN

A { $\begin{matrix} = \\ > \\ < \\ >= \\ <= \end{matrix} \} B$ **A { $\begin{matrix} + \\ - \\ * \\ / \end{matrix} \} B$**

Result is UNKNOWN if A is Unknown,
 B is Unknown,
 or they're both UNKNOWN

Three-Valued Logic

As mentioned earlier, when considering three-valued logic the truth table changes accordingly. The diagram on the facing page illustrates additions to the truth tables for AND and OR for inclusion of the UNKNOWN condition.

Also shown are the basic facts concerning comparability and computability involving NULL. ***Any comparison or algebraic expression involving NULL results in NULL.*** By way of example: $A = B$ and $A <> B$ are the same; they're both unknown if A or B or both are unknown.

There are some instances where comparisons involving NULL must be considered in order to have answers that are consistent with the real world. These instances are introduced later in the chapter (see the discussions on removing duplicates with SELECT DISTINCT and sorting result tables with ORDER BY for the particulars).

Once again, care should be taken whenever columns that allow NULLs appear in a WHERE clause. This is especially true for predicates involving NOT.

Using Parentheses

```
SELECT EMPNO, EDLEVEL, JOB
FROM EMP
WHERE (EDLEVEL = 14 AND
      JOB = 'MANAGER')
      OR EDLEVEL > 17
```

EMPNO	EDLEVEL	JOB
000010	18	PRES
000011	18	PRES
000020	18	MANAGER
000030	20	MANAGER
000100	14	MANAGER
000110	19	SALESREP
000140	18	ANALYST
000220	18	DESIGNER

Using Parentheses

Parentheses are used in logical statements the same way they are used in algebraic statements. Their effect is to *override* the defined order of evaluation of the operators. Algebraic order is: exponentiation first, followed by multiply/divide (considered of equal precedence), followed by add/subtract (also considered of equal precedence). When operators are of equal precedence they are simply applied in the order that they appear.

In predicate logic, ANDs are considered logical multiplying operators and ORs are considered logical summing operators. Unary NOT has the highest precedence, followed by AND, followed by OR.

Using Parentheses

```
SELECT EMPNO, EDLEVEL, JOB
FROM EMP
WHERE EDLEVEL = 14 AND
      (JOB = 'MANAGER'
      OR EDLEVEL > 17)
```

EMPNO	EDLEVEL	JOB
000100	14	MANAGER

Actually, the logical rules are exactly the same as the algebraic rules. Without parentheses, the algebraic expression:

$$A * B + C$$

Would be evaluated by doing the multiplication first and then the addition. Which is equivalent to:

$$(A * B) + C$$

Using the facts stated above about ANDs and Ors:

$$A \text{ AND } B \text{ OR } C$$

is of the same form and precedence of operators as the arithmetic statement above, and is evaluated:

$$(A \text{ AND } B) \text{ OR } C$$

In the other cases parentheses must be used, either

$$A * (B + C)$$

or logically

$$A \text{ AND } (B \text{ OR } C)$$

Other WHERE Clause Operations

- **IN**
- **BETWEEN**
- **LIKE**
- **IS NULL**

Other WHERE Clause Operations

In addition to basic predicates, the WHERE clause can contain various keyword predicates.

IN	for testing set membership
BETWEEN...AND	for specifying inclusive ranges of values.
LIKE	for partial matching (wildcard searches)
IS NULL	for identifying columns that are unknown

The operations performed by the above keywords are similar to basic predicates in that each fall into the category of conditional statements. Remember that conditional testing is somewhat different in three-valued logic and that care must be taken whenever NULLs are involved.

IN

```
SELECT    PROJNO, PROJNAME, PRSTAFF
FROM      PROJ
WHERE PROJNO IN ('AD3112', 'MA2110', 'OP2012')
```

List of Values

PROJNO	PROJNAME	PRSTAFF
AD3112	PERSONNEL PROGRAMMG	1.0
MA2110	W L PROGRAMMING	9.0
OP2012	APPLICATIONS SUPPORT	1.0

The IN Comparison Operator

The IN predicate compares a value on the left of IN, to a set of values on the right of IN. Expression2 is a set of one or more values specified by any combination of expressions. The result of IN evaluates to true if the value of expression1 is among the values of expression2. That is to say, expression1 is equal to at least one of the values represented by expression2. As with all comparisons, the values for expression1 and expression2 must be compatible, either character, numeric, or date/time.

The IN predicate in the example on the facing page is equivalent to:

```
PROJNO = 'AD3112' OR
PROJNO = 'MA2110' OR
PROJNO = 'OP2012'
```

The form PROJNO NOT IN... (or NOT PROJNO IN...) is equivalent to

```
PROJNO <> 'AD3112' AND
PROJNO <> 'MA2110' AND
PROJNO <> 'OP2012'
```

BETWEEN

```
SELECT  EMPNO, LASTNAME, SALARY
FROM    EMP
WHERE   SALARY BETWEEN 40175 AND 46500
```

Inclusive at
both ends

EMPNO	LASTNAME	SALARY
000020	THOMPSON	41250.00
000050	GEYER	40175.00
000010	LUCCHESI	46500.00

BETWEEN

The BETWEEN predicate determines whether a given value lies between two other given values specified in ascending order. Each of the predicate's two forms has an equivalent search condition, as shown below:

column1 BETWEEN *value1* AND *value2*

is the same as:

column1 >= *value1* AND column1 <= *value2*

column1 NOT BETWEEN *value1* AND *value2*

or equivalently

NOT(column1 BETWEEN *value1* AND *value2*)

is the same as

column1 < *value1* OR column1 > *value2*

As with all expressions, constants (or variables in a programming language) must agree in type with the columns involved in the comparison.

value1 and *value2* must be specified in ascending order. Thus *value1* <= *value2* must hold. This is especially of interest when the SQL statement containing the BETWEEN predicate is embedded in a programming language and *value1* or *value2* (or both) are represented by variables. If *value1* = *value2* then the statement degenerates to one of equality.

LIKE

```
SELECT  PROJNO, PROJNAME
FROM    PROJ
WHERE   PROJNO LIKE 'I%'
```

PROJNO	PROJNAME
IF1000	QUERY SERVICES
IF2000	USER EDUCATION

0 or more
unknown characters

```
SELECT  PROJNO, PROJNAME
FROM    PROJ
WHERE   PROJNO LIKE '_ _2000'
```

PROJNO	PROJNAME
IF2000	USER EDUCATION
OP2000	GEN SYSTEMS SERVICES

Exactly 1
unknown character

LIKE

The LIKE predicate searches for *strings* that have a certain pattern. The match-expression is the string to be tested for conformity to the pattern specified in pattern-expression. Underscore and percent sign characters in the pattern have a special meaning instead of their literal meanings unless escape-expression is specified, as discussed later.

The pattern is used to specify the degree of similarity for values in the match-expression where:

- ◆ The underscore character (`_`) represents any single character.
- ◆ The percent sign (`%`) represents a string of zero or more characters.
- ◆ Any other character represents a single occurrence of itself.

These examples show a pattern search for all projects where the PROJNO column starts with 'I' (example 1) or the PROJNO column has the string '2000' beginning in position 3 (example 2).

If the pattern-expression needs to include either the underscore or the percent character, the *escape-expression* is used to specify a character to precede either the underscore or percent character in the pattern.

DEPTNAME LIKE '%SYSTEMS%'	Will evaluate to TRUE if the string 'SYSTEMS' appears anywhere in the department name.
DEPTNAME LIKE '%SYSTEMS'	Will evaluate to TRUE if the string 'SYSTEMS' appears at the end of the department name.
DEPTNAME LIKE 'S%'	Will evaluate to TRUE if the department name <i>starts with an</i> 'S'.
STRING LIKE '\$_%' ESCAPE '\$'	All values of STRING that start with '_'

Other examples:

In the last example, the dollar sign '\$' has been specified as the escape character. This means that the special interpretation given to the characters '_' and '%' can be disabled by preceding them with a '\$'.

Performance Tip. Searches involving patterns where the first character is unknown can be costly because the database may be unable to effectively use indexes to evaluate the predicate.

IS NULL

```
SELECT  DEPTNO, MGRNO  
FROM    DEPT  
WHERE   MGRNO IS NULL
```

DEPTNO	MGRNO
D01	
F22	
G22	
H22	
I22	
J22	

```
SELECT  DEPTNO, MGRNO  
FROM    DEPT  
WHERE   MGRNO IS NOT NULL
```

DEPTNO	MGRNO
A00	000010
B01	000020
C01	000030
D11	000060
D21	000070
E01	000050
E11	000090
E21	000100

The NULL predicate never
returns unknown;
only true or false.

Concept

IS NULL

The NULL predicate tests a column or expression for missing information. The NULL predicate never returns unknown. If the value of the expression evaluates to unknown, the result is *true*. If the value is not unknown, the result is *false*. If NOT is specified, the result is reversed (which is to be expected since this keyword behaves in a two-valued manner). IS NOT NULL is used to eliminate NULLs in a result table and appears perhaps even more frequently than its counterpart.

Remember, null is not equal to anything, including itself. This can lead to confusing results. Statements like SALARY = SALARY, do not return all salaries, only those that are have values i.e. those that are not null. But SALARY <> SALARY returns nothing; all occurrences of SALARY <> SALARY where SALARY is unknown evaluate to unknown, the rest evaluate to false. Users of SQL simply have to be aware of the rules for using and interpreting nulls, and apply those rules to all columns allowing null, especially those in the WHERE clause.

Internally, the database implements nulls by using a tag-value format to flag the null columns with a special marker called the null byte. Each column allowing nulls contains this extra byte the value of which determines if the associated column is populated or missing. Even if the value is the empty string, the column is considered populated.

We mention this here because the whole concept of nulls is visible to the application program which has the responsibility of programming for this null byte. Application programs must create a variable to capture and test the null byte as described above, taking appropriate action based on the result of the test. This must be done for every column in the programs view that is defined as nullable.

In SQL Server, the behavior of null values will be dependent on the session setting ANSI_NULLS. When ANSI_NULLS is “on” then nulls will behave as described a above. When ANSI_NULLS is “OFF” SQL Server will return the first result shown here for the predicate MGRNO = NULL.

CASE Expressions

```
SELECT  LASTNAME,  
        CASE GENDER  
          WHEN 'M' THEN 'MALE'  
          WHEN 'F' THEN 'FEMALE'  
          ELSE 'UNKNOWN'  
        END  
FROM    EMP  
WHERE   DEPTNO = 'D21'
```

LASTNAME	
PULASKI	FEMALE
JEFFERSON	MALE
MARINO	MALE
SMITH	MALE
JOHNSON	FEMALE
PEREZ	FEMALE
PULASKI	FEMALE
JEFFERSON	MALE

CASE Expressions

Simple WHEN

The CASE expression is very powerful. CASE implements conditional logic to both the SELECT clause and the WHERE clause of an SQL statement. This conditional logic takes the common programming form:

```
IF condition 1 THEN action 1
ELSE IF condition 2 THEN action 2
ELSE IF condition n THEN action n
...
ELSE otherwise action
```

The CASE expression have the general form:

```
CASE
  WHEN condition 1 THEN action 1
  WHEN condition 2 THEN action 2
  ...
  WHEN condition n THEN action n
  ELSE otherwise action
END
```

CASE expressions are enclosed between the keywords CASE and END and can appear in the SELECT clause and the WHERE clause of an SQL statement.

A CASE expression allows an expression to be selected based on the evaluation of one or more conditions. In general, the value of the case-expression is the value of the result-expression following the first (leftmost) case that evaluates to true. If no case evaluates to true and the ELSE keyword is not present, the result is the value of the result-expression or NULL.

This example is a simple-when. A simple-WHEN specifies that the value of the expression prior to the first WHEN keyword is tested for equality with the value of each expression that follows the WHEN keyword. The data type of the expression prior to the first WHEN keyword must be comparable to the data types of each expression that follows the WHEN keywords. It also specifies the result for when that condition is true. In some programming literature, the expression preceding the first WHEN keyword is called the *case selector*.

In the example the value of the GENDER column is being translated from the abbreviation to the full name.

CASE Expressions

```
SELECT    LASTNAME,  
          CASE  
            WHEN SALARY > 100000  
              THEN 'HIGH'  
            WHEN SALARY > 50000  
              THEN 'MID'  
            ELSE 'LOW'  
          END  
FROM      EMP  
WHERE     DEPTNO = 'A00'
```

LASTNAME	
HAAS	MID
HAAS	MID
LUCCHESI	LOW
O'CONNELL	LOW

Searched WHEN

The example on the facing page is a searched-WHEN. A searched-WHEN specifies a search-condition that is applied to each row or group of table data to be evaluated, and the result when that condition is true.

Each WHEN clause specifies a search condition (subject, relational operator, and object) with the resulting action for a true condition in the THEN clause.

In the example, various salary ranges are being translated into a constant associated with the range.

DISTINCT

```
SELECT  JOB
FROM    EMP
WHERE   DEPTNO = 'D11'
```

JOB
MANAGER
DESIGNER
DESIGNER
DESIGNER
DESIGNER
DESIGNER
DESIGNER
DESIGNER
DESIGNER

```
SELECT  DISTINCT JOB
FROM    EMP
WHERE   DEPTNO = 'D11'
```

JOB
MANAGER
DESIGNER

DISTINCT

Notice that the first result contains duplicate rows. More than one employee who works in department D11 has the job title 'DESIGNER', so the result is repeated.

SELECT DISTINCT eliminates all but one of each set of duplicate rows of the result table.

NOTE: ORACLE also has SELECT **UNIQUE** JOB ...

DISTINCT

```
SELECT  DISTINCT JOB, EDLEVEL  
FROM    EMP  
WHERE   DEPTNO = 'D11'
```

JOB	EDLEVEL
DESIGNER	16
DESIGNER	17
DESIGNER	18
MANAGER	16

Two rows are duplicates of one another only if each value in the first row is equal to the corresponding value in the second row. *For determining duplicate rows, two null values at the same relative column position in two different rows are considered equal.*

Performance Tip. The use of DISTINCT may result in a sort of all qualifying rows to eliminate duplicates. Indexes on the affected columns may mitigate this. Use with caution!

ORDER BY

```
SELECT    LASTNAME, FIRSTNME, SALARY
FROM      EMP
WHERE     DEPTNO = 'D11'
ORDER BY  SALARY
```

LASTNAME	FIRSTNME	SALARY
JONES	WILLIAM	18270.00
WALKER	JAMES	20450.00
SCOUTTEN	MARILYN	21340.00
PIANKA	ELIZABETH	22250.00
YOSHIMURA	MASATOSHI	24680.00
ADAMSON	BRUCE	25280.00
BROWN	DAVID	27740.00
LUTZ	JENNIFER	29840.00
STERN	IRVING	32250.00

ORDER BY

The ORDER BY clause specifies an ordering of the rows of the result table. If a single sort-key is identified, the rows are ordered by the values of that sort-key.

ORDER BY

```
SELECT LASTNAME, FIRSTNME, SALARY
FROM EMP
WHERE DEPTNO = 'D11'
ORDER BY SALARY DESC
```

LASTNAME	FIRSTNME	SALARY
STERN	IRVING	32250.00
LUTZ	JENNIFER	29840.00
BROWN	DAVID	27740.00
ADAMSON	BRUCE	25280.00
YOSHIMURA	MASATOSHI	24680.00
PIANKA	ELIZABETH	22250.00
SCOUTTEN	MARILYN	21340.00
WALKER	JAMES	20450.00
JONES	WILLIAM	18270.00

DESCending

Each individual expression in the sort key can be either ascending or descending by specifying the keywords ASC, or DESC immediately following the column in the ORDER BY clause. ASC is the default if none is specified.

ORDER BY

```
SELECT  LASTNAME, FIRSTNAME, SALARY
FROM    EMP
WHERE   LASTNAME LIKE 'S%'
ORDER BY LASTNAME ASC, FIRSTNAME DESC
```

ASCending is
the default

LASTNAME	FIRSTNAME	SALARY
SCHNEIDER	ETHEL	26250.00
SCOUTTEN	MARILYN	21340.00
SETRIGHT	MAUDE	15900.00
SMITH	PHILIP	17750.00
SMITH	DANIEL	19180.00
SPENSER	THEODORE	26150.00
STERN	IRVING	32250.00

Ordering Multiple Columns

The other option in the syntax is an expression in the ORDER BY. This allows for the following:

```
SELECT  SALARY, BONUS ...  
FROM EMP  
ORDER BY SALARY + BONUS
```

Although not allowed in ANSI SQL, all three major database vendors allow columns to appear in the ORDER BY that do not appear in the select list.

```
SELECT  DEPTNO, EMPNO, LASTNAME...  
FROM EMP  
ORDER BY DEPTNO, LASTNAME, SALARY
```

would work and save the expense of returning an additional column if it is not needed.

Unnamed columns are permitted in the result. To include these unnamed columns in the ORDER BY an integer in the range of 1 to the number of columns in the result table, could be specified. The integer *n* would represent the *n*th column of the result. The WHERE clause in example on the facing page could be coded with the integer form of ORDER BY as:

```
ORDER BY 1, 3 DESC
```

with 1 representing the relative position of LASTNAME and 3 the relative position of SALARY in the result. This technique should be avoided. The ANSI standard has eliminated the option and the database vendors are likely to do the same sometime in the future. Any unnamed column in the result can be named with an alias. The alias represents a column of the result and the ORDER BY applies to the result.

Performance Tip. Be careful of the following:

```
SELECT  EMPNO, DEPARTMENT, SALARY  
WHERE   DEPARTMENT = 'A00'  
ORDER BY DEPARTMENT, EMPNO, SALARY
```

There's only one department in the result and EMPNO is unique, the sort key could be reduced by specifying
ORDER BY EMPNO and obtaining the same order.

This page intentionally left blank

Chapter 4

Built-in Functions

Table of Contents

CHAPTER 4 BUILT-IN FUNCTIONS4-1

Overview.....4-3

 Objectives 4-3

Introduction to Built-In Functions.....4-5

The CAST Function.....4-6

Conversion Functions4-9

String Functions4-11

 Parsing Text.....4-11

 Text Length4-13

 Other String Manipulation Functions.....4-15

Numeric Functions.....4-17

Displaying Dates4-19

 Db2 Date Formats4-19

 SQL Server Dates4-21

 Common Oracle Date Formats:4-22

 Oracle TO_CHAR Function.....4-23

 PostgreSQL Date Functions4-26

 PostgreSQL TO_CHAR Function4-27

Date Arithmetic.....4-29

 Date Subtraction4-29

 Db2.....4-30

 SQL Server4-31

 Oracle.....4-33

 PostgreSQL.....4-35

Removing NULLs4-37

Ranking and Numbering Rows4-38

 Rank Function.....4-38

 Dense Rank Function.....4-39

 Row Numbering.....4-41

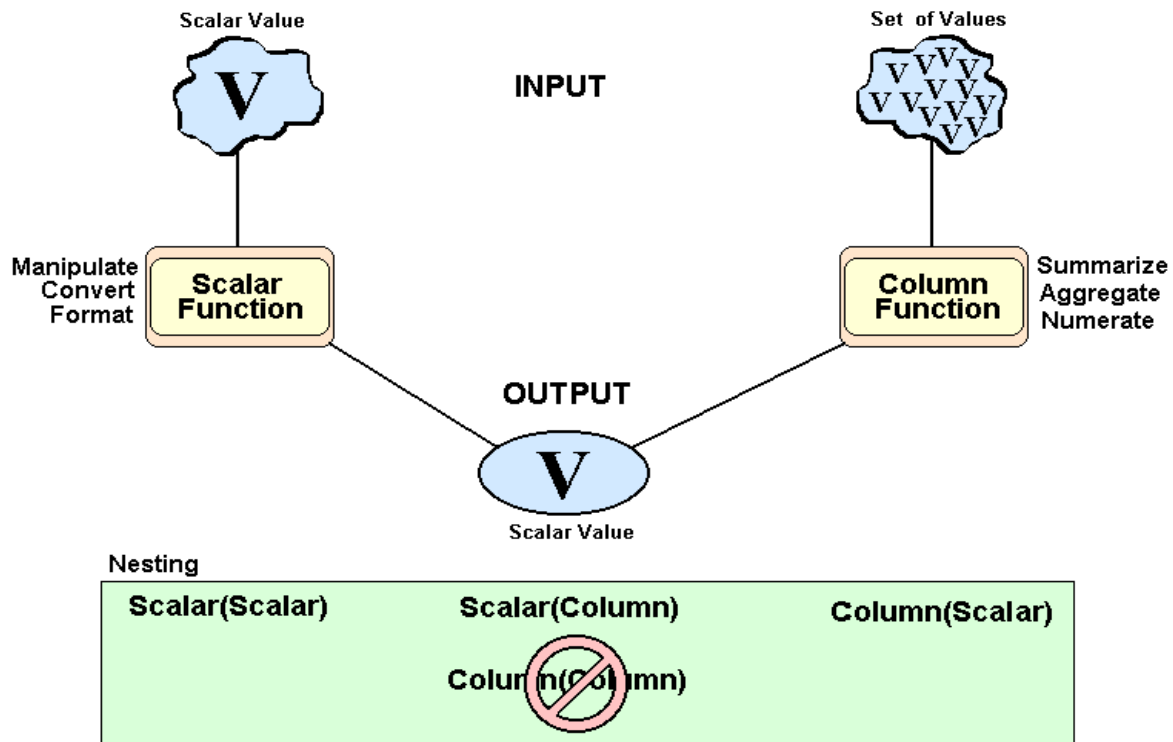
Overview

This chapter differentiates between the two kinds of built-in functions supported by relational databases; scalar functions and column functions. The presentation includes a survey of the most frequently encountered scalar functions, organized by family. String functions, numeric functions and date/time functions are covered in terms of both manipulation and conversion. Column functions will be covered in the next chapter.

Objectives

- ◆ By the end of this chapter, you should be able to:
- ◆ State the difference between scalar functions and column functions.
- ◆ Describe the benefits of the CAST function as an alternative to other conversion functions
- ◆ Define the basic families of string, numeric, and date/time functions
- ◆ Remove NULLs from a result set

Introduction to Built-In Functions



Introduction to Built-In Functions

Built-in functions are supplied with relational databases. A built-in function is composed of a function name followed by one or more operands that are enclosed in parentheses. The operands, called arguments are specified by expressions. ***The result of a built-in function is a single value (scalar value) derived by applying the operation of the function to the arguments.***

As shown in the diagram above, there are two classifications of built-in functions: ***scalar functions*** and ***column functions***. Although both produce scalar values as output, the input arguments differ. A scalar function takes a scalar value as input while a column function (sometimes called a group function) takes a set of values as input.

In the SQL syntax, the term function is used only when defining an expression. This means that a function can be used *only* where an expression is used. Functions can be nested in any combination as long as the output of one function is suitable as input to the function in which it's nested. The exception is that ***a column function cannot be nested directly inside of another column function.***

There is a great deal of divergence in the functions supported by each database vendor. Some functions with different names are duplicates of one another. This redundancy is caused by functions supported by a particular platform prior to their adoption by the ANSI standard under different names. Built-in functions are documented in the SQL Reference guides for each platform.

Scalar functions can be classified as:

- ◆ Those that convert from one data type to another
- ◆ Those that manipulate the format of a data type

Within these classifications, scalar functions can be designated as those that take:

- ◆ character string arguments,
- ◆ numeric arguments,
- ◆ date/time arguments,
- ◆ null arguments (either as input or output).

Due to the scope of the functions available, this presentation will limit coverage to a survey of those functions that are most frequently used, and those that can serve as models for others that are less frequently used.

The CAST Function

Assumption: CHARCOL is CHAR(10)
DECIMALCOL is DECIMAL(7, 2)

Change from CHAR(10) to CHAR(20):

CAST (CHARCOL AS CHAR(20))

Change from DECIMAL(7, 2) to DECIMAL (9, 3):

CAST (DECIMALCOL AS DECIMAL(9, 3))

Change from DECIMAL(7, 2) to Integer:

Cast (DECIMALCOL AS INTEGER)

The CAST Function

There are many occasions when a value with a given data type needs to be *cast* (changed) to a different data type or to the same data type with a different length, precision, or scale. *A data type that can be changed to another data type is castable from the source data type to the target data type.*

Some platforms support individual scalar functions to change data types as described above. For instance, the function VARCHAR can be used to convert fixed-length character strings to variable-length, DECIMAL can convert numbers to integers, or from one precision and scale to another perhaps larger precision and scale.

The CAST Function

```
SELECT EMPNO,
        CAST (EMPNO AS INTEGER) AS ENOI
FROM EMP
WHERE DEPTNO = 'A00'
```

Column Defined
as CHAR(6)

EMPNO	ENOI
000010	10
000011	11
000110	110
000120	120

The single function called CAST is universally supported and should be favored for converting data types.

In Db2, you will often see the function coded directly on the column and works exactly like the CAST:

```
SELECT INT(EMPNO), YEAR(HIREDATE), DEC(SALARY,9,3)
```

Conversion Functions

Db2

```
SELECT  EMPNO,  
        INTEGER(EDLEVEL)  
....
```

Oracle

```
SELECT  EMPNO,  
        TO_NUMBER(EDLEVEL)  
...
```

**Sql Server
PostgreSQL**

```
SELECT  EMPNO,  
        CAST(EDLEVEL AS INT)  
...
```

Db2 and Oracle can also execute the CAST logic, but the others must code the CAST logic.

Conversion Functions

There are also platform specific functions for converting from one data type to another. The first example shows the use of a Db2 function that converts a character string (which contains only numbers) to the equivalent integer.

The second example shows the equivalent function in Oracle which results in a column with the NUMBER datatype.

These functions will work only on columns that contain numeric data, and will also work on columns defined as character (as long as the column contains numeric data).

As shown applying those functions directly will not work in SQL Server or PostgreSQL.

Parsing Text

```
SELECT EMPNO,  
        LASTNAME,  
        SUBSTR(LASTNAME,1,4)  
FROM EMP  
WHERE DEPTNO = 'E21'
```

Db2
Oracle

SUBSTRING
for SQL Server

EMPNO	LASTNAME	
000100	SPENSER	SPEN
000320	MEHTA	MEHT
000330	LEE	LEE
000340	GOUNOT	GOUN

Starting Position
and Length

SQL Server

PostgreSQL

String Functions

Parsing Text

ANSI standards call for a function called SUBSTRING that will parse text and pick out the desired characters. Db2 and Oracle implement this function as SUBSTR, while SQL Server calls it SUBSTRING. The function accepts 3 arguments; the column or expression that contains the text to be parsed, the starting position and the length.

Like most scalar functions, substring may be used in either the SELECT clause or in the WHERE clause. Performance problems may arise when functions are placed on columns in the WHERE clause.

Performance Tip. SUBSTR like any other column manipulation may affect the ability of a database to use an index. If you were interested in seeing all departments that started with the letter 'D', use LIKE as in

WHERE DEPTNO LIKE 'D%'

rather than

WHERE SUBSTR(DEPTNO, 1, 1) = 'D'

Direct comparisons are usually better than ones which manipulate a column before the comparison.

NOTE: On most platforms, applying a function on a column in the 'WHERE' portion of the SQL statement may keep the optimizer from choosing to use an index on the column containing the function. Best practices state to avoid this whenever possible.

Text Length

```
SELECT EMPNO,  
        LASTNAME,  
        LENGTH(LASTNAME) AS LEN  
FROM EMP  
WHERE DEPTNO = 'E21'
```

Db2
Oracle

LEN
for SQL Server

EMPNO	LASTNAME	LEN
000100	SPENSER	7
000320	MEHTA	5
000330	LEE	3
000340	GOUNOT	6

SQL Server

PostgreSQL

3 use LENGTH, yet SQL server uses only LEN.

Text Length

The LENGTH function for Db2 and Oracle will determine the actual length of a variable length character column. If this function is used with a fixed length character column, the result will always be the same since the column size is fixed (e.g. LENGTH on a CHAR(10) column will *always* return a value of 10).

SQL Server implements a similar function called LEN. A major difference is that LEN will also work on fixed length columns and any trailing spaces will be truncated.

Other String Manipulation

UPPER(string) ← Convert to upper case

LOWER(string) ← Convert to lower case

RTRIM(string) ← Remove trailing blanks

RTRIM(string, 'Q') ← Remove trailing Qs

LTRIM(string) ← Remove leading blanks

Oracle
PostgreSQL

Both Oracle and PostgreSQL can use the RTRIM for characters other than space.

Other String Manipulation Functions

The **RTRIM** function removes blanks from the end of a string expression; the **LTRIM** function removes blanks from the beginning. The length attribute of the result is the same as the length attribute of *string-expression*. The actual length of the result is the length of the expression minus the number of characters removed. ***If all of the characters are removed, the result is an empty string.*** If the argument can be null, the result can be null; if the argument is null, the result is unknown.

The **UPPER** and **LOWER** functions will change the case of a character string to all upper or lower case as desired. These functions can be useful for doing case-insensitive searches:

```
SELECT *
FROM EMP
WHERE UPPER(LASTNAME) = 'SMITH'
```

Note that applying the **UPPER** function will cause the optimizers to not choose the index on **LASTNAME** (if one exists). DBAs can however create a function based index on the **UPPER(LASTNAME)**.

Oracle has a **TRIM** function that will trim spaces both leading and trailing.

Oracle and PostgreSQL can use the **RTRIM** function for other characters beside spaces.

Bonus: How could you combine the RTRIM and LENGTH functions for Db2, Oracle, or PostgreSQL to find the actual length (excluding trailing spaces) of a fixed length character string?

Numeric Functions

```
SELECT    SALARY / 12  
FROM      EMP  
WHERE     EMPNO = '000100'
```

 2179.16666666

```
SELECT    ROUND(SALARY / 12,2)
```

 2179.17000000

```
SELECT    CAST (ROUND(SALARY / 12,2)  
          AS DECIMAL(9,2))
```

 2179.17

CEIL or CEILING can be used in Oracle and Db2.

CEILING *must* be used in SQL Server and PostgreSQL.

DECIMAL cast is rounded automatically by Oracle, SQL Server, and PostgreSQL. Truncated by DB2.

For example: CAST (2.71828 AS DECIMAL(5,2)) will return 2.72 or 2.71 for Db2.

The **ROUND** function returns *expression1* rounded to *expression2* places to the right of the decimal point if *expression2* is positive or to the left of the decimal point plus one if *expression2* is zero or negative (i.e. ROUND(748.58,-2) returns 700).

Numeric Functions

The **CEIL**(Oracle and Db2) or **CEILING** (SQL Server, PostgreSQL, or Db2) function returns *the smallest integer value* that is greater than or equal to the argument. The **FLOOR** function returns the *largest integer value* that is less than or equal to the argument. The arguments to both CEIL and FLOOR are expressions that return values of any built-in numeric data type. The results have the same data type and length attribute as their arguments. The results can be null.

The example on the facing page shows various results based on a decimal divide of SALARY/12. The divisor is an integer constant. The result is decimal based on the data type of SALARY. The first example shows the default precision resulting from the division. As shown in the remaining examples ROUND performs as expected but does not affect the precision of the result. DECIMAL is required to reduce the default precision. **CAST(SALARY/12 as DECIMAL(9, 2))** could also have been used.

These functions are particularly useful in programming in order to return a value of a specific precision and scale.

When casting a number to a datatype with lower precision:

```
CAST (2.71828 AS DECIMAL(5,2))
```

Oracle, SQL Server, and PostgreSQL will round the result to 2.72. Db2 will truncate the result to 2.71.

If truncation is desired on **Oracle or PostgreSQL**, the **TRUNC** function may be used:

```
TRUNC(expression, # of places)
```

For example:

```
TRUNC (2.71828 , 2)  will return 2.71
```

If truncation is desired on SQL Server, the ROUND function may be used with a third argument of 1 indicating truncation (0 is the default and indicates normal rounding):

```
ROUND(expression, # of places, 1)
```

For example:

```
ROUND(2.71828,2,1)  will return 2.71000
```

Oracle and PostgreSQL: Round(1234.567,1) returns 1234.6 (no trailing zeros).

Numeric Functions

Assume INTEGERCOL is defined as INTEGER and has the value 10; the following results can be expected:

INTEGERCOL / 3	= 3
INTEGERCOL / 3.0	= 3.333333333333333
CAST ((INTEGERCOL / 3) AS DECIMAL(7,3))	= 3.000
CAST(INTEGERCOL AS DECIMAL(7,3)) / 3	= 3.333333333333333
CAST(INTEGERCOL/3.0 AS DECIMAL(7,3))	= 3.333

Db2 **SQL Server** **PostgreSQL**

Numeric Functions

The example on the facing page shows a column, **INTEGERCOL**, participating in several arithmetic operations. The first example shows the result of dividing an integer by another integer. *At least one operand must be decimal if a decimal result is expected.*

The DECIMAL function is used too late in the third example. The integer result of the divide is simply converted to decimal.

All of the other examples suffice to return the correct decimal result.

Oracle: In order to see leading trailing zeros in the decimal places (for example you want to see .10 and not just .1) use the TO_CHAR function on your result:

```
to_char(cast(salary/12 as dec(7,2)), '99990.00')
```

Db2 Date Functions

A date of February 14, 1990:

1990-02-14 ← International Standards Org. (ISO)

02/14/1990 ← USA Format

14.02.1990 ← EUR Format

```
SELECT CHAR(datecol,ISO)
```

Db2

Displaying Dates

Db2 Date Formats

Db2 recognizes 3 date formats. The default display format is chosen by the installer. The formats are shown here. Regardless of which format is chosen as the default, dates may be entered in any of the valid formats and may be converted for display using the CHAR function as shown.

SQL Server Dates

SQL Server Date Format is a Date / Time:
1990-02-14 00:00:00.000

```
SELECT CONVERT(CHAR(10),datecol,126)
```

Code 126 yields:
1990-02-14

SQL Server

The CHAR(10) ensures that only the first 10 bytes of the Date / Time returned result are brought back. Depending on the format of the date you want back, this may need to be adjusted.

SQL Server Dates

SQL Server stores both date and time. When it is desirable to display only the date, the CONVERT function may be used to display the date in a variety of formats. In this example, format 126 is used to display the date in YYYY-MM-DD format. Consult the SQL Server documentation for a list of formats and codes.

When inserting and comparing dates, any of the valid formats may be used:

```
SELECT LASTNAME, HIREDATE
FROM EMP
WHERE HIREDATE > '1980-01-01'
```

Other Common Formats:

```
SELECT EMPNO,
       CONVERT(char(12), HIREDATE, 107)
...
```

displays Feb 14, 1990

```
SELECT EMPNO,
       CONVERT(char(12), HIREDATE, 101)
...
```

displays 02/14/1990

Oracle TO_CHAR Function

```
SELECT TO_CHAR(datecol, 'MM/DD/YYYY')
```

 Oracle

Common Oracle Date Formats:

Date Formats
YEAR – Year spelled out
MONTH – Full name of month
MON-3 letter abbreviation for month
DY – 3 letter abbreviation for day of week
DAY – full name of week day
DD-Numeric day of month

Oracle TO_CHAR Function

Oracle includes a very powerful function for displaying dates in any desired format. The TO_CHAR function may be used with any allowable combination of YYYY, MM & DD to display the date in any format.

The default date expected by Oracle is YY-MMM-DD. For example:

90-FEB-14

Examples: Note dashes, spaces, commas, slashes:

```
SELECT EMPNO, TO_CHAR(HIREDATE, 'YYYY MM DD')  
FROM EMP
```

```
SELECT EMPNO, TO_CHAR(HIREDATE, 'YYYY Month DD')  
FROM EMP
```

```
SELECT EMPNO, TO_CHAR(HIREDATE, 'YYYY MM Day')  
FROM EMP
```

```
SELECT EMPNO, TO_CHAR(HIREDATE, 'Month Year DD')  
FROM EMP
```

```
SELECT EMPNO, TO_CHAR(HIREDATE, 'Month DD, YYYY')  
FROM EMP
```

Oracle: To return only the current date or only the current time:

```
TO_CHAR(SYSDATE, 'YYYY-MM-DD') AS "DATE" ,
```

```
TO_CHAR(SYSDATE, 'HH24:MI:SS') AS "TIME"
```

Oracle TO_CHAR Function

ORACLE NON INDEXABLE PREDICATE:

WHERE TO_CHAR(HIREDATE, 'YYYY MM DD') = '1970 01 01'

ORACLE INDEXABLE PREDICATE:

WHERE HIREDATE = TO_DATE('1970-01-01', 'YYYY-MM-DD')

Note: Functions on columns in predicates typically cause the predicate to be non-indexable, unless a function based index gets created.

Oracle TO_CHAR Function

Oracle dates contain both date and time. Data in these columns may default to times of all zeros, and other data with actual times. When coding for rows that contain a specific date, you have to be careful when coding the logic in order to get the best possible performance.

Other examples:

```
WHERE HIREDATE = TO_DATE('1970-01-01', 'YYYY-MM-DD')  
OR WHERE TRUNC (HIREDATE) = TO_DATE ('1970-01-01', 'YYYY MM DD')
```

should be coded as:

```
SELECT EMPNO, LASTNAME, HIREDATE  
FROM EMPLOYEE  
WHERE HIREDATE BETWEEN TO_DATE( '01-JAN-1970 00:00:00',  
                                'DD-MON-YYYY HH24:MI:SS' )  
AND TO_DATE( '01-JAN-1970 23:59:59',  
             'DD-MON-YYYY HH24:MI:SS' )
```

```
WHERE TO_CHAR(HIREDATE, 'YYYY MM DD') = '1970 01 01'
```

could be efficiently coded as follows but only if the time portion of the data contained zeros:

```
WHERE HIREDATE = TO_DATE('1970-01-01', 'YYYY-MM-DD')
```

PostgreSQL Date Functions

A date of February 14, 1990 will automatically be returned as: 1990-02-14, unless you apply the TO_CHAR function with a format.

```
SELECT TO_CHAR( datecol , 'MM/DD/YYYY')
```

PostgreSQL

PostgreSQL Date Functions

There are many ways to return a date in PostgreSQL, these are a few:

```
SELECT EMPNO, TO_CHAR(HIREDATE, 'YYYY MM DD')
```

```
SELECT EMPNO, TO_CHAR(HIREDATE, 'YYYY Month DD')
```

```
SELECT EMPNO, TO_CHAR(HIREDATE, 'YYYY MM Day')
```

```
SELECT EMPNO, TO_CHAR(HIREDATE, 'Month Year DD')
```

```
SELECT EMPNO, TO_CHAR(HIREDATE, 'Month DD, YYYY')
```

To check a date, use the TO_DATE function:

```
WHERE HIREDATE > TO_DATE ('1960 01 01', 'YYYY MM DD')
```

PostgreSQL TO_CHAR Function

PostgreSQL NON INDEXABLE PREDICATE:

WHERE TO_CHAR(HIREDATE, 'YYYY MM DD') = '1970 01 01'

PostgreSQL INDEXABLE PREDICATE:

WHERE HIREDATE = TO_DATE('1970-01-01', 'YYYY-MM-DD')

Note: Functions on columns in predicates typically cause the predicate to be non-indexable, unless a function based index gets created.

PostgreSQL TO_CHAR Function

Same as with ORACLE, you need to watch how date logic is coded. Best practices state to not code any functions or mathematics directly on columns in the WHERE portion of the query or you may lose what is called indexability.

For best performance use the TO_DATE function on the date value and not the TO_CHAR function on the column.

Date Arithmetic

Return the number of days between two dates:

```
SELECT DAYS(HIREDATE) - DAYS(BIRTHDATE)  
FROM   EMP  
WHERE  EMPNO = '000100'
```

Db2

```
SELECT HIREDATE - BIRTHDATE  
FROM   EMP  
WHERE  EMPNO = '000100'
```

Oracle

PostgreSQL



8584 Days

Different ways to get a days difference between 2 dates depends on the RDBMS.

Date Arithmetic

Date Subtraction

When two dates are subtracted using Oracle or PostgreSQL, the result is an integer representation of the number of days that have passed between the two dates.

When dates are subtracted using Db2, the result is a *date duration* that is not covered here but is a number representing the number of years, months, and days difference together.

To determine the number of days that have passed between two dates using Db2, the DAYS function should be used. DAYS takes in a date and returns the number of days that have passed since 0001-01-01. This is usually used as shown to compare to another DAYS value and determine how many days have passed.

```
SELECT CAST (HIREDATE - BIRTHDATE AS INTEGER)
```

SQL Server uses a DATEDIFF function for the same purpose:

```
SELECT DATEDIFF(DAY,BIRTHDATE,HIREDATE)
```

Oracle : To add days to a date, just add as follows:

```
SELECT EMPNO, HIREDATE + 7  
FROM EMP
```

PostgreSQL : To add days to a date, just add as follows:

```
SELECT EMPNO, HIREDATE + 7 ➔ or SELECT EMPNO, HIREDATE + 7 DAYS  
FROM EMP
```

```
SELECT EMPNO, HIREDATE + 1 MONTHS or HIREDATE = 1 YEARS
```

Date Arithmetic – Db2

```
SELECT  HIREDATE + 1 MONTH + 5 DAYS,  
        HIREDATE  
FROM    EMP  
WHERE   EMPNO = '000100'
```

Db2

	HIREDATE
1980-07-24	1980-06-19

Db2

Db2 uses expressions called *labeled durations* to add or subtract a time value to a date. The result is always another date. In this example, 1 month and 5 days are being added to the hiredate. The database understands all the rules for valid dates, so the result will always be a valid date.

Sometimes a date must be adjusted to ensure that it is valid. For example, '2010-01-31' + 1 MONTH will yield a result of '2010-02-28'.

Date Arithmetic – SQL Server

```
SELECT  DATEADD(MONTH,1,HIREDATE),
        HIREDATE
FROM    EMP
WHERE   EMPNO = '000100'
```

SQL Server

	HIREDATE
1980-07-19 00:00:00.000	1980-06-19 00:00:00.000

```
SELECT  DATEADD(DAY,5,
        DATEADD(MONTH,1,HIREDATE) )
```

Adds 1 month
5 days

SQL Server

SQL Server uses the DATEADD function to perform the same functions as the labeled duration in Db2. The function may be nested multiple times to add combinations of years, months and days. Negative numbers may be used to subtract periods of time.

Date Arithmetic - Oracle

```
SELECT  ADD_MONTHS(HIREDATE,1)
        HIREDATE
FROM    EMP
WHERE   EMPNO = '000100'
```

Oracle

	HIREDATE
1980-07-19 00:00:00.000	1980-06-19 00:00:00.000

```
SELECT  ADD_MONTHS(HIREDATE,1) + 5
```

Adds 1 month
5 days

Oracle

In Oracle, adding a constant to a date will be assumed to be a number of days. Oracle also implements an `ADD_MONTHS` function to add (or subtract) a number of months from a date.

`TO_DATE`:

In order to add time values to a literal date, the `TO_DATE` function must be used to first validate the date and convert it then to a `DATE` datatype. For Example:

```
SELECT ADD_MONTHS(TO_DATE('2011-12-12', 'YYYY-MM-DD'), 4)
FROM DUAL
```

When using the `TO_DATE` function, you must also provide the format of the inputted data or an error occurs., unless it is the default date format as follows:

```
SELECT ADD_MONTHS(TO_DATE('12-Dec-2011'), 4)
FROM DUAL
```

;

Remember:

`TO_DATE` converts character data into a valid `DATE`.

`TO_CHAR` converts `DATE` into character data.

Ex) `WHERE HIREDATE >= TO_DATE('2016-07-10', 'YYYY-MM-DD')`

Oracle: To add days to a date, just add as follows:

```
SELECT EMPNO, HIREDATE + 7
FROM EMP
```

```
select empno, hiredate,
       last_day(hiredate) as "Last Day of Month"
from emp
;
```

Date Arithmetic - PostgreSQL

```
SELECT  HIREDATE + INTERVAL '1 MONTHS'  
        HIREDATE  
FROM    EMP  
WHERE   EMPNO = '000100'
```

	HIREDATE
1980-07-19 00:00:00.000	1980-06-19 00:00:00.000

PostgreSQL

```
SELECT HIREDATE + INTERVAL '1 MONTHS 5 DAYS'
```



Adds 1 month
5 days

PostgreSQL uses the INTERVAL function to add years, months, days to a date.

PostgreSQL

In PostgreSQL, adding a constant to a date will be assumed to be a number of days. PostgreSQL also allows for a constant followed by (MONTHS, DAYS, YEARS) but the constant by itself defaults to DAYS. PostgreSQL also implements an INTERVAL function to add (or subtract) a number of YEARS, months, and days together in same calculations.

TO_DATE:

In order to add time values to a literal date, the TO_DATE function must be used to first validate the date and convert it then to a DATE datatype. For Example:

```
SELECT TO_DATE('1970-03-31', 'YYYY-MM-DD') + interval '1 months'
```

When using the TO_DATE function, you must also provide the format of the inputted data or an error occurs., unless it is the default date format as follows:

Remember:

TO_DATE converts character data into a valid DATE.

TO_CHAR converts DATE into character data.

Ex) WHERE HIREDATE >= TO_DATE('2016-07-10', 'YYYY-MM-DD')

PostgreSQL: To add days to a date, just add as follows. By havinf just the constant the default is DAYS or you could code HIREDATE + 7 DAYS:

```
SELECT EMPNO, HIREDATE + 7  
FROM EMP
```

PostgreSQL does not have a LAST_DAY function.

Removing Nulls

```
SELECT  DEPTNAME,  
        COALESCE (MGRNO, 'No Manager')  
FROM    DEPT
```

DEPTNAME	
SPIFFY COMPUTER SERVICE DIV.	000010
PLANNING	000020
INFORMATION CENTER	000030
DEVELOPMENT CENTER	No Manager
MANUFACTURING SYSTEMS	000060
ADMINISTRATION SYSTEMS	000070
SUPPORT SERVICES	000050
OPERATIONS	000090
SOFTWARE SUPPORT	000100
BRANCH OFFICE F2	No Manager
...	...

Removing NULLs

The **COALESCE** function returns the first argument that is *not* null.

The arguments are evaluated in the order in which they are specified, and the result of the function is the first argument that is not null. The result can be null only if all arguments can be null. The result is null only if all arguments are null. All input arguments must be of compatible data types.

There may be more than 2 arguments, however, the function is most commonly used to supply a literal for a null value in a column as shown in this example.

All three platforms support COALESCE. Db2 also supports the equivalent functions **VALUE** and **IFNULL** that are widely used on that platform.

Oracle implements functions **NVL** and **NVL2** with similar functionality. **NVL** works like **COALESCE**, but only accepts 2 arguments. **NVL2** accepts 3 arguments and can replace the first argument with the second (if the first is not null) or the third (if the first is null).

SQL Server also has a function called **ISNULL** that works like Oracle's **NVL** function.

The **CASE** statement may be used to duplicate the functionality offered by **COALESCE**:

```
CASE  
  WHEN e1 IS NOT NULL  
    THEN e1  
  ELSE e2  
END
```

Rank Function

```
SELECT LASTNAME, SALARY,
       RANK() OVER (ORDER BY SALARY DESC) AS R
FROM EMP
```



LASTNAME	SALARY	R
HAAS	52750.00	1
HEMMINGER	46500.00	2
LUCCHESI	46500.00	2
THOMPSON	41250.00	4

Ranking and Numbering Rows

Rank Function

The RANK function orders and ranks a result. In this example, the result is being ranked by SALARY in descending sequence, so the highest salary has a rank of 1. When there is a tie, all rows with the value receive the same rank and an appropriate number of ranks are “skipped”. In this example, since there were 2 salary values in second place, the value of 3 is skipped.

The result set may be ordered using an ORDER BY for the entire SELECT, and this order need not be the same as the column being ranked. When this is done, 2 sorts may need to be performed to achieve the desired result.

The ORDER BY clause associated with the RANK does not guarantee the order of the final result. An ORDER BY should be placed at the end of the statement if final order is important.

Dense Rank Function

```
SELECT LASTNAME, SALARY,  
       DENSE_RANK() OVER (ORDER BY SALARY DESC) AS R  
FROM EMP
```



LASTNAME	SALARY	R
HAAS	52750.00	1
HEMMINGER	46500.00	2
LUCCHESI	46500.00	2
THOMPSON	41250.00	3

Dense Rank Function

DENSE_RANK works like RANK, except no numbers are “skipped” when there is a tie. In this example, THOMPSON receives a rank of 3 even though she is the 4th highest salary.

Row Numbering

```
SELECT    LASTNAME, SALARY,  
          ROW_NUMBER() OVER (ORDER BY SALARY DESC) AS R  
FROM EMP
```

LASTNAME	SALARY	R
HAAS	52750.00	1
HEMMINGER	46500.00	2
LUCCHESI	46500.00	3
THOMPSON	41250.00	4

Row Numbering

ROW_NUMBER may be used to simply assign a unique number to each row of a result keeping the requested order. Since the rows are not ranked equivalent values are given subsequent numbers.

Db2 and PostgreSQL Only:

On these two databases, if you don't care what order the numbers are assigned, you can leave the order blank.

```
SELECT LASTNAME, SALARY,  
       ROW_NUMBER() OVER () AS R  
FROM EMP
```

When this is done, the order will depend on the specific access path Db2 chooses. If you place an ORDER BY over the entire result, the row numbers may be scrambled since they are assigned *before* the final ORDER BY is accomplished.

This page intentionally left blank

Chapter 5

Aggregation

Table of Contents

CHAPTER 5 AGGREGATION5-1

Overview.....5-3

 Objectives 5-3

Scalar Functions.....5-4

Column Functions.....5-5

 Common Column Functions 5-6

 How Column Functions Work..... 5-7

 Example 5-8

 COUNT Function 5-9

 Column Functions with Columns5-10

The GROUP BY Clause 5-11

Restricting Groups with HAVING 5-15

GROUPing by Multiple Columns 5-16

Overview

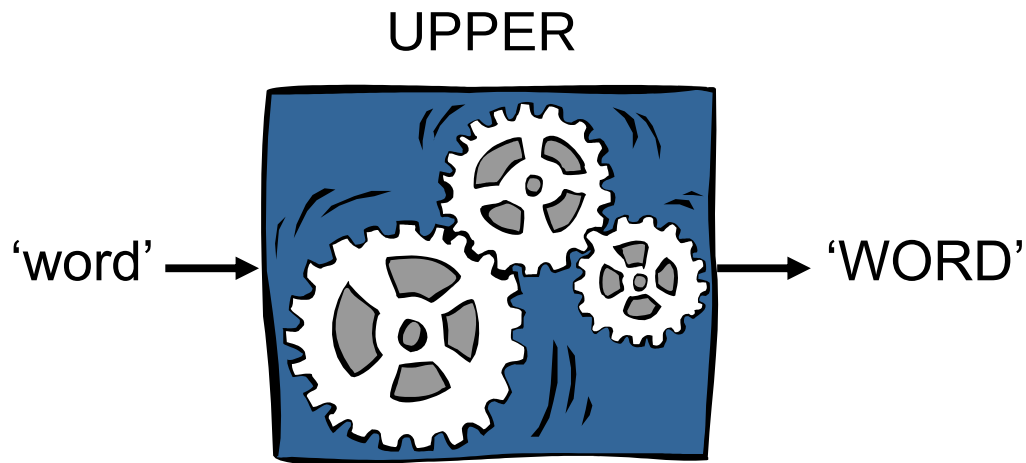
This chapter introduces the second type of built in functions that are used for summarization of groups of data. These functions are called *column functions*. Column functions and the concepts of grouping and summarizing results conclude the topics.

Objectives

By the end of this chapter, you should be able to:

- ◆ Summarize data with column functions.
- ◆ Aggregate data with the GROUP BY clause.
- ◆ Filter grouped data with the HAVING clause.

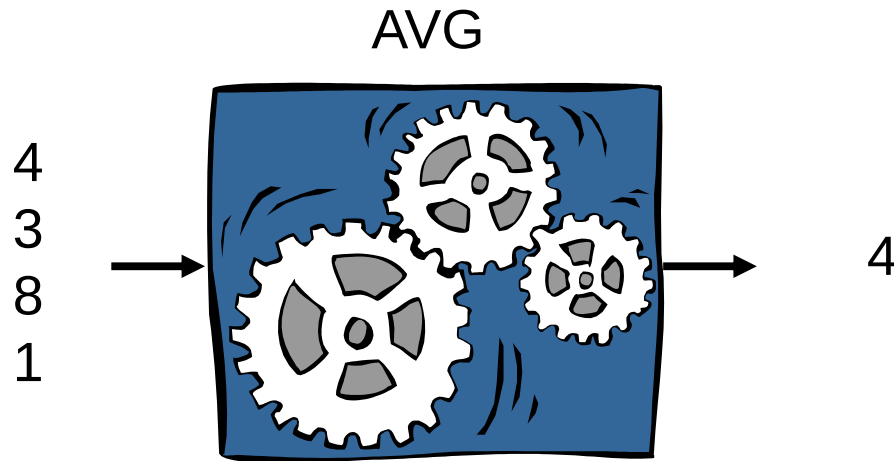
Scalar Functions



Scalar Functions

The built-in functions described in the previous chapter are referred to as *scalar* functions. They take in a finite list of arguments (often a single value) and produce a scalar (single value) result.

Column Functions



Column Functions

Column functions apply to groups of values. Unless otherwise specified (see the next topic on GROUP BY) a single group is defined by the WHERE clause. The result of applying a column function to a group of values is a scalar value representing the characteristics of the group. Column functions are also called *aggregate* functions.

Column Functions

Most commonly used Column Functions:

- AVG(column) - the average value of the column
- SUM(column) - the total value of the column
- MIN(column) - the minimum value of the column
- MAX(column) - the minimum value of the column
- COUNT(*) - the number of rows in the result
- COUNT(column) - the number of non null entries for the column
- COUNT(DISTINCT column)
 - the number of unique values for a column



Common Column Functions

Each function has keywords **ALL** or **DISTINCT**. **ALL** is the default and specifies all values in the group are to be used as an argument to the function. **DISTINCT** invokes a sort to remove duplicates resulting in a unique set of values used as an argument to the function.

ALL group functions (except COUNT(*)) automatically eliminate nulls from the argument set.

Thus:

```
SELECT COUNT(ALL SALARY)
```

and

```
SELECT COUNT(SALARY)
```

are the same request for *all the values*, while

```
SELECT COUNT(DISTINCT SALARY)
```

requests *only unique values* be counted.

How Column Functions Work

```
SELECT SUM(SALARY)
FROM EMP
WHERE DEPTNO = 'A00'
```

WHERE clause
determines which
rows are SUMmed

DEPTNO	SALARY
A00	46500.00
A00	29250.00
A00	52750.00
A00	52750.00
B01	41250.00
C01	38250.00
C01	23800.00
C01	28420.00
...	...

181250.00

Result is one row
summary

How Column Functions Work

Column functions apply to *groups of values*. The groups are defined by the WHERE clause and the GROUP BY clause. (GROUP BY is covered in the next topic). ***If no GROUP BY clause is specified, the single group is defined entirely by the WHERE clause.***

The qualifying row values are the argument set for the various column functions as shown in the diagram on the facing page. ***The count of an empty table is zero, all other column functions return unknown for empty argument sets.*** The result of applying the column function to this single group is a scalar value.

Example

```
SELECT    AVG(BONUS) AS AVG,  
          MIN(BONUS) AS MIN,  
          MAX(BONUS) AS MAX  
FROM      EMP  
WHERE     DEPTNO = 'A00'
```

AVG	MIN	MAX
875.00	600.00	1000.00

Example

The example on the facing page shows the application of column functions in various combinations. Since there is a WHERE clause, the summarized values represent information about the qualified rows. If no WHERE clause were present, the summarized values would represent a summary of the entire table.

Notice the result set is only one row. Absent a GROUP BY clause, whenever a column function is present in a SELECT, the result will ALWAYS be one row.

COUNT Function

```
SELECT    COUNT(*) AS NUM_ROWS,  
          COUNT(MAJPROJ) AS NON_NULL_MAJPROJ,  
          COUNT(DISTINCT MAJPROJ) AS UNIQ_MAJPROJ  
FROM      PROJ
```

NUM_ROWS	NON_NULL_MAJPROJ	UNIQ_MAJPROJ
20	15	7

COUNT Function

The COUNT function has three commonly used applications. COUNT(*) will count all the rows that qualify for the WHERE clause. COUNT(column) will be used in conjunction with a column and will count the number of rows that have a non-null value for the column. COUNT DISTINCT must also be used in conjunction with a particular column and will return a count of the number of unique values for that column (null is not counted as a distinct value).

Column Functions With Columns

Since multiple DEPTNOs qualify, which should be displayed in the 1 row summary?

```
SELECT  DEPTNO, SUM(SALARY) AS PAYROLL  
FROM    EMP  
WHERE   DEPTNO LIKE 'D%'
```

Error!

Column Functions with Columns

This example returns an error. Since the SUM function was used in the SELECT statement, the database intends to return a one row summary result. Because there could be many different values for DEPTNO on the different rows that contribute to the summary, the database does not know which value for DEPTNO to return in the one row summary.

The message received will vary by platform. It will indicate some form of “inconsistent grouping”.

The GROUP BY Clause

```
SELECT  DEPTNO, SUM(SALARY) AS PAYROLL  
FROM    EMP  
WHERE   DEPTNO LIKE 'D%'  
GROUP BY DEPTNO
```

One row for every unique
value for DEPTNO

The GROUP BY Clause

In the previous examples the column functions returned a single scalar result. The group defined by a WHERE clause was the argument to the built-in function producing a scalar result. It is often necessary to define multiple groups and have the column function apply to each separate group returning a set of groups, by department for instance.

As shown in the example on the facing page, GROUP BY is required to inform the database that the intermediate result consisting of all departments and individual salaries is to be divided into groups based on the value of DEPTNO. The column function SUM(SALARY) is then applied to each group resulting in one row per group representing characteristics of the group.

The GROUP BY Clause

```
SELECT  DEPTNO, SUM(SALARY) AS PAYROLL
FROM    EMP
WHERE   DEPTNO LIKE 'D%'
GROUP BY DEPTNO
```

DEPTNO	SALARY
D11	32250.00
D11	25280.00
D11	22250.00
D11	24680.00
D11	21340.00
D11	20450.00
D11	27740.00
D11	18270.00
D11	29840.00
D21	22180.00
D21	28760.00
D21	19180.00
D21	17250.00
D21	27380.00
D21	36170.00

222100.00

150920.00

DEPTNO	PAYROLL
D11	222100.00
D21	150920.00

The GROUP BY Clause

The basic framework for a SELECT statement is thus extended accordingly:

```
SELECT  
FROM  
WHERE  
GROUP BY  
HAVING  
ORDER BY
```

The database will likely invoke a sort to group the rows according to the column (or columns) in the GROUP BY.

Performance Tip. When dealing with expressions and column functions consider the following:

AVG(SALARY) * 1.1

is superior to

AVG(SALARY * 1.1)

The first form can use an available index to calculate the average while the second form cannot due to the presence of the expression.

The GROUP BY clause may contain multiple columns. The result will always be one row for every unique combination of grouped values. For example:

GROUP BY DEPTNO, EDLEVEL

will provide a result with a row for every combination of DEPTNO AND EDLEVEL. Any selected column functions would be applied to each group.

Restricting Groups with HAVING

```
SELECT  DEPTNO, AVG(SALARY) AS AVG
FROM    EMP
GROUP BY DEPTNO
HAVING  AVG(SALARY) > 40000
```

DEPTNO	AVG
A00	45312.50
B01	41250.00
C01	30156.67
D11	24677.78
D21	25153.33
E01	40175.00
E11	20998.00
E21	23827.50

DEPTNO	AVG
A00	45312.50
B01	41250.00
E01	40175.00

Result *without*
HAVING

Restricting Groups with HAVING

The HAVING clause is provided to apply predicates to the summarized values and eliminate any row that does not evaluate to true. *The HAVING clause operates on the summarized values the same way that the WHERE clause operates on the individual column values.* HAVING must be preceded by GROUP BY and may contain only the columns mentioned in the GROUP BY clause (although not necessarily mentioned in the SELECT).

Performance Tip. Do not specify WHERE clause predicates in the HAVING clause. Although legal, the performance implications are costly due to a much larger sort set. Consider the following which attempts to produce average salary by department for female employees:

```
SELECT DEPTNO, GENDER, AVG(SALARY)
FROM   EMP
GROUP BY DEPTNO, GENDER
HAVING GENDER = 'F'
```

Although the result is correct, the sort for GROUP BY included the entire company, male and female. The following is the more efficient form:

```
SELECT DEPTNO, GENDER, AVG(SALARY)
FROM   EMP
WHERE  GENDER = 'F'
GROUP BY DEPTNO, GENDER
```

This form eliminates the unwanted rows prior to the sort.

GROUPing by Multiple Columns

```
SELECT  DEPTNO, JOB, AVG(SALARY) AS AVG  
FROM    EMP  
WHERE   DEPTNO < 'B99'  
GROUP BY DEPTNO, JOB
```

DEPTNO	JOB	AVG
A00	CLERK	29250.00
A00	PRES	52750.00
A00	SALESREP	46500.00
B01	MANAGER	41250.00

GROUPing by Multiple Columns

When multiple columns are specified in the GROUP BY clause the result will be one row for every unique combination of values in the columns specified.

Chapter 6

Joins

Table of Contents

CHAPTER 6 JOINS.....6-1

Overview.....6-3

 Objectives 6-3

Inner Join.....6-4

 Inner Joins -DEPT Table 6-5

 Inner Joins -EMP Table 6-6

 Inner Joins – Example 1 6-7

 Inner Joins – SQL86 6-8

 Inner Joins – SQL92 6-9

 Inner Joins – Example 26-10

 Cross Joins6-11

 Outer Joins.....6-12

 Left Outer Join Example6-13

 Left Outer – Local Predicate6-15

 Right Outer Join Example6-17

 Full Outer Join.....6-19

 Full Outer Join Example.....6-20

 Left Outer Join – On Clause Extensions6-23

Self Joins / Other Joins6-25

 Self Referencing Foreign Keys6-26

 Comparing Rows to Other Rows6-28

 Joining to a Table Multiple Times6-30

Overview

Objectives

By the end of this chapter, you should:

- ◆ Be able to compare and contrast inner joins with the different types of outer join
- ◆ Understand the ON clause extensions
- ◆ Be familiar with self-joins

Inner Joins

- Must contain Join Predicates
- Good coding requires Correlation Id's
- Order of tables does not matter
- Order of predicates does not matter
- Result Set contains only those rows that have the same values in each table

Inner Join

A **join** is used to combine rows and data from multiple tables. A join is performed whenever two or more tables is listed in the FROM clause of an SQL statement.

A join is nothing more than writing a query that takes a set of rows from one or more tables and combines them together. The tables that will participate in the join are listed in the FROM clause of the SQL query and then, depending on the syntax we choose, rows from each table are linked by either the WHERE clause or the JOIN clause. Of course, any columns from the tables involved in the join may be included in the select list for display.

There are different kinds of joins (Inner, Outer, Self).

Inner joins are the most common type of join. Inner joins return all rows from multiple tables where the join condition is met, with the join condition being where a column(s) in one table is equal to a column in another table.

This screen lists some of the specific points related to inner joins. The biggest point here is that the results sets ***only contain*** those rows whose values between the tables are met. This will become clearer in the examples to follow.

Inner Joins - DEPT Table

DEPTNO	DEPTNAME	MGRNO
A00	SPIFFY COMPUTER SERVICE DIV.	000010
B01	PLANNING	000020
C01	INFORMATION CENTER	000030
D01	DEVELOPMENT CENTER	<null>
D11	MANUFACTURING SYSTEMS	000060
D21	ADMINISTRATION SYSTEMS	000070
E01	SUPPORT SERVICES	000050
E11	OPERATIONS	000090
E21	SOFTWARE SUPPORT	000100
F22	BRANCH OFFICE F2	<null>
G22	BRANCH OFFICE G2	<null>
H22	BRANCH OFFICE H2	<null>
I22	BRANCH OFFICE I2	<null>
J22	BRANCH OFFICE J2	<null>

Inner Joins -DEPT Table

This screen shows the Department table that is used in many of the join examples in this chapter. Notice that some departments have been assigned a manager and some departments have not. To get the name for a manager of a department, a join to the Employee table must be coded.

Inner Joins - EMP Table

EMPNO	DEPTNO	LASTNAME
000010	A00	HAAS
000020	B01	THOMPSON
000030	C01	KWAN
000050	E01	GEYER
000060	D11	STERN
000070	D21	PULASKI
000090	E11	HENDERSON
000100	E21	SPENSER
000110	A00	LUCCHESI
000120	A00	O'CONNELL
000130	C01	QUINTANA
000140	C01	NICHOLLS
000150	D11	ADAMSON
000160	D11	PIANKA
000170	D11	YOSHIMURA
.....		

Inner Joins -EMP Table

This screen shows the Employee table that is used in many of the join examples in this chapter. Notice that each employee has been assigned a department number. To get the name for the department, a join to the Department table must be coded.

Inner Joins - Example 1

```
SELECT  D.DEPTNO, D.DEPTNAME,  
        D.MGRNO, E.LASTNAME  
FROM    DEPT D, EMP E  
WHERE   D.MGRNO = E.EMPNO
```

OR

```
SELECT  D.DEPTNO, D.DEPTNAME,  
        D.MGRNO, E.LASTNAME  
FROM    DEPT D JOIN EMP E  
ON      D.MGRNO = E.EMPNO
```

Either query gets those departments where the value in its MGRNO column from the DEPT table equals a value in the EMPNO column from the EMP table (the join columns).

Inner Joins – Example 1

This sample SQL join query needs to get data from both the DEPT and EMP tables, so an SQL statement is written that contains both tables.

Note the following:

1. **Correlation Ids:** Notice the 2 correlation IDs of 'E' and 'D'. These are used to help differentiate from which tables columns are from.
2. **From Clause:** contains all tables needed
3. **Join Predicate:** Contains the join syntax for the columns to be matched between the 2 tables.

Also notice the 2 different SQL syntaxes that can be coded, both ANSI ISO standard.

This sample SQL join query is an example of an inner join. This query will only return pairs of rows where MGRNO = EMPNO.x`

Inner Joins - SQL86

```
SELECT    D.DEPTNO, D.DEPTNAME,  
          D.MGRNO, E.LASTNAME  
FROM      DEPT D, EMP E  
WHERE     D.MGRNO = E.EMPNO
```

DEPTNO	DEPTNAME	MGRNO	LASTNAME
A00	SPIFFY COMPUTER SERVICE DIV.	000010	HAAS
B01	PLANNING	000020	THOMPSON
C01	INFORMATION CENTER	000030	KWAN
E01	SUPPORT SERVICES	000050	GEYER
D11	MANUFACTURING SYSTEMS	000060	STERN
D21	ADMINISTRATION SYSTEMS	000070	PULASKI
E11	OPERATIONS	000090	HENDERSON
E21	SOFTWARE SUPPORT	000100	SPENSER

Inner Joins – SQL86

This screen shows how the join is coded based on the SQL 86 syntax, and also shows the results. As can be seen, only 8 departments become part of the result set because the other departments do not have a manager assigned.

Inner Joins - SQL92

```
SELECT  D.DEPTNO, D.DEPTNAME,
        D.MGRNO, E.LASTNAME
FROM    DEPT D INNER JOIN EMP E
ON      D.MGRNO = E.EMPNO
```

DEPTNO	DEPTNAME	MGRNO	LASTNAME
A00	SPIFFY COMPUTER SERVICE DIV.	000010	HAAS
B01	PLANNING	000020	THOMPSON
C01	INFORMATION CENTER	000030	KWAN
E01	SUPPORT SERVICES	000050	GEYER
D11	MANUFACTURING SYSTEMS	000060	STERN
D21	ADMINISTRATION SYSTEMS	000070	PULASKI
E11	OPERATIONS	000090	HENDERSON
E21	SOFTWARE SUPPORT	000100	SPENSER

Inner Joins – SQL92

This screen shows how the join is coded based on the SQL-92 syntax, and also shows the results. As can be seen, only 8 departments become part of the result set because the other departments do not have a manager assigned.

Also notice that the join predicate is now part of the ON clause, and not the WHERE clause.

The keywords “INNER JOIN” may be replaced with “JOIN” as “INNER” is the default. This is a common practice.

Inner Joins Example 2

```
SELECT  D.DEPTNO, D.DEPTNAME,
        D.MGRNO, E.LASTNAME
FROM    DEPT D, EMP E
WHERE   D.MGRNO = E.EMPNO
        AND D.DEPTNO LIKE 'D%'
```

OR

```
SELECT  D.DEPTNO, D.DEPTNAME,
        D.MGRNO, E.LASTNAME
FROM    DEPT D INNER JOIN EMP E
        ON D.MGRNO = E.EMPNO
        WHERE D.DEPTNO LIKE 'D%'
```

DEPTNO	DEPTNAME	MGRNO	LASTNAME
D11	MANUFACTURING SYSTEMS	000060	STERN
D21	ADMINISTRATION SYSTEMS	000070	PULASKI

Inner Joins – Example 2

This screen shows how to code an Inner join both ways when both filtering predicates join predicates are needed. Notice in the SQL 92 syntax, the join predicate must be part of the ON clause.

The results are the same no matter which way the SQL is coded. The style of join is also not significant for performance. It is only a matter of preference which SQL syntax is chosen to code.

Cross Joins

- Uses the syntax CROSS JOIN with no join predicate
- Also called a Cartesian join
- A join without join predicate outputs a Cartesian product. Cartesian product result-set contains the number of rows in the first table, multiplied by the number of rows in second table.
- Same as having an SQL 86 two table join with no join predicate

SQL Server

Oracle

PostgreSQL



Cross Joins

```
SELECT E.EMPNO, E.DEPTNO, D.DEPTNAME
FROM EMP E, DEPT D
AND
SELECT E.EMPNO, E.DEPTNO, D.DEPTNAME
FROM EMP E CROSS JOIN DEPT D
```

Produce the same cartesian output (number of rows in emp table x number of rows in dept table)

If the following is coded, it defaults to an inner join based on the join predicate.

```
SELECT E.EMPNO, E.DEPTNO, D.DEPTNAME
FROM EMP E CROSS JOIN DEPT D
WHERE E.DEPTNO = D.DEPTNO
;
```

```
Db2: SELECT E.EMPNO, E.DEPTNO, D.DEPTNAME
FROM EMP E,
DEPT
```

Outer Joins

- Can be Left, Right, or Full Outer Joins
- Must contain 'On Clause' Join Predicates
- Good coding requires Correlation Id's
- Order of tables is important
- Filtering predicates part of 'Where' clause
- Result Set contains those rows that have a value in the other table, along with those rows that do not have a value in the other table (the exceptions)



Outer Joins

Result sets from outer joins ***contain*** both the rows whose values between the tables are met, and the rows where the join condition is not met.

With outer joins, unlike inner joins, it is very important which table is coded first. The table coded first will be the table where exceptions come from if no join is met between the tables.

Examples follow for the 3 different outer joins (Left, Right, Full).

Left Outer Join Example

```
SELECT  D.DEPTNO, D.DEPTNAME,
        D.MGRNO, E.LASTNAME
FROM    DEPT D LEFT OUTER JOIN EMP E
ON      D.MGRNO = E.EMPNO
```

DEPTNO	DEPTNAME	MGRNO	LASTNAME
A00	SPIFFY COMPUTER SERVICE DIV.	000010	HAAS
B01	PLANNING	000020	THOMPSON
C01	INFORMATION CENTER	000030	KWAN
E01	SUPPORT SERVICES	000050	GEYER
D01	DEVELOPMENT CENTER	<null>	<null>
D11	MANUFACTURING SYSTEMS	000060	STERN
D21	ADMINISTRATION SYSTEMS	000070	PULASKI
E11	OPERATIONS	000090	HENDERSON
E21	SOFTWARE SUPPORT	000100	SPENSER
F22	BRANCH OFFICE F2	<null>	<null>
G22	BRANCH OFFICE G2	<null>	<null>
H22	BRANCH OFFICE H2	<null>	<null>
I22	BRANCH OFFICE I2	<null>	<null>
J22	BRANCH OFFICE J2	<null>	<null>

Left Outer Join Example

This screen shows the Left outer join syntax. Remember that it is important to list the tables in the correct order.

The clause **LEFT OUTER JOIN** includes rows from the table that is specified before **LEFT OUTER JOIN** that have no matching values in the table that is specified after **LEFT OUTER JOIN**.

The table to the left of the words Left Outer Join in the driver table. This is the table where exceptions will come from no matter if the join condition is met on the other table or not.

In this example, the DEPT table is the driver table. The query will retrieve all departments since there is no filtering on that table. If the join is met then the EMP information will also be displayed. When no match is made to the joined Employee table, any columns specific to that table will return as null. In this example that would be LASTNAME.

The keyword "OUTER" is optional here. A "LEFT JOIN" is always an "OUTER" join. The syntax "LEFT JOIN" is synonymous with "LEFT OUTER JOIN".

Left Outer Join – Local Predicate

```
SELECT  D.DEPTNO, D.DEPTNAME,  
        D.MGRNO, E.LASTNAME  
FROM    DEPT D LEFT OUTER JOIN EMP E  
ON      D.MGRNO = E.EMPNO  
WHERE  D.DEPTNO LIKE 'D%'
```

DEPTNO	DEPTNAME	MGRNO	LASTNAME
D01	DEVELOPMENT CENTER	<null>	<null>
D11	MANUFACTURING SYSTEMS	000060	STERN
D21	ADMINISTRATION SYSTEMS	000070	PULASKI

Left Outer – Local Predicate

This screen shows another example of a Left outer join, but this one containing other predicates besides the Join Predicate. Any other predicates specific to the filtering of data is part of the Where clause.

By adding 'WHERE D.DEPTNO LIKE 'D%', this is stating that all departments that begin with 'D' will be returned, whether the joined condition to the employee table is met or not.

Right Outer Join Example

```

SELECT  D.DEPTNO, D.DEPTNAME,
        D.MGRNO, E.LASTNAME
FROM    EMP E RIGHT OUTER JOIN DEPT D
ON      D.MGRNO = E.EMPNO

```

DEPTNO	DEPTNAME	MGRNO	LASTNAME
A00	SPIFFY COMPUTER SERVICE DIV.	000010	HAAS
B01	PLANNING	000020	THOMPSON
C01	INFORMATION CENTER	000030	KWAN
E01	SUPPORT SERVICES	000050	GEYER
D01	DEVELOPMENT CENTER	<null>	<null>
D11	MANUFACTURING SYSTEMS	000060	STERN
D21	ADMINISTRATION SYSTEMS	000070	PULASKI
E11	OPERATIONS	000090	HENDERSON
E21	SOFTWARE SUPPORT	000100	SPENSER
F22	BRANCH OFFICE F2	<null>	<null>
G22	BRANCH OFFICE G2	<null>	<null>
H22	BRANCH OFFICE H2	<null>	<null>
I22	BRANCH OFFICE I2	<null>	<null>
J22	BRANCH OFFICE J2	<null>	<null>

Right Outer Join Example

This screen shows the Right outer join syntax. Remember that it is important to list the tables in the correct order.

The clause **RIGHT OUTER JOIN** includes rows from the table that is specified after **RIGHT OUTER JOIN** that have no matching values in the table that is specified before **RIGHT OUTER JOIN**.

The table to the right of the words Right Outer Join is the driver table. This is the table where exceptions will come from no matter if the join condition is met on the other table or not.

This is just the opposite of the Left Outer Join syntax, it just depends on the order of the tables in the SQL and whether the words Left or Right are coded.

As can be seen from the SQL, the DEPT table is the driver table. So the results will contain the departments that have a manager match on the Employee table, and also contain the departments that do not have a manager match on the Employee table.

When no match is made to the joined Employee table, any columns specific to that table will return as null. In this example that would be LASTNAME.

As shown, the results are the same as the previous Left Outer Join example, but coded as a Right Outer Join. Both perform the same.

The keyword "OUTER" is optional here. A "RIGHT JOIN" is always an "OUTER" join. The syntax "RIGHT JOIN" is synonymous with "RIGHT OUTER JOIN".

Full Outer Join

```
SELECT D.DEPTNO, D.DEPTNAME,  
       D.MGRNO, E.LASTNAME  
FROM   EMP E FULL OUTER JOIN DEPT D  
ON     D.MGRNO = E.EMPNO
```

In this example:

- The RDMS first gets the departments where D.MGRNO = E.EMPNO
- The RDMS then gets the departments where D.MGRNO <> E.EMPNO
- The RDMS then gets the employees where the empno not = mgrno

- There are 14 rows in the DEPT table
- There are 33 rows in the EMP table
- There are 8 rows where D.MGRNO = E.EMPNO
- That leaves 6 other departments where D.MGRNO <> E.EMPNO
- That leaves 25 other employees where D.MGRNO <> E.EMPNO

Full Outer Join

This screen shows via a picture of exactly what goes on in getting the 48 rows returned due to the Full Outer Join.

- ◆ From executing an Inner Join there were 8 rows
- ◆ From the Employee table that leaves $33 - 8 = 25$ rows
- ◆ From the Department table that leave $14 - 8 = 6$ rows

For a total of 39 rows.

The keyword "OUTER" is optional here. A "FULL JOIN" is always an "OUTER" join. The syntax "FULL JOIN" is synonymous with "FULL OUTER JOIN".

Full Outer Join

DEPTNO	DEPTNAME	MGRNO	LASTNAME
A00	SPIFFY COMPUTER SERVICE DIV.	000010	HAAS
B01	PLANNING	000020	THOMPSON
C01	INFORMATION CENTER	000030	KWAN
E01	SUPPORT SERVICES	000050	GEYER
D11	MANUFACTURING SYSTEMS	000060	STERN
D21	ADMINISTRATION SYSTEMS	000070	PULASKI
E11	OPERATIONS	000090	HENDERSON
E21	SOFTWARE SUPPORT	000100	SPENSER
D01	DEVELOPMENT SUPPORT	<null>	<null>
F22	BRANCH OFFICE F2	<null>	<null>
G22	BRANCH OFFICE G2	<null>	<null>
H22	BRANCH OFFICE H2	<null>	<null>
I22	BRANCH OFFICE I2	<null>	<null>
J22	BRANCH OFFICE J2	<null>	<null>
<null>	<null>	<null>	GOUNOT
<null>	<null>	<null>	LEE
... 23 more employees			

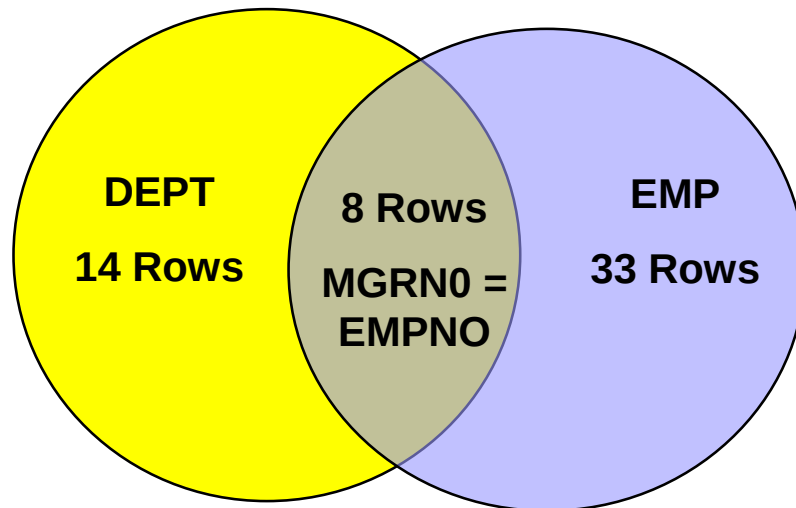
Full Outer Join Example

As can be seen from this screen, it does not matter the order of the tables in SQL. The important thing to recognize here is that exceptions (nulls) can be returned from both tables.

- ◆ For LASTNAME when the Mgrno is not found on the Employee table
- ◆ For DEPTNO, DEPTNAME, MGRNO when Empno is not found as a Mgrno on the Department table.

The clause ***FULL OUTER JOIN*** includes all inner join rows PLUS exceptions from the first table AND exceptions from the other table.

Full Outer Join



In this example, the SQL will return

- ◆ All rows where MGRNO=EMPNO (Inner Join)
- ◆ All departments whose MGRNO is not found on the EMP table
- ◆ All employees whose EMPNO is not found on the DEPT table

Left Join – ON Clause Extensions

```

SELECT      D.DEPTNO, D.DEPTNAME,
            D.MGRNO, E.LASTNAME
FROM DEPT D LEFT OUTER JOIN EMP E
ON        D.MGRNO = E.EMPNO
AND      D.DEPTNO LIKE 'D%'
  
```

DEPTNO	DEPTNAME	MGRNO	LASTNAME
A00	SPIFFY COMPUTER SERVICE DIV.	000010	<null>
B01	PLANNING	000020	<null>
C01	INFORMATION CENTER	000030	<null>
E01	SUPPORT SERVICES	000050	<null>
D01	DEVELOPMENT CENTER	<null>	<null>
D11	MANUFACTURING SYSTEMS	000060	STERN
D21	ADMINISTRATION SYSTEMS	000070	PULASKI
E11	OPERATIONS	000090	<null>
E21	SOFTWARE SUPPORT	000100	<null>
F22	BRANCH OFFICE F2	<null>	<null>
G22	BRANCH OFFICE G2	<null>	<null>
H22	BRANCH OFFICE H2	<null>	<null>
I22	BRANCH OFFICE I2	<null>	<null>
J22	BRANCH OFFICE J2	<null>	<null>

Left Outer Join – On Clause Extensions

As can be seen from this screen, the results are different when filtering predicates are part of an ON clause versus a WHERE clause.

When predicates are part of a WHERE clause, only the rows that meet that criteria will be returned to the result set. For example:

WHERE D.DEPTNO LIKE 'D%' will only return those rows that have a DEPTNO column beginning with the letter 'D'.

But:

ON D.MGRNO = E.EMPNO

AND D.DEPTNO LIKE 'D%' will return all departments, but only those rows that have a DEPTNO column beginning with 'D', will a join take place to the EMP table in order to get the manager's name.

By adding the predicate D.DEPTNO like 'D%' to the 'on clause' only filters what is to be joined. It does not filter data from the final result set.

Note: Only predicates in a 'where clause' will filter rows from a result set.

Predicates in an 'on clause' determines which rows the RDMS should send to the other table to be joined.

Some 'On' clause rules:

If coding Full Outer Joins, any predicates added to the 'On' clause must be equal predicates only. No range predicates, or predicates, between predicates, etc...

For Left or Right Outer Joins, any 'On' clause predicates are OK, except subquery predicates.

Self Joins / Other Joins

- When a query has the same table in it more than once
- Self Referencing Foreign Keys Joins
- Comparing rows in a table to other rows in the same table
- When a table gets joined to multiple times
- Natural joins
- Examples to follow

Self Joins / Other Joins

This screen lists some of the specific points related to self joins. The obvious part of self joins is that there exists a table listed in the From clause more than once. This screen states some of the most common reason for having to code self joins.

Self Referencing Foreign Keys



DEPTNO	DEPTNAME	MGRNO	ADMRDEPT
A00	SPIFFY COMPUTER SERVICE DIV.	000010	A00
B01	PLANNING	000020	A00
C01	INFORMATION CENTER	000030	A00
D01	DEVELOPMENT CENTER	<null>	A00
D11	MANUFACTURING SYSTEMS	000060	D01
D21	ADMINISTRATION SYSTEMS	000070	D01
E01	SUPPORT SERVICES	000050	B01
E11	OPERATIONS	000090	E01
E21	SOFTWARE SUPPORT	000100	E01
F22	BRANCH OFFICE F2	<null>	E01
G22	BRANCH OFFICE G2	<null>	E01
H22	BRANCH OFFICE H2	<null>	E01
I22	BRANCH OFFICE I2	<null>	E01
J22	BRANCH OFFICE J2	<null>	E01

Self Referencing Foreign Keys

Sometimes in designing tables, there will be a column that actually points back to the primary key of the same table. Bill of Material tables are a perfect example. The DEPT table is designed this way. Each department actually rolls into an administrative department. So for each administrative department, there must already exist a department with the same value.

How do we get the name for the administrative department value in the table?

Self Referencing Foreign Keys

```
SELECT    D1.DEPTNAME,
          D2.DEPTNAME AS ADMR_NAME
FROM DEPT D1, DEPT D2
WHERE D1.ADMRDEPT = D2.DEPTNO
```

DEPTNAME	ADMR_NAME
SPIFFY COMPUTER SERVICE DIV.	SPIFFY COMPUTER SERVICE DIV.
PLANNING	SPIFFY COMPUTER SERVICE DIV.
INFORMATION CENTER	SPIFFY COMPUTER SERVICE DIV.
SUPPORT SERVICES	SPIFFY COMPUTER SERVICE DIV.
DEVELOPMENT CENTER	SPIFFY COMPUTER SERVICE DIV.
MANUFACTURING SYSTEMS	DEVELOPMENT CENTER
ADMINISTRATION SYSTEMS	DEVELOPMENT CENTER
OPERATIONS	DEVELOPMENT CENTER
SOFTWARE SUPPORT	DEVELOPMENT CENTER
BRANCH OFFICE F2	SUPPORT SERVICES
BRANCH OFFICE G2	SUPPORT SERVICES
BRANCH OFFICE H2	SUPPORT SERVICES
BRANCH OFFICE I2	SUPPORT SERVICES
BRANCH OFFICE J2	SUPPORT SERVICES

This screen actually shows the SQL that needs to be coded in order to get the expected results of DEPTNAME for DEPTNO column, and DEPTNAME for ADMRDEPT column.

As can be seen, in order to accomplish this, the DEPT table needs to be coded twice, but a different correlation Id on each. This is what differentiates one DEPT table from the other.

Comparing Rows to Other Rows

EMPNO	LASTNAME	EDLEVEL	SALARY
000010	HAAS	18	52750.00
000020	THOMPSON	18	41250.00
000030	KWAN	20	38250.00
000110	LUCCHESI	19	46500.00
000120	O'CONNELL	14	29250.00
000130	QUINTANA	16	23800.00
000140	NICHOLLS	18	28420.00
000190	WALKER	16	20450.00
000200	BROWN	17	27740.00

REQUEST:

Show employees who have a
higher EDLEVEL, but lower
SALARY than other employees..

Comparing Rows to Other Rows

Sometimes when comparing data in a table to other data in the same table, the table needs to be noted twice in the From clause. In this example, a report is requested to show those employees who have a higher education, yet lower salary than others in the same table.

How is this accomplished?

Comparing Rows to Other Rows

```
SELECT E1.EMPNO, E1.EDLEVEL, E1.SALARY,
       E2.EMPNO, E2.EDLEVEL, E2.SALARY
FROM EMP E1 JOIN EMP E2
  ON E1.EDLEVEL > E2.EDLEVEL
 AND E1.SALARY < E2.SALARY
```

EMPNO	EDLEVEL	SALARY	EMPNO	EDLEVEL	SALARY
000030	20	38250.00	000010	18	52750.00
000030	20	38250.00	000020	18	41250.00
000030	20	38250.00	000110	19	46500.00
000110	19	46500.00	000010	18	52750.00
000130	16	23800.00	000120	14	29250.00
000140	18	28420.00	000120	14	29250.00
000190	16	20450.00	000120	14	29250.00
000200	17	27740.00	000120	14	29250.00

This screen shows the actual SQL syntax to accomplish the request of employees being compared to other employees in the same table.

The key here is to SQL, it must look like 2 separate table, even though it is the same table. This is accomplished by using 2 different correlation Ids.

Joining to a Table Multiple Times

PROJ Table

PROJNO	PROJNAME	DEPTNO	EMPNO
AD3100	ADMIN SERVICES	D01	000010
AD3110	GENERAL AD SYSTEMS	D21	000070
AD3111	PAYROLL PROGRAMMING	D21	000230
AD3112	PERSONNEL PROGRAMMING	D21	000250
AD3113	ACCOUNT.PROGRAMMING	D21	000270
IF1000	QUERY SERVICES	C01	000030
IF2000	USER EDUCATION	C01	000030
MA2100	WELD LINE AUTOMATION	D01	000010
MA2110	W L PROGRAMMING	D11	000060
MA2111	W L PROGRAM DESIGN	D11	000220
MA2112	W L ROBOT DESIGN	D11	000150

Joining to a Table Multiple Times

Many times in SQL programming, a table needs to be joined to from multiple places for the same reason. Take a look at the PROJ table. It contains a column called EMPNO that is the employee responsible for the project (the project manager).

Where would we get the name for this person?

Joining to a Table Multiple Times

Report Request:

DEPTNO	MGRNO	LASTNAME	PROJNO	RESPEMP	RESPEMP NAME
--------	-------	----------	--------	---------	--------------

Create a report of departments with associated projects. Each department shows its manager, and manager's last name. Each project shows its manager (responsible employee) and project manager's last name.

Here is a request for a report where 2 joins are needed:

- ◆ One to join to the EMP table in order to get the name for the DEPT MGRNO.
- ◆ A second join to the EMP table to get the name for the PROJ EMPNO.

Joining to a Table Multiple Times

```
SELECT D.DEPTNO, D.MGRNO,  
       E1.LASTNAME, P.PROJNO,  
       P.EMPNO AS RESPEMP,  
       E2.LASTNAME AS RESPEMP_NAME  
FROM   DEPT D,  
       PROJ P,  
       EMP E1,  
       EMP E2  
WHERE  D.DEPTNO = P.DEPTNO  
AND    D.MGRNO = E1.EMPNO  
AND    P.EMPNO = E2.EMPNO
```

OR

```
SELECT D.DEPTNO, D.MGRNO,  
       E1.LASTNAME, P.PROJNO,  
       P.EMPNO AS RESPEMP,  
       E2.LASTNAME AS RESPEMP_NAME  
FROM   DEPT D JOIN PROJ P  
       ON   D.DEPTNO = P.DEPTNO  
JOIN   EMP E1  
       ON   D.MGRNO = E1.EMPNO  
JOIN   EMP E2  
       ON   P.EMPNO = E2.EMPNO
```

Joining to a Table Multiple Times

This screen shows the actual SQL syntax to accomplish the request of getting a name for the department manager, and getting a name for the project manager (responsible employee).

The key here is having the SQL list the EMP table twice, once to be joined to the DEPT table, and once to be joined to the PROJ table.

Chapter 7

Data Modification

Table of Contents

CHAPTER 7 DATA MODIFICATION7-1

Overview.....7-3

 Objectives 7-3

Data Modification Statements7-4

Insert.....7-5

 Insert with Multiple Values..... 7-6

 Db2 Multi-row Insert..... 7-7

UPDATE.....7-9

 Searched UPDATE..... 7-9

 UPDATE with SELECT7-10

Merge Statement.....7-11

 Merge with Subselect7-11

 MERGE with Values7-12

 Oracle.....7-13

 PostgreSQL MERGE Uses ON CONFLICT7-14

DELETE7-15

 SQL TRUNCATE Table7-16

 TRUNCATE Db2 LUW7-17

 TRUNCATE - Oracle7-19

 TRUNCATE – SQL Server7-20

 PostgreSQL TRUNCATE.....7-21

 Db2 SELECT FROM7-22

 SELECT FROM MERGE7-23

Unit of Work & Transaction Processing.....7-24

 Db2 Commit.....7-25

 Db2 Rollback7-27

 Oracle Transaction.....7-30

 PostgreSQL Transaction.....7-32

Constraints.....7-33

Overview

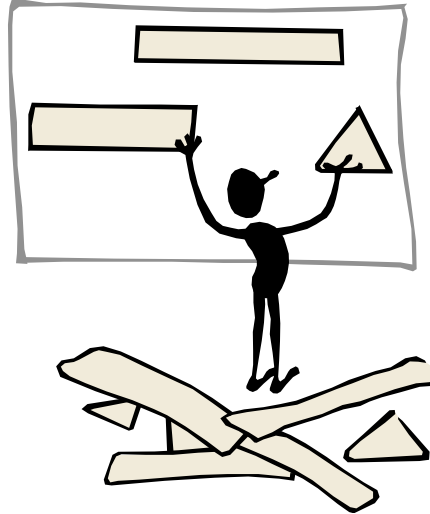
Objectives

After successful completion of this section the student should be able to:

- ◆ Understand the following data modification statements
 - INSERT
 - UPDATE
 - DELETE
 - MERGE
 - TRUNCATE
- ◆ Be able to organize these statements in logical units of work

Data Modification Statements

- INSERT
- UPDATE
- DELETE
- MERGE
- TRUNCATE



Data Modification Statements

There are 5 statements in SQL that support data modification on a table. Each will be discussed in some detail in this chapter.

Insert

```
INSERT INTO EMP  
      (EMPNO, FIRSTNAME, MIDINIT, LASTNAME)  
VALUES ('111111', 'JOHN', 'A', 'WAYNE')
```

Single Row

```
INSERT INTO EMP  
      (EMPNO, FIRSTNAME, MIDINIT, LASTNAME)  
SELECT  CE,CF,CM,CL  
FROM NEW_EMP  
WHERE JOB = 'ANALYST'
```

Copy Rows from
Another Table

Insert

The INSERT statement adds rows to a table. In the first example, a new row is being added to the EMP table. This statement specifies list of column names. The VALUES clause is followed by values for each of the specified column names. The columns which are not specified must be defined as nullable or with default value.

The second example uses a subquery with one or more result rows to be inserted to the target table of the INSERT.

Insert with Multiple Values

```
INSERT INTO EMP  
      (EMPNO, FIRSTNME, MIDINIT, LASTNAME)  
VALUES ('111111', 'JOHN', 'A', 'WAYNE'),  
      ('123456', 'JANE', 'F', 'SMITH')
```

Multiple Rows

Db2 *LUW Only!!*

SQL Server

PostgreSQL

Insert with Multiple Values

SQL Server, PostgreSQL, and Db2 on the Linux, Unix and Windows platforms allows for multiple sets of values to be inserted with a single insert statement. SQL Server only allows for up to 1000 rows per statement. Db2 does not have a documented maximum.

Db2 Multi-row Insert

```
INSERT INTO EMP  
  (EMPNO,FIRSTNME, MIDINIT, LASTNAME))  
  VALUES (:array1, :array2, :array3, :array4)  
FOR 5 ROWS  
NOT ATOMIC CONTINUE ON SQLEXCEPTION
```

Db2 *z/OS Only!!*

Multi-row Insert

Db2 Multi-row Insert

On Db2 for z/OS, multi-row insert is supported programmatically. This involves loading host variable arrays with multiple rows of data and executing a statement to insert them as shown here. Full coverage of this feature is beyond the scope of this course. Proper error handling is essential for this feature to be successfully implemented.

Searched UPDATE

UPDATE	EMP
SET	SALARY = SALARY * 1.1
WHERE	DEPTNO = 'D11'

All employees in D11 will be updated

UPDATE	EMP
SET	LASTNAME = 'JONES', FIRSTNAME = 'FRED'
WHERE	EMPNO = '000120'

Primary Key ensures only 1 row will be updated

UPDATE

Searched UPDATE

In the basic UPDATE, each column to be changed is identified in the SET clause. The qualifying rows, as always, are identified with predicates in the WHERE clause. The term *searched update* comes from the concept of the WHERE clause searching for the qualified rows to be updated. This is in contrast to the positioned UPDATE.

The UPDATE clause identifies a single table. The columns appearing in the SET clause can appear in any order but can appear only once. Using the keyword NULL, a column can be explicitly set to unknown. In the basic form, expression means: a constant, a column (from among the columns of the single table being updated), or an expression that produces a single value.

Based on the WHERE clause, the UPDATE statement may affect zero rows, 1 row, many rows, or all rows. All rows are updated if the WHERE clause predicates evaluate to true for all rows, or there is no WHERE clause present.

Order is not important. In order to be able to assure that exactly the same updates occur for each set of rows regardless of the order in which the columns are accessed, certain rules must apply. ***Whenever the database encounters a column on the right side of an assignment; the original value of the column (prior to any changes) is used to calculate the new value of the target column.***

As with INSERT, the database performs all maintenance associated with indexes defined on the table. UPDATES require a delete of the old value followed by an insert of the new value into each affected index. This extra processing must be considered for performance estimates, especially in on-line systems.

UPDATE with SELECT

```
UPDATE    EMP
SET       (SALARY, BONUS) =
          (SELECT SALARY, BONUS
           FROM EMP
           WHERE EMPNO = '000100')
WHERE     EMPNO = '000200'
```

Oracle Db2

UPDATE with SELECT

As shown in the syntax diagram and examples on the facing page, UPDATE can contain SELECT statements in the SET clause. (SELECT statements in the WHERE clause are covered in chapter 8).

In this example, the salary and bonus for employee 000200 are being changed to the salary and bonus of employee 000100. A SELECT statement is being used to return (as a table) the salary and bonus for employee 000100.

This example also uses a row-value expression. This feature is a shorthand for using two scalar fullselects; i.e. one to set the SALARY column, as in the first example, and a similar one to set the BONUS column. Both SALARY and BONUS represented by a row-value expression are set to the result table of the SELECT that returns the SALARY and BONUS columns of employee 000200. The row value expression is only valid in Oracle and Db2.

MERGE with Subselect

```
MERGE INTO NAMES N
USING (SELECT EMPNO, LASTNAME, FIRSTNME
      FROM EMP
      WHERE DEPTNO LIKE 'D%') E
ON (N.EMPNO = E.EMPNO)
WHEN MATCHED THEN
  UPDATE SET N.LASTNAME = E.LASTNAME,
            N.FIRSTNME = E.FIRSTNME
WHEN NOT MATCHED THEN
  INSERT (N.EMPNO, N.LASTNAME, N.FIRSTNME)
  VALUES (E.EMPNO, E.LASTNAME, E.FIRSTNME)
```

Merge Statement

Merge with Subselect

Combining insert and update processing is possible in Oracle and Db2 using a SQL statement called *MERGE*. This is sometimes referred to as an “upsert”. An execution of a *MERGE* statement will supply a set of values for a row (or rows) and a set of criteria for determining if the rows are already present in the table. If the row is already present, an update may be performed. If the row is not present, an insert may be coded. This simplifies coding by allowing all these functions to be performed in a single database call.

This example shows rows to be merged into a table called NAMES. The new rows come from a subselect that retrieves some columns and rows from the EMP table. The values on NAMES will either be inserted or updated depending on whether the EMPNO is already present.

This syntax is supported on SQL Server, Oracle and the Linux, Unix and Windows platform of Db2.

MERGE with Values

```
MERGE INTO ITEM I
  USING (VALUES (1, 'WIDGIT'))
    AS NEWITEM (ITEMNO, ITEMNAME)
  ON I.ITEMNO = NEWITEM.ITEMNO
  WHEN MATCHED THEN
    UPDATE SET ITEMNAME = NEWITEM.ITEMNAME
  WHEN NOT MATCHED THEN
    INSERT (ITEMNO, ITEMNAME)
      VALUES (NEWITEM.ITEMNO, NEWITEM.ITEMNAME)
```

Db2 **SQL Server**

MERGE with Values

SQL Server and Db2 (on all platforms) also support the use of specified values in the place of a subselect. These values may be literals or host variables / parameter markers in a program.

MERGE with Values

```
MERGE INTO ITEM I  
  USING (SELECT 1 AS ITEMNO,  
          'WIDGIT' AS ITEMNAME  
        FROM DUAL) NEWITEM  
  ON I.ITEMNO = NEWITEM.ITEMNO  
 WHEN MATCHED THEN  
   UPDATE SET ITEMNAME = NEWITEM.ITEMNAME  
 WHEN NOT MATCHED THEN  
   INSERT (ITEMNO,ITEMNAME)  
   VALUES (NEWITEM.ITEMNO, NEWITEM.ITEMNAME)
```

 Themis
Leaders in IT Education

Oracle

Oracle does not handle the USING with VALUES clause, so you have to hard code the values using the DUAL dummy table.

PostgreSQL MERGE Uses ON CONFLICT

```
INSERT INTO ITEM (ITEMNO, ITEMNAME)
VALUES (1, 'WIDGET')
ON CONFLICT (ITEMNO)
DO UPDATE SET ITEMNAME = 'WIDGET'
;
```

PostgreSQL

 Themis
Leaders in IT Education

PostgreSQL MERGE Uses ON CONFLICT

PostgreSQL does not include the MERGE statement, but has similar logic using the INSERT with the ON CONFLICT conditions. This is called UPSERTing in PostgreSQL.

Can also do: ON CONFLICT NO NOTHING

This statement will first try to insert a row based on the values, and if the value exists (meaning there was a conflict) then execute the update logic.

You could also add in a SELECT and process multiple rows. If there was a PROJ2 table with rows that is needed to merge with the PROJ table, you could code the following:

```
INSERT INTO PROJ2 (PROJNO, PROJNAME, EMPNO, DEPTNO)
SELECT PROJNO, PROJNAME, EMPNO, DEPTNO FROM PROJ
ON CONFLICT (PROJNO) DO UPDATE SET PROJNAME =EXCLUDED.PROJNAME;
```

Note: When doing upserts in PostgreSQL 9.5+ you must refer to the excluded data (that which failed to insert) by the alias **EXCLUDED**.

DELETE

```
DELETE FROM PROJ  
WHERE PROJNO = 'OP1000'
```

One Row

```
DELETE FROM PROJ  
WHERE PRENDATE < '2010-02-01'
```

Many Rows

```
DELETE FROM PROJ
```

ALL Rows!

DELETE

The DELETE statement is simpler than INSERT or UPDATE. This is because DELETE is not column-oriented; entire rows are deleted.

The WHERE clause qualifies the rows to be deleted. This can affect zero rows, one row, many rows, or all rows.

All rows are deleted if the WHERE clause predicates evaluate to true for all rows, or there is no WHERE clause present. This is referred to as a “mass delete” and is very efficient on a segmented or universal tablespace, since logging of the deleted rows is eliminated.

As with UPDATE, the term searched DELETE is used when the DELETE contains a WHERE clause.

SQL TRUNCATE Table

TRUNCATE TABLE xxxxxxxxxxxx

- Quickly removes all data in a table
- Same Function as DELETE with no WHERE logic
- No WHERE logic allowed (all or nothing)
- Reduces overhead of logging
- Typically faster than DELETE FROM
- Cannot TRUNCATE when a foreign key references the table to be truncated
- Great especially when there is no need to worry about rolling back rows
- Options: Depends on the RDBMS

SQL TRUNCATE Table

TRUNCATE is a statement used to empty a table. In Db2, TRUNCATE works very similarly to a DELETE with no WHERE clause and has some additional options depending on the RDBMS. No WHERE clause can be coded.

Removing rows with the TRUNCATE TABLE statement can be more efficient than dropping and re-creating a table. Dropping and re-creating a table invalidates dependent objects of the table, requires you to re-grant object privileges on the table, and requires you to re-create the indexes, integrity constraints, and triggers on the table and re-specify its storage parameters. Truncating has none of these effects.

Removing rows with the TRUNCATE TABLE statement can be faster than removing all rows with the DELETE statement, especially if the table has numerous triggers, indexes, and other dependencies.

DROP and TRUNCATE are DDL commands, whereas DELETE is a DML command. Therefore DELETE operations can be rolled back (undone), while DROP and TRUNCATE operations cannot be rolled back.

TRUNCATE – Db2

TRUNCATE TABLE PROJ

**DROP STORAGE / REUSE STORAGE
IGNORE DELETE TRIGGERS or
RESTRICT WHEN DELETE TRIGGERS
CONTINUE IDENTITY
IMMEDIATE**

*Db2 LUW
And z/OS
Options*

TRUNCATE Db2 LUW

TRUNCATE is a statement used to empty a table. In Db2, TRUNCATE works very similarly to a DELETE with no WHERE clause with some additional options:

DROP STORAGE: All storage allocated for the table is released and made available.

REUSE STORAGE: All storage allocated for the table will continue to be allocated for the table, but the storage will be considered empty.

IGNORE DELETE TRIGGERS: Any delete triggers that are defined for the table are not activated by the truncate.

RESTRICT WHEN DELETE TRIGGERS: An error is returned if delete triggers are defined on the table (SQLSTATE 428GJ).

CONTINUE IDENTITY: If an identity column exists for the table, the next identity column value generated continues with the next available value.

IMMEDIATE: Specifies that the truncate operation is processed immediately and cannot be undone (by any ROLLBACK). If IMMEDIATE is not specified, a ROLLBACK statement *can* undo the truncate operation.

TRUNCATE – ORACLE

TRUNCATE TABLE PROJ

**PRESERVE/PURGE MATERIALIZED
VIEW LOG
DROP STORAGE / REUSE STORAGE**

*ORACLE
Options*

TRUNCATE - Oracle

Removing rows with the TRUNCATE TABLE statement can be more efficient than dropping and re-creating a table. Dropping and re-creating a table invalidates dependent objects of the table, requires you to re-grant object privileges on the table, and requires you to re-create the indexes, integrity constraints, and triggers on the table and re-specify its storage parameters. Truncating has none of these effects. In Oracle, the high water mark of the table is reset, rendering the table as empty very quickly.

Removing rows with the TRUNCATE TABLE statement can be faster than removing all rows with the DELETE statement, especially if the table has numerous triggers, indexes, and other dependencies. Indexes, triggers, and dependencies are not affected.

The MATERIALIZED VIEW LOG clause lets you specify whether a materialized view log defined on the table is to be preserved or purged when the table is truncated.

PRESERVE: Specify PRESERVE if any materialized view log should be preserved when the master table is truncated. This is the default.

PURGE: Specify PURGE if any materialized view log should be purged when the master table is truncated.

DROP STORAGE: All storage allocated for the table is released and made available.

REUSE STORAGE: All storage allocated for the table will continue to be allocated for the table, but the storage will be considered empty.

TRUNCATE – SQL Server

**TRUNCATE TABLE PROJ
WITH PARTITIONS**

*SQL
Server
Options*

TRUNCATE – SQL Server

For example: `WITH (PARTITIONS (1, 2, 6 TO 8))` syntax causes partition numbers 1, 2, 6, 7, and 8 to be truncated.

In SQL Server:

TRUNCATE has the same functionality as a DELETE with no WHERE clause without the overhead of logging.

The counters on any identity columns will be reset.

When a table is truncated, the row deletions are not logged which is why rolling back is not possible.

Pages of data are deallocated pages and returned to the system for use in other areas

A TRUNCATE TABLE statement can be rolled back in SQL Server by using a transaction.

```
EX) BEGIN TRAN;  
    TRUNCATE TABLE dbo.xxxxxxx;  
    ROLLBACK;
```

PostgreSQL TRUNCATE

```
TRUNCATE [ TABLE ] [ ONLY ] name [ * ] [, ... ]  
[ RESTART IDENTITY | CONTINUE IDENTITY ]  
[ CASCADE | RESTRICT ]
```

*PostgreSQL
Options*

PostgreSQL TRUNCATE

PostgreSQL also has a TRUNCATE statement to empty a table, but with some other options.

Allows to not only remove all data from a table but also reset associated sequence generator by specifying RESTART IDENTITY option. By default, it will not reset the value.

➔ TRUNCATE TABLE table_name RESTART IDENTITY;

To remove data from the main table and all tables that have foreign key references to the main table, you use CASCADE option. By default, the TRUNCATE TABLE statement does not remove any data from the table that has foreign key references. To remove data from the main table and all tables that have foreign key references to the main table, you use CASCADE option. ➔ TRUNCATE TABLE table_name CASCADE;

Even though the TRUNCATE TABLE statement removes all data from a table, it does not fire any ON DELETE TRIGGERS associated with that table. To fire the trigger when the TRUNCATE TABLE command applied to a table, you must define BEFORE TRUNCATE and/or AFTER TRUNCATE triggers for that table.

The TRUNCATE TABLE can be transaction-safe. If you place it within the transaction statements such as BEGIN ... ROLLBACK, the truncation operation will be rolled back safely.

Db2 SELECT FROM

```
SELECT HIREDATE  
FROM FINAL TABLE  
(INSERT INTO EMP (EMPNO, LASTNAME, HIREDATE)  
VALUES ('999000', 'Caccavale', CURRENT DATE)  
)
```

```
SELECT EMPNO  
FROM OLD TABLE  
(DELETE FROM EMP WHERE JOB = 'CLERK')
```

Db2

Db2 SELECT FROM

In Db2, data modification may be performed inside the same statement that returns one or more of the modified values. In this example, the CURRENT DATE special register is being used to populate the HIREDATE column. If the user needs to know the value of the inserted date, they may SELECT from the INSERT using the FINAL TABLE keyword. The insert is performed and the resulting HIREDATE is returned as a result set to the caller.

FINAL TABLE indicates that values should be returned as they appear AFTER the modification. If a SELECT FROM DELETE is performed, the FINAL TABLE keyword is changed to OLD TABLE (show the rows as they looked before the delete). SELECT FROM UPDATE statements may use either FINAL TABLE or OLD TABLE.

SELECT FROM MERGE

```
SELECT ITEMNAME, UPD_IND FROM FINAL TABLE
```

```
(MERGE INTO ITEM I
```

```
  INCLUDE (UPD_IND CHAR(1))
```

```
  USING (VALUES (1, 'SOCKET') )
```

```
    AS NEWITEM (ITEMNO, ITEMNAME)
```

```
  ON I.ITEMNO = NEWITEM.ITEMNO
```

```
  WHEN MATCHED THEN
```

```
    UPDATE SET ITEMNAME = NEWITEM.ITEMNAME,
```

```
      UPD_IND = 'U'
```

```
  WHEN NOT MATCHED THEN
```

```
    INSERT (ITEMNO,ITEMNAME,UPD_IND)
```

```
      VALUES (NEWITEM.ITEMNO, NEWITEM.ITEMNAME,'I') )
```

**Db2
z/OS
Only**

SELECT FROM MERGE

When selecting from a MERGE statement, it may be desirable to determine whether an INSERT or UPDATE operation was performed. This may be accomplished by using the INCLUDE clause to create an indicator variable. In the example shown, an INCLUDE column is created called UPD_IND. When an update is performed, the UPD_IND is set to a "U". When an insert is performed, the UPD_IND is set to an "I". This value may be returned to the application via the SELECT.

This works on mainframe Db2 z/OS only.

Unit of Work & Transaction Processing

- A unit of work is a group of SQL statements that modify table data where all the changes are either committed or rolled back (undone) together.
- A transaction is a logical atomic unit of work.
- A transaction begins when the SET TRANSACTION statement (Oracle) or BEGIN TRANSACTION (SQL Server) is executed.
- A transaction begins implicitly when:
 - A statement that modifies data is executed or
 - SELECT ... FOR UPDATE statement is executed



Unit of Work & Transaction Processing

Applications that perform data modification will need to handle transactions or units of work. These are groups of statements that will need to succeed or fail together as a group. Statements should be grouped logically so that the database is always left in a consistent state even if the transaction fails.

A transaction is the propagation of one or more changes to the database. They are a fundamental concept of all database systems within programs and/or SQL scripts. The essential point of a transaction is that it bundles multiple queries into a single, all-or-nothing operation. So, within a transaction, the COMMIT affects all the queries, as does the ROLLBACK.

Db2 Commit

```
DELETE FROM CUST
WHERE CUSTNO = 4;

INSERT INTO CUST (CUSTNO, CUSTNAME)
VALUES (321, 'Themis Inc.');
```



```
INSERT INTO CUST_ORDER
(ORD_NUM, CUST_NUM, PRODUCT)
VALUES (999,321, 'SQL Class');
```


COMMIT WORK;

Db2

*Release
Locks!*

Db2 Commit

Here is an example of several statements in an application. It has been determined that these statements should stand together. When the group is complete a COMMIT is performed. In Db2, this may be expressed either as “COMMIT WORK” or simply “COMMIT”. After the commit the work done by the statements is marked as permanent and all locks taken by the statements are released.

Db2 Rollback

```
DELETE FROM CUST
WHERE CUSTNO = 4;

COMMIT;

INSERT INTO CUST (CUSTNO, CUSTNAME)
VALUES (321, 'Themis Inc.');
```



```
INSERT INTO CUST_ORDER
(ORD_NUM, CUST_NUM, PRODUCT)
VALUES (999,321, 'SQL Class');
```

ROLLBACK WORK;

Db2 Themis
Leaders in IT Education

Rollback does just what it states. It rolls back any modifications that have taken place. In this example a ROLLBACK would undo the delete and both inserts.

Db2 Rollback

In Db2 a ROLLBACK statement will undo the work done by all statements since the last commit point. You can also set SAVEPOINTS to roll back to.

SAVEPOINT

ON ROLLBACK RETAIN CURSORS.

ON ROLLBACK RETAIN CURSORS means not to close cursors opened after establishing a savepoint and issuing a rollback.

ON ROLLBACK RETAIN LOCKS means that any locks acquired after establishing the savepoint are not released upon a rollback to a savepoint.

SQL Server Transaction

BEGIN TRAN;

```
DELETE FROM CUST  
WHERE CUSTNO = 4;
```

```
INSERT INTO CUST (CUSTNO, CUSTNAME)  
VALUES (321, 'Themis Inc.');
```

```
INSERT INTO CUST_ORDER  
(ORD_NUM, CUST_NUM, PRODUCT)  
VALUES (999,321, 'SQL Class');
```

COMMIT TRAN;

SQL Server

SQL Server Transaction

SQL Server has statements to begin, commit and rollback transactions. Transactions may optionally be named and nested.

Oracle Transaction

```
DELETE FROM CUST  
WHERE CUSTNO = 4;
```

```
INSERT INTO CUST (CUSTNO, CUSTNAME)  
VALUES (321, 'Themis Inc.');
```

```
INSERT INTO CUST_ORDER  
(ORD_NUM, CUST_NUM, PRODUCT)  
VALUES (999,321, 'SQL Class');
```

```
COMMIT;
```

Oracle Transaction

The commit and rollback statements work as described on the other platforms. You can also code SAVEPOINT:

```
UPDATE....  
SAVEPOINT sp1;  
DELETE....  
SAVEPOINT sp2;  
UPDATE....
```

```
ROLLBACK TO SAVEPOINT sp1;
```

This would leave the first UPDATE still pending waiting for a commit or rollback.

Oracle Transaction

SET TRANSACTION READ ONLY NAME 'TRAN1';

**SELECT
FROM;**

**SELECT SUM
FROM;**

COMMIT;

You use the SET TRANSACTION statement to begin a read-only or read-write transaction, establish an isolation level, or assign your current transaction to a specified rollback segment. Read-only transactions are useful for running multiple queries while other users update the same tables, and not have the chance of getting locked out.

If you set a transaction to READ ONLY, subsequent queries within the transaction see only changes committed before the transaction began. The use of READ ONLY does not affect other users or transactions. Very common in PL/SQL code as follows:

BEGIN

```
SET TRANSACTION READ ONLY;
FOR EMP IN (SELECT LASTNAME FROM EMP WHERE DEPTNO = 'D11')
LOOP
    SET ..... ➔ processing logic for each row;
END LOOP;
```

```
FOR DEPT IN (SELECT DEPTNO, DEPTNAME FROM DEPT WHERE LOC = 'NY')
LOOP
    SET ..... ➔ processing logic for each row;    END LOOP;
END;
```

PostgreSQL Transaction

BEGIN TRANSACTION; → OR JUST BEGIN

```
DELETE FROM CUST  
WHERE CUSTNO = 4;
```

```
INSERT INTO CUST (CUSTNO, CUSTNAME)  
VALUES (321, 'Themis Inc.');
```

```
INSERT INTO CUST_ORDER  
(ORD_NUM, CUST_NUM, PRODUCT)  
VALUES (999,321, 'SQL Class');
```

COMMIT TRANSACTION; → OR JUST COMMIT

PostgreSQL

 Themis
Leaders in IT Education

PostgreSQL Transaction

PostgreSQL has statements to begin a transaction, commit and rollback actions within. Transactions have the following four standard properties, usually referred to by the acronym → ACID:

Atomicity – Ensures that all operations within the work unit are completed successfully; otherwise, the transaction is aborted at the point of failure and previous operations are rolled back to their former state.

Consistency – Ensures that the database properly changes states upon a successfully committed transaction.

Isolation – Enables transactions to operate independently of and transparent to each other.

Durability – Ensures that the result or effect of a committed transaction persists in case of a system failure.

Constraints

- Primary Key
- Unique (non primary key)
- Foreign Key
- Table Check Constraint



Constraints

Data modification statements may encounter errors when business rules that have been defined in the database are violated. Applications that perform data modification may need to be coded to deal with these errors. The messages are platform specific.

Primary and unique keys may be defined on a table. These constraints prohibit a duplicate row. When an insert or update is attempted that would duplicate another row with the same key values, an error is returned.

Foreign keys govern the relationship a table has with another table. All rows in a child table must have a valid row in the parent. The database will return an error whenever a statement is issued that would result in an orphan situation.

Table check constraints help further define the domain of a column on a table. They are rules that may be defined in the database that limit the possible set of values that may be placed in a column. Errors result when a modification is made that would violate one of these rules.

This Page Intentionally Left Blank

Chapter 8

SQL Subqueries

Table of Contents

CHAPTER 8 SQL SUBQUERIES8-1

Overview.....8-3

 Objectives 8-3

Why Subqueries?8-5

Non Correlated Subqueries8-7

 Non Correlated Example 8-9

Correlated Subqueries8-10

 Simple Correlated Example8-11

 Existence Check.....8-15

 Example Differences8-20

Scalar Fullselects.....8-21

 Example #18-22

 Example #28-23

Overview

Objectives

After successful completion of this section the student should be able to perform the following:

- ◆ Provide understanding of Non Correlated Subqueries
- ◆ Provide understanding of Correlated Subqueries
- ◆ Provide understanding of Exists vs In
- ◆ Provide understanding of Not Exists vs Not In
- ◆ Provide understanding of using Subqueries in place of Distinct

Why Subqueries?

- Many times it's the only way to get desired results
- Can be used in place of distinct
- Can be used in tuning to rewrite a query a different way and still get the same result

Why Subqueries?

Many times, adding a subquery to an SQL statement is the only way to get the desired results. Other times they can be used to rewrite SQL statements for better performance.

Subqueries can be written one of 2 ways, correlated and non-correlated.

- ◆ Each one has its place and purpose.
- ◆ Each one operates very differently.
- ◆ At times, either one can be used.
- ◆ At times, can be used in place of Distinct.

Non Correlated Subqueries

- Non correlated subqueries make no reference to the outer table(s).
- They execute once only, and pass the result(s) to the outer SELECT.
- The results are always sorted for ordering and uniqueness.
- The SELECT statement that surrounds the subquery is the *outer SELECT*.

Non Correlated Subqueries

SQL statements containing non-correlated subqueries always start the processing at the subquery. The subquery gets executed and passes the result set up to the outer query, then the outer query executes. It is a 2 step process. If the subquery is building a list due to an IN statement, then that list will get sorted in ascending order before passing the results to the outer query. For example:

```
SELECT * FROM EMP
WHERE JOB = 'DESIGNER'
AND DEPTNO IN
  (SELECT DEPTNO
   FROM PROJ
   WHERE MAJPROJ = 'MA2100');
```

In this SQL statement, if the subquery finds many DEPTNO values based on the logic inside the subquery, then that list of values will first be sorted and then the list generated for the outer query. There are two reasons for the sort:

- ◆ To get the list of values in ascending order
- ◆ To eliminate any duplicate values

Having a sorted de-duped list of values improves the performance of the outer query.

Non Correlated Example

Provide a report of employees whose education levels are higher than the average education level of all employees.

```
SELECT EMPNO,  
       LASTNAME,  
       DEPTNO,  
       EDLEVEL  
FROM EMP E1  
WHERE E1.EDLEVEL >  
      (SELECT AVG(E2.EDLEVEL)  
       FROM EMP E2 )
```

Outer
Select

Subquery

Note: Subquery here has no reference
to the table in the Outer Select.

Non Correlated Example

This is very simple request to list all employees that make more than the average salary across all employees. Because the average salary is not tracked on any table, it must be calculated. This is done with a subquery.

Correlated Subqueries

- Correlated subqueries make a reference to an outer table via a join predicate.
- The RDMS will re-execute (re-evaluate) the subquery when it examines a new row (in the outer WHERE clause), or a group of rows (in the outer HAVING clause) as it is executing the outer Select.
- The processing always starts at the outer query, and for each row retrieved, passes the value to the subquery to execute.
- Unlike a Non Correlated subquery, a Correlated subquery gets executed many times during its unit of work.



Correlated Subqueries

SQL statements containing Correlated Subqueries always have a join predicate in the subquery, joining a column from one of its tables to a table in the outer query.

The subquery 'EXISTS' and 'NOT EXISTS' are usually correlated because they tend to need a row by row comparison.

Simple Correlated Example

Provide a report of employees whose education levels are higher than the average education level of their respective department.

Example: EMPNO '000010' works in department 'A00'. Is this employee's EDLEVEL > the average of all employees in 'A00'. If so, send their information to the result set.

Simple Correlated Example

This example will require a correlated subquery since each individual piece of data (an employee's EDLEVEL) will need to be compared to an aggregate piece of data (the average EDLEVEL) where the aggregate is determined by information from the individual (the average for the department the employee works in).

Simple Correlated Example

```
SELECT EMPNO,  
       LASTNAME,  
       DEPTNO,  
       EDLEVEL  
FROM EMP E1  
WHERE E1.EDLEVEL >  
      (SELECT AVG(E2.EDLEVEL)  
       FROM EMP E2  
       WHERE E2.DEPTNO = E1.DEPTNO)
```

Join predicate in
subquery!

Simple Correlated Example

Here is the SQL needed in order to find those employees that make more than the average salary within their own department. In order for this to work correctly, a join on the DEPTNO column is needed in the subquery.

Note: When coding correlated subqueries, it is especially important to code correlations identifiers on the tables from both the outer query and the subquery. It makes for much easier reading and understanding, and alleviates SQL warnings on the join in the subquery.

What the RDMS does:

A **correlated subquery** is evaluated for each qualified row of the outer query that is referred to. In executing the example, the RDMS:

1. Reads a row from table EMP (will process every since there is no WHERE logic)
2. Searches for the value of DEPTNO from that row, in a table stored in memory.
3. The in-memory table saves executions of the subquery. If the subquery has already been executed with the value of DEPTNO, the result of the subquery is in the table and the RDMS does not execute it again for the current row. Instead, the RDMS can skip to step 5.
4. Executes the subquery, if the value of DEPTNO is not in memory. That requires searching the EMP table to check whether there is any DEPTNO for the values being passed from the outer query.
5. Stores the value of DEPTNO and the AVG(EDLEVEL) result of the subquery in memory.
6. Returns the values of the current row of EMP to the application.

The RDMS repeats this whole process for each qualified row of the EMP table.

Note: Since many employees will work within the same DEPTNO, the subquery will only execute once for each unique DEPTNO, storing the AVG(EDLEVEL) value in memory. Since there are 8 different DEPTNO values across the EMP table, then for this example, the subquery will be executed 8 times.

Existence Check

Provide a report of employees who work on Project 'IF2000'.
Show Employee Number, Last Name, and Salary.

Tables Needed:

- EMP Employee table
- EMPPROJECT Employee/Project/Activity

Existence Check

This request for a report is a good example of where a subquery can be used in place of Distinct. This report requires processing from both the EMP table, and the EMPPROJACT table.

In the EMPPROJACT, an employee can have multiple rows because of the different activities associated with a project. For example:

EMPNO	PROJNO	ACTNO	EMPTIME	EMSTDATE	EMENDATE
000030	IF1000	10	0.5	06/01/1982	01/01/1983
000130	IF1000	90	1.00	01/01/1982	10/01/1982
000130	IF1000	100	0.5	10/01/1982	01/01/1983
000140	IF1000	90	0.5	10/01/1982	01/01/1983
000030	IF2000	10	0.5	01/01/1982	01/01/1983
000140	IF2000	100	0.5	03/01/1982	07/01/1982
000140	IF2000	100	1.00	01/01/1982	03/01/1982
000140	IF2000	110	0.5	10/01/1982	01/01/1983
000140	IF2000	110	0.5	03/01/1982	07/01/1982

Existence Check

If we try a join, we get the following results:

```
SELECT E.EMPNO, E.LASTNAME, E.SALARY
FROM EMP      E,
     EMPPROJECT EP
WHERE E.EMPNO = EP.EMPNO
     AND EP.PROJNO = 'IF2000'
```

EMPNO	LASTNAME	SALARY
000030	KWAN	38250.00
000140	NICHOLS	28420.00
000140	NICHOLS	28420.00
000140	NICHOLS	28420.00
000140	NICHOLS	28420.00

Duplicates because this employee works on multiple activities associated with project 'IF2000'

Existence Check

The first thought to fulfill this request would be to code and execute an SQL Join between the 2 tables, zeroing in on the employees that are a part of Project = 'IF2000'.

As can be seen, when a join is coded there are duplicates that show up due to the multiple activities some employees are assigned for that project.

What can be done to eliminate the duplicates?

Existence Check

```
SELECT DISTINCT E.EMPNO, E.LASTNAME, E.SALARY
FROM EMP      E,
     EMPPROJECT EP
WHERE E.EMPNO = EP.EMPNO
      AND EP.PROJNO = 'IF2000'
```

EMPNO	LASTNAME	SALARY
000030	KWAN	38250.00
000140	NICHOLS	28420.00

Add a DISTINCT?

Existence Check

The easiest and first thought most developers think of is to add the DISTINCT keyword as part of the select, which will eliminate the duplicates.

In order for the RDMS to eliminate duplicates via the DISTINCT, it loads the data into a workfile, and executes a sort in order to get the data ordered. There is overhead involved in this sort. A workfile must be allocated, then loaded, then sorted, and from there unique values are pulled for the final result set.

The same solution to eliminate duplicate values can also be done via a subquery, and not the Distinct.

Existence Check

```
SELECT E.EMPNO, E.LASTNAME, E.SALARY
FROM EMP E
WHERE E.EMPNO IN
  (SELECT EP.EMPNO
   FROM EMPPROJECT EP
   WHERE EP.PROJNO = 'IF2000' )
```

EMPNO	LASTNAME	SALARY
000030	KWAN	38250.00
000140	NICHOLS	28420.00

**OR: Code a Non
Correlated Subquery to
eliminate duplicates**

Existence Check

There are 2 different ways to eliminate the duplicates needed for this example, both ways requiring a subquery.

The first way is to code a correlated subquery, as shown on the opposite page.

This query is checking to see if each EMPNO value is in the list of EMPNOs generated by the subquery. An EMPNO could be in the list multiple times, but the results will not show the EMPNO multiple times. Remember that the list of values coming out of a non-correlated subquery will have its results sorted in order to eliminate duplicates in the list, and to get the list in ascending order.

If a Distinct causes a sort, and the non-correlated subquery causes a sort, which one is better?

Existence Check

```
SELECT E.EMPNO, E.LASTNAME, E.SALARY
FROM EMP E
WHERE EXISTS
  (SELECT 1
   FROM EMPPROJECT EP
   WHERE E.EMPNO = EP.EMPNO
        AND EP.PROJNO = 'IF2000' )
```

EMPNO	LASTNAME	SALARY
000030	KWAN	38250.00
000140	NICHOLS	28420.00

OR: Code a Correlated Subquery to eliminate duplicates

Existence Check

The other way to eliminate duplicates is by coding the SQL using a correlated subquery with the Exists clause.

The logic here is as each EMPNO value is passed to the subquery for execution, the question “Does that join condition exist for the particular EMPNO value?” is processed. Even if it exists multiple times in the subquery, the value will still only get written out once.

Writing it this way eliminates sorting, but the subquery will get executed multiple times.

Which way is best ?

1. Using the Distinct
2. Writing a subquery using the ‘In’
3. Writing a subquery using the ‘Exists’

Example Differences

- Distinct. By coding the distinct, the RDMS will sort the final result set to eliminate duplicates.
- Non Correlated Subquery. The RDMS will also do a sort, but only of the results that come out of the subquery (in this example, EMPNO only).
- Correlated Subquery. In this example, the subquery will get executed for every row from the outer Select to see if that EMPNO exists in the EMPPROJECT table. It gets executed for every row because there is no WHERE logic other than EXISTS which would eliminate rows from being checked.



Example Differences

This screen sums up 3 different processes to eliminate duplicates for the example given. Which one is best? It depends:

How big is the Distinct sort? The larger the size of sorts the more expensive the query and runtime. This is typically the worst.

Non Correlated Subquery

This is typically better because the sort is smaller than the Distinct sort. The sort is done on only one column for however many values come out of the subquery.

Correlated Subquery

This subquery will execute multiple times so it is imperative that the subquery uses a index when it gets processed for each value sent to it via the join. If the subquery is execute as 'Index Only' and there is matching index occurring, executing many times can be very fast and efficient.

So the answer is 'It Depends'. The nice thing about these is that we have options for duplicate data at times and they all execute very different within DB2. Depending on the processing, one will usually run better than the others.

Scalar Fullselects

- Any SELECT that returns a single value (one column, one row) may be used as an expression in the SELECT or WHERE clause.
- Allows more complex operations to be completed in a single step.

Scalar Fullselects

A scalar fullselect is a fullselect that returns at most one column and one row or alternatively, an empty result.

When enclosed in parenthesis and specified within an expression, a scalar fullselect can return an atomic value or null. If more than one value is qualified, an error is encountered and no value is returned. If the scalar fullselect returns zero rows, its result is interpreted as null (no error results in this situation unless the context where the scalar fullselect appears does not permit nulls).

Scalar Fullselect Example #1

List all departments from the department table along with the maximum salary for that department.

```
SELECT  D.DEPTNAME
        ,D.LOCATION
        ,(SELECT MAX(SALARY)
           FROM EMP E
           WHERE D.DEPTNO = E.DEPTNO)AS MAXSAL
FROM DEPT D
```

Example #1

This example shows a scalar fullselect appearing in a select-list. The result will include the maximum salary (determined by querying the employee table) for the departments with additional columns being retrieved from the department table. Empty departments will also appear with the maximum salary column set to null. The following example is an outer join solution that produces the same result but with a higher degree of complexity:

```
SELECT DEPTNO, DEPTNAME, MAX(SALARY) AS MAXPAY
FROM   DEPT AS D
       LEFT OUTER JOIN
       EMP  AS E
       ON D.DEPTNO = E.DEPTNO
GROUP BY DEPTNO, DEPTNAME
```

Scalar Fullselect Example #2

List all departments where the average employee bonus is greater than the average employee salary.

```
SELECT D.DEPTNAME  
       ,D.LOCATION  
FROM DEPT D  
WHERE (SELECT AVG(BONUS) FROM EMP E1  
       WHERE D.DEPTNO = E1.DEPTNO)  
       >  
       (SELECT AVG(SALARY) FROM EMP E2  
       WHERE D.DEPTNO = E2.DEPTNO)
```

Example #2

This example shows scalar fullselects appearing in a search condition. The query returns all departments where the average employee bonus exceeds the average employee salary for the department. An alternative solution looks like this:

```
SELECT DEPTNAME, D.LOCATION  
FROM DEPT AS D  
     INNER JOIN  
     EMP AS E  
     ON D.DEPTNO = E.DEPTNO  
GROUP BY DEPTNAME, D.LOCATION  
HAVING AVG(BONUS) >  
       (SELECT AVG(SALARY)  
        FROM EMPLOYEE  
        WHERE E.DEPTNO = D.DEPTNO)
```

This Page Intentionally Left Blank